BUKIDNON STATE UNIVERSITY

COLLEGE OF TECHNOLOGIES

INFORMATION TECHNOLOGY DEPARTMENT

BukSU Fitness Gym - Digital Logging System

A Project Presented to

Navidad, Czarissa Louise A.

Caseres, Klevie Jun R.

Cagande, Gil Nicholas T.

Abella, Joseph L.

of the Information Technology Department

In Partial Fulfillment of the Requirement for the Courses

IT133 – Systems Integration and Architecture (SIA) 1

IT137 Elective 3 – Integrative Programming and Technologies 2

IT132 – Advanced Database Systems

IT135 – Information Assurance and Security 1

By:

Neil Marc Bayron

Rienier Marl S. Borong

Ivan Lyod Panoril

Senen Jeremiah Patricio

December 13, 2024

Chapter 1
INTRODUCTION

1.1. Background of the Organization

1.1.1. History

Bukidnon State University's Fitness Gym officially began operations on September 20, 2023, marking a significant step towards promoting health and wellness within the campus community. Designed to cater exclusively to all bona fide students, instructors, and personnel of the university, this facility provides a convenient and accessible venue for physical exercise and fitness activities. Located beside the Valley Stairs, directly in front of the Rubia Dormitory, the gym ensures easy access for its users. It operates during weekdays from 5:00 PM to 6:30 PM, aligning with the schedules of both students and faculty, making it a practical option for after-class or after-work physical activities.

The gym emphasizes not only the importance of regular physical activity but also the need to maintain safety and care for equipment. To achieve this, it strictly enforces the use of proper workout attire, including gloves, to minimize risks of injury and to preserve the quality and longevity of the gym's facilities. Adhering to safety protocols and operational limits, the fitness gym accommodates a specific number of users at a time and follows a "first come, first served" policy to manage attendance fairly and efficiently.

Under the supervision of PE Instructor and Fitness Gym In-Charge, Mr. Tom Cruz, the facility is dedicated to fulfilling its primary purpose: to serve as a supportive environment where students and faculty members can enhance their physical health and overall well-being. The gym is equipped with a variety of fitness tools and machines, ensuring that users of all fitness levels can find suitable exercises to meet their goals.

BukSUans are encouraged to take advantage of this new amenity as a step toward fostering a healthier, more active lifestyle. Whether they are looking to maintain their fitness, relieve stress, or simply explore the benefits of exercise, the BukSU Fitness Gym offers an ideal space for personal growth and wellness development within the supportive framework of the university community.

1.1.2. Nature of Operations

The BukSU Fitness Gym operates as a campus based fitness facility dedicated to promote health and wellness among faculty and students of Bukidnon State University. Primary objectives include encouraging physical fitness, healthy lifestyle, and creating an inclusive environment where the campus community can achieve their fitness goals in the gym.

Membership Access: Open to all bonafide Faculty and Students of Bukidnon State University.

Operating Hours: The gym follows set hours that align with the campus schedule.

Facilities and Equipment: Equipped with exercise machines, free weights, cardio equipment, and fitness tools to cater to diverse workout preferences.

Rules and Guidelines: Enforces cleanliness, proper use of equipment, and respectful behavior among members. Ensuring safety protocols are followed during gym use.

Integration with Campus Initiatives: Collaborates with BukSU's wellness programs and events to align with the university's mission of comprehensive development.

1.1.3. Demographics

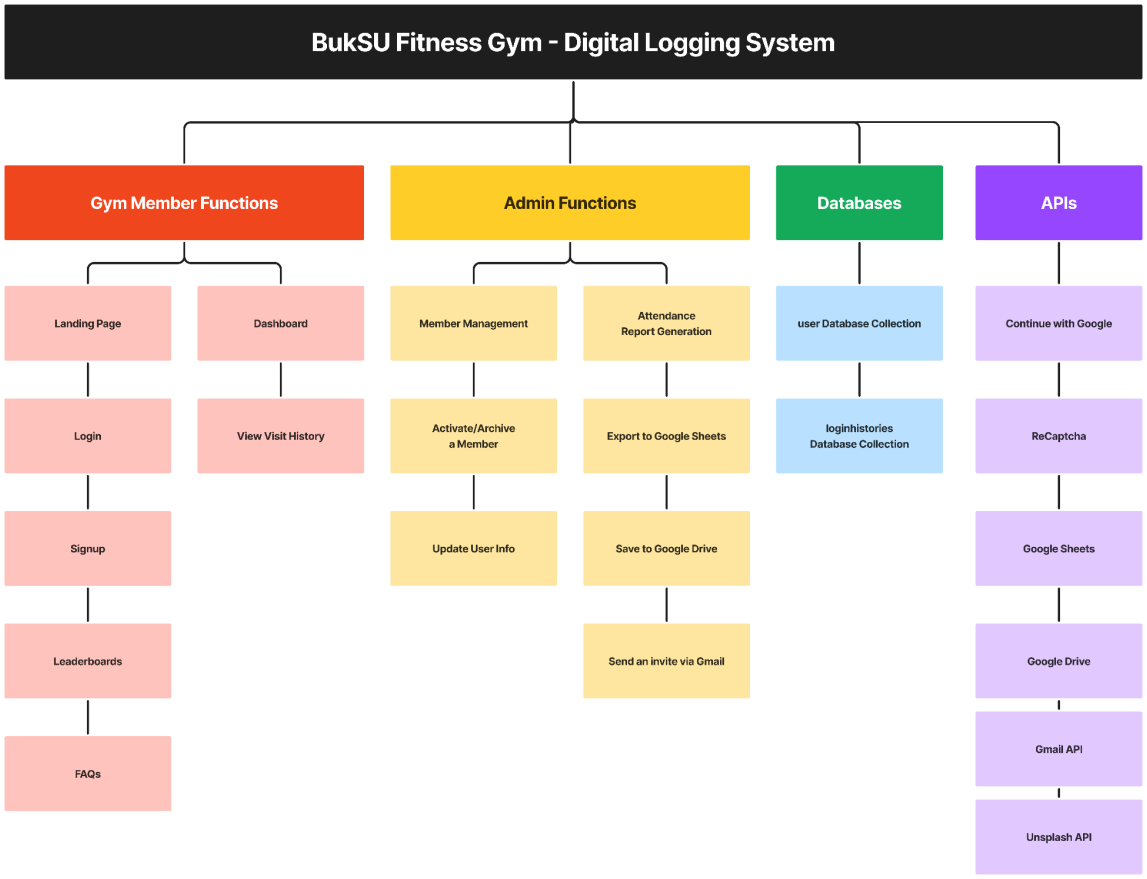
The BukSU Fitness Gym caters to the Bukidnon State University community, including students and faculty. It serves a diverse demographic, fitness levels, and academic roles. Most users are students seeking an accessible and affordable facility to complement their active lifestyles, while faculty use the gym to maintain physical health and achieve work-life balance. Its inclusive environment ensures all campus community members feel welcome and supported in pursuing their wellness goals .

1.1.4. Organization Chart


ORGANIZATIONAL CHART
JOY M. MIRASOL, Ph.D.
University President
HAZEL JEAN M. ABEJUELA, Ph.D.
Vice-President for Academic Affairs (VPAA)
ANN SHEILA C. DEL ROSARIO, Ph.D.
Faculty, College of Education
Secondary Education Department
Dean, Bukidnon State University College of Education
DR. REGINA B. DUMDUMAYA
Faculty, College of Education
Physical Education Department


LOWENA C. BASTIDA
CHERISH MAE C. SORRES
TOM S. CRUZ
JOVEN P. DADAP
FRANK ANTHONY A. EMNACE
LOVER JOHN A. GASPAR
REGIE L. MAAGAD
DR. ENRIGO C. NAMOC
ALADIN G. OPISO
CRISTITITO P. ORNOPIA
JICMER E. PACQUIAO
LAURENCE ANTHONY S. LABADAN
HARLEY DAVID SON A. RODRIGUEZ
DR. MARY DAPHNE P. SILVESTRE
DR. IRISH KIM G. TINULA
ALMAE GRACE D. DURAN

1.1.5. Functional Chart



1. Gym Member Functions (Red Section)

This section represents the features available to gym members for interacting with the system.

- **Landing Page**:
    - Entry point for gym members to log in or sign up.
    - Includes options for:
        - **Login**: Authenticate via Google Sign-In or manual credentials.
        - **Signup**: Create a new account (Google-based or manual).
    - Includes FAQs and leaderboards for engagement.

- **Dashboard**:
    - A personalized area where members can:

        - **View Visit History**: Access their attendance records.
        - **Leaderboards**: View rankings based on attendance frequency.
        - **FAQs**: Access answers to common questions.

2. Admin Functions (Yellow Section)

This section outlines the tools and functionalities available to gym administrators for managing members and gym operations.

- **Member Management**:
    - Admins can:
        - **Activate/Archive a Member**: Approve/reject membership requests and deactivate accounts.
        - **Update User Info**: Edit member profiles (First Name and Last Name).
- **Attendance Report Generation**:
    - Admins can generate detailed reports on member attendance.
    - Options include:
        - **Export to Google Sheets**: Automatically format and export reports.
        - **Save to Google Drive**: Store reports securely in the cloud.
        - **Send an Invite via Gmail**: Email members directly (e.g., for membership confirmations or notifications).

3. Databases (Green Section)

The databases store and manage all the critical data required by the system.

- **User Database Collection**:
    - Stores member details, login credentials, and membership statuses.
- **LoginHistories Database Collection**:
    - Records user attendance logs (e.g., login/logout times) and tracks gym usage.

4. APIs (Purple Section)

The system integrates various third-party APIs to enhance functionality and ensure seamless operation.

- **Continue with Google**:
    - Allows users to log in or sign up using their Google accounts securely.
- **ReCaptcha**:
    - Adds security to prevent bots or unauthorized access.
- **Google Sheets**:
    - Automates report generation and formatting for attendance and membership data.
- **Google Drive**:
    - Provides secure storage for generated reports and files.
- **Gmail API**:
    - Enables the system to send automated emails (e.g., membership status notifications, invitations).
- **Unsplash API**:
    - Supplies high-quality images for dynamic and visually appealing backgrounds.

1.2. Business Processes

1.2.1. Information Asset Identification

The BukSU Fitness Gym Digital Logging System used key information assets to ensure efficient operation and data management.

User Data: Includes user profiles, such as name, email, and university ID, collected during registration.

Database: Hosted MongoDB Atlas, storing user and visit logs.

Barcode Scanning: Encoded user information linked on their unique id barcodes for seamless sign in and attendance tracking.

Visit Records: Tracks gym visit history.

Report Analytics: Total data used for generating reports.

1.2.2. List of Information Assets

A comprehensive list of the system's information assets is essential to manage the resources that enable the BukSU Fitness Gym Digital Logging System to function effectively. These assets include:

- Name, email address, university ID.
- Google Authentication (OAuth2) tokens for secure user login.
- Attendance Records
- Unique Barcodes (linked to user id)
- MongoDB Atlas Database
- System Configuration Files
- Analytics Reports

1.2.3. Threat Identification

The BukSU Fitness Gym Digital Logging System faces potential threats that could compromise its functionality, security, or data integrity. These threats include:

Data Breaches: Potential exposure of sensitive user information, such as personal details and attendance records, due to hacking or insufficient encryption.

Malware or Ransomware Attacks: Threats that could corrupt or encrypt system data, demanding a ransom or causing permanent loss.

Insider Threats: Risk arising from intentional actions by authorized users.

Data Loss: Issues stemming from database errors and software bugs.

Barcode Spoofing: The possibility of unauthorized individuals replicating or tampering with user barcodes to gain illegitimate access.

Unauthorized Access: The risk of unauthorized individuals gaining access to user accounts or system data through weak authentication measures or compromised credentials.

1.2.4. Vulnerability Database

The vulnerability database provides a detailed account of potential weaknesses within the system that could be exploited by various threats, threatening the system's security, functionality, and user trust. By

identifying these vulnerabilities, the database serves as a foundational resource for assessing the system's exposure to risks and prioritizing mitigation strategies.

1. Weak Authentication Mechanisms
    - Users can access the system with minimal authentication checks.
    - Unauthorized users could gain access to sensitive user data.
2. Unencrypted Data Storage
    - User data and visit records are stored in plaintext in the database.
    - In case of a breach, attackers could directly access sensitive information.
3. Barcode Manipulation
    - The system does not validate barcodes against an active user list.
    - An attacker could create a fake barcode to log unauthorized visits.
4. System Dependency on Internet Connectivity
    - The system fails to operate during network outages.
    - Users cannot log visits or access the system without internet access.
5. Lack of Monitoring and Logging
    - No system monitoring tools are in place to track unusual activities.
    - Malicious actions could go undetected, leading to data theft.

1.2..5. Vulnerability Assessment

The vulnerability assessment evaluates the identified weaknesses within the system to determine their potential impact and likelihood of exploitation. This process helps prioritize vulnerabilities based on the level of risk they pose to the system’s security and functionality. By systematically analyzing each vulnerability, the assessment provides actionable insights for implementing effective mitigation strategies.

1. Identification of Vulnerabilities:
    - Analyze the vulnerability database to compile a comprehensive list of system weaknesses, such as weak authentication mechanisms, unencrypted data, and outdated software.
2. Action Plan Development:
    - Create targeted action plans for addressing each vulnerability, specifying steps such as software updates, enhanced security measures, or system redesigns.

3. Risk Evaluation:
    - Assess the likelihood of each vulnerability being exploited, considering factors such as the system's exposure, accessibility, and existing safeguards.

Example Assessment:

Weak Authentication Mechanisms

Likelihood: High (due to reliance on user credentials)

Impact: Critical (could lead to unauthorized access to user data and system functions)

Mitigation Priority: Immediate

Unencrypted Data Storage

Likelihood: Medium (limited exposure but susceptible if breached)

Impact: High (compromised data privacy and security)

Mitigation Priority: High

Barcode Reveal

Likelihood: Low (requires advanced knowledge to exploit)

Impact: Medium (could allow unauthorized gym access)

Mitigation Priority: Medium

1.3. Background of the System

The BukSU Fitness Gym Digital Logging System is an improvement in the processes of user identification, tracking attendance, and management at Bukidnon State University. The system supplants traditional manual logging into a digital platform that makes use of barcodes for sign-in and Google Authentication to secure access by users. This means that the gym staff can keep accurate visit records and update users on their history of attendance and the count of visits.

The system integrates Google Authentication for robust protection of user data from unauthorized access. Barcode-based sign-in allows users to check in at the gym seamlessly, contactlessly, and minus the logbook. This system uses MongoDB Atlas, which ensures scalability and reliability through the accommodation of a growing user base while maintaining the highest performance.

This is quite relevant today because the demand for efficient, paperless, and secure solutions is growing. It fits the mode in which modern need emphasizes the digital transformation of operations in organizations. Not only does it reduce paper but ensure sustainability in the management of various huge volumes of data.

1.4. Objectives

1.4.1. General Objectives

The ultimate goal of the BukSU Fitness Gym Digital Logging System project is to develop a reliable, secure, and user-friendly platform for managing gym attendance and user records within Bukidnon State University. This system aims to replace traditional manual methods with an efficient digital solution that enhances both user experience and administrative processes.

The program is designed primarily for the students and faculty of the university, providing them with a seamless method to log gym visits, track attendance history, and ensure secure access through Google Authentication. It will also assist gym administrators in managing and monitoring gym usage effectively.

1.4.2. Specific Objectives

To achieve the ultimate goal of the Digital Logging System for the BukSU Fitness, there are a few set tasks that embody the key steps of the development. Firstly, the system needs the creation and development of user registration and authentication modules, which will have the secure integration of Google OAuth2 for authorized users and a registration process to acquire and validate user details. A barcode-based check-in method is another important feature, which allows for rapid gym visits through the scanning of the barcodes.

Together with the database setup and management, the task includes the creation of a secure MongoDB Atlas database to save the user accounts, visit logs, and system configurations while ensuring the state of the database as the third normal form for the effective management of the data. Also, a user dashboard should be designed, which would show real-time data of the number of users and user history, among others, and be user-friendly, at a time. Moreover, the system will contain administrative tools for gym workers, so that they can monitor user attendance, produce reports, and dispense with system settings following a very restricted procedure for authorized personnel.

Error handling and system logging are the components, however, to prevent system crashes, to render the error messages in the most meaningful way, and to, therefore, follow the users' activities, among other things. Through rigorous testing, and the quality checkup, the reliability and satisfaction of users will be ensured, whereby functional tests, performance assessments, and security inspections will validate all the functionalities. Finally, the system will be deployed on a stable platform for live use, accompanied by a maintenance plan for regular updates, backups, and troubleshooting. Those phases are crucial for developing a secure, fast, user-friendly digital logging system.

1.5. Scope and Limitations

Scope of the System

The BukSU Fitness Gym Digital Logging System is aimed at modernizing the Certification of Attendance process in the gym to make it more efficient and secure. This system provides solutions such as Google OAuth2 login, using barcode for easy signing in, and a dashboard where users can see their visit count in real-time. Through management tools, the Gym administrators can Track attendances, manage user personal information, as well as create reports. Users can access the platform by using modern web browsers and are able to check-in with barcode scanners or mobile devices. A stable internet connection is necessary for the proper functioning of the system, and it uses MongoDB Atlas for secure database storage on the cloud.

Limitations of the System

The system can only be used by the BukSU, and external users are not supported. With no offline functions, this system is out of reach during internet outages. Sophisticated properties like equipment monitoring, fitness wearable device support, or prediction of future performances are not equipped to this gym. Furthermore, the development of the mobile app, which was originally planned, had to be postponed due to time and resource constraints. It covers only these restrictions that fall within the project's timeline and can be supported by the available resources.

1.6. Definition of Terms

Authentication: The process of verifying a user's identity to grant access to the system. In this project, Google OAuth2 is used for secure authentication.

Barcode: A machine-readable code representing data in the form of patterns. It is used in this system for user identification and attendance tracking.

Cloud Storage: A method of storing data on remote servers accessed via the internet. MongoDB Atlas serves as the cloud database for this project.

Database: An organized collection of data stored electronically. The system uses MongoDB Atlas to manage user profiles and attendance records.

Google OAuth2: An authentication protocol that allows users to log in securely using their Google accounts.

MongoDB Atlas: A cloud-based NoSQL database platform used to store and manage the system's data securely.

Real-time Updates: The immediate processing and display of data as it is entered into the system, allowing users to see their visit counts and attendance history instantly.

User Dashboard: A graphical interface that provides users with access to their attendance history, visit counts, and other system-related information.

Visit History: A record of a user's gym attendance, including dates and times, stored in the system for tracking purposes.

Web-based System: A software application accessible through a web browser, enabling users to log in and access the system without installing additional software.

Chapter 2
RISK & SYSTEM ANALYSIS

2.1. Risk Analysis

This part is employing the risk analysis tools, the project team can better prepare for uncertainties, improve decision-making processes, and enhance the overall management of risks associated with the BukSU Fitness Gym - Digital Logging System

2.1.1 Likelihood Scale

The likelihood scale is used to evaluate how probable it is for identified risks to occur within the BukSU Fitness Gym Digital Logging System.

Likelihood	Description
Very Low (1)	The risk is highly unlikely to occur under normal operating conditions.
Low (2)	The risk has a small chance of occurring but could happen under certain circumstances.
Moderate (3)	The risk has a reasonable chance of occurring during system operation.
High (4)	The risk is likely to occur, especially without proper controls in place.
Very High (5)	The risk is certain to occur and poses an immediate threat if not addressed.

This scale allows the development team to systematically assess risks and focus on those with higher probabilities of occurrence.

2.1.2. Risk Impact Scale

The risk impact scale evaluates the potential consequences of identified risks on the BukSU Fitness Gym Digital Logging System.

Impact Level	Description
Very Low (1)	Minimal impact, the issue is negligible and does not affect system functionality or user experience.
Low (2)	Minor impact, the issue slightly affects functionality or user experience but can be resolved quickly.
Moderate (3)	Noticeable impact, the issue affects core functions and may require additional resources to resolve.
High (4)	Significant impact, the issue disrupts critical system operations and could harm user trust.
Very High (5)	Severe impact, the issue causes system-wide failure, data loss, or major security breaches.

2.1.3. Risk Rating Worksheet

The risk rating worksheet combines the likelihood and impact scales to prioritize risks for the system. Each risk is assigned a score by multiplying its likelihood and impact levels, enabling a clear understanding of its overall severity.

Risk	Likelihood (1-5)	Impact (1-5)	Priority
Data Breaches	3	5	High
Malware or Ransomware attack	3	5	High
Insider Threats	3	4	High
Data Loss	3	5	High
Barcode Spoofing	2	3	Medium
Unauthorized Access	4	5	Critical

Risk Priority Levels:

- Critical (16-25): Requires immediate attention and mitigation.
- High (10-15): Needs prompt action to prevent significant system or data issues.
- Medium (6-9): Moderate concern, mitigation efforts can be scheduled after higher-priority risks.
- Low (1-5): Minimal concern, can be addressed during routine maintenance.

2.1.4. Risk Control Strategy

Ctrl+Alt+Elite identifies potential risks associated with the system's operation and outlines effective control strategies to mitigate these risks. These strategies ensure the system remains secure, reliable, and user-friendly while maintaining operational efficiency.

Identified Risks and Control Strategies

1. Security Risks

- **Risk**: Unauthorized access to sensitive data (e.g., user credentials, attendance logs).
- **Control Strategy**:
    - Implement strong authentication mechanisms, including Google Sign-In and encrypted passwords.
    - Use JWT (JSON Web Tokens) for secure session management.
    - Apply SSL/TLS encryption for all data transmission.
    - Conduct regular vulnerability assessments to identify and address security loopholes.

2. Data Loss or Corruption

- **Risk**: Accidental deletion, corruption, or loss of data in the database.
- **Control Strategy**:
    - Use MongoDB Atlas backups to ensure that data can be restored in case of an issue.
    - Implement write-safe operations and validations to avoid accidental data loss.

    - Regularly monitor the database for anomalies using MongoDB Atlas monitoring tools.

3. Unauthorized Access or Abuse by Admins

- **Risk**: Admin misuse of privileges or unauthorized changes to user data.
- **Control Strategy**:
    - Enforce role-based access control (RBAC) to limit admin actions.
    - Maintain audit logs of all administrative actions for accountability.
    - Regularly review admin permissions to ensure compliance with security policies.

4. User Data Privacy Breach

- **Risk**: Leakage or unauthorized sharing of user data.
- **Control Strategy**:
    - Comply with data privacy laws (e.g., Data Privacy Act of 2012, GDPR) by anonymizing sensitive user data where applicable.
    - Encrypt sensitive fields like email addresses and IP addresses in the database.
    - Restrict data access to only authorized users or processes.

5. Integration Failures

- **Risk**: Failure of third-party APIs (e.g., Google APIs, Unsplash) causing disruptions.
- **Control Strategy**:
    - Implement retry mechanisms for failed API calls.
    - Use circuit breakers to temporarily bypass failing APIs while maintaining core functionalities.
    - Monitor API usage and set up alerts for issues or quota limits.

6. User Errors

- **Risk**: Incorrect inputs or actions by users leading to system malfunctions or data inconsistencies.
- **Control Strategy**:
    - Provide input validation and informative error messages for user inputs.

- Use ReCaptcha to prevent bot-related errors or malicious activities.
- Offer FAQs and user guides to help users navigate the system effectively.

7. Scalability Challenges

- **Risk**: System performance degradation as the number of users and data increases.
- **Control Strategy**:
  - Use NoSQL database indexing to improve query performance.
  - Optimize backend code and queries for scalability.

8. Poor User Experience

- **Risk**: Users abandoning the system due to unintuitive design or slow performance.
- **Control Strategy**:
  - Design a **user-friendly interface** with intuitive navigation and clear instructions.
  - Regularly collect **user feedback** and iterate on improvements.
  - Optimize frontend performance to reduce load times.

2.2. Requirements

2.2.1 Functional Requirements

The functional requirements define the essential features and capabilities of the BukSU Fitness Gym Digital Logging System. These requirements specify what the system must do to meet the needs of its users, including administrators and gym members, while ensuring the system operates efficiently and securely.

Admin	**Description**
Login	Admin must be able to log in to the system using Google Authentication.
Generate Reports	Admin must be able to generate reports on gym attendance trends and user activity.
Manage System Configuration	Admin must have access to configure system settings, such as user management.

User	
Login	Users must be able to log in to the system using Google Authentication.
Sign In via Barcode	Users must be able to sign in to the gym by scanning their unique barcode.
View Attendance History	Users must be able to view their visit history, including dates and times of attendance.
Track Visit Count	Users must be able to view their total gym visits in real-time on the dashboard.

2.2.2 Non-Functional Requirements

The non-functional requirements specify the quality attributes, performance criteria, and operational constraints of the system.

Performance	
System Responsiveness	The system must process user actions, such as logins and barcode scans, within 3-5 seconds.
Scalability	The system must handle up to (how many student/faculty) simultaneous users without performance degradation.
Usability	
User Interface	The user interface must be intuitive and user-friendly, accessible to users with minimal technical expertise.
Availability	
System Uptime	The system must maintain a minimum uptime of 99.5%, ensuring high availability.
Reliability	
Fault Tolerance	The system must remain functional in the event of minor server or network failures.
Data Integrity	The system must ensure data consistency and accuracy across all modules.

2.2.3 Database Schema

The Buksu Fitness Gym - Digital Logging System employs a NoSQL database schema using MongoDB, which allows for flexible and scalable data storage. Below is the description of the database schema for the key collections, LoginHistory and User.

1. loginhistories Collection

The `loginhistories` collection stores data about user login sessions. This schema is essential for tracking user activity, including timestamps and device details.

Schema Structure:

```
const LoginHistorySchema = new mongoose.Schema({
  userId: { type: mongoose.Schema.Types.ObjectId, ref: "User",
required: true },
  loginTime: { type: Date, default: Date.now },
  logoutTime: { type: Date },
  ipAddress: { type: String },
  device: { type: String },
});
```

Explanation:

Field	Type	Description
`userId (Foreign Key)`	ObjectId	References the `user` collection who initiated the login session.
`loginTime`	Date	The time when the user logged into the system. Defaults to the current date and time.
`logoutTime`	Date	The time when the user logged out of the system.

`ipAddress`	String	Captures the user's IP address during the login session
`device`	String	Identifies the type of device (e.g. desktop, mobile) used for the login session.

Purpose:

- Tracks user sessions for security and analytics.
- Enables auditing of login activity to detect unusual patterns or potential unauthorized access.

2. user Collection

The `user` collection stores information about the gym members and administrators. It is critical for managing authentication, roles, and access control.

Schema Structure:

```
const UserSchema = new mongoose.Schema({
  firstName: { type: String },
  lastName: { type: String },
  email: { type: String, unique: true, required: true },
  googleId: { type: String },
  password: { type: String, required: false },
  isActive: { type: Boolean, default: false },
  createdAt: { type: Date, default: Date.now },
  isAdmin: { type: Boolean, default: false },
});
```

Explanation:

Field	**Type**	**Description**

firstName	String	Stores the user's first name.
lastName	String	Stores the user's last name.
email	String	Stores the user's unique email address. Required for login and communication.
googleId	String	Stores the Google OAuth ID for users who sign up using Google.
password	String	Stores the user's password (if not signing up with Google). Optional field.
isActive	Boolean	Indicates whether the user's account is active. Default is false.
createdAt	Date	The timestamp when the user was created. Defaults to the current date and time.
isAdmin	Boolean	Indicates if the user has administrative privileges. Default is false.

Purpose:

- Manages user details and ensures proper authentication.
- Differentiates between regular users and admin roles.
- Supports both Google-based and manual signup processes.

Database Relationships

1. One-to-Many Relationship:

The userId field in the loginhistories collection references the _id field in the user collection. This establishes a one-to-many relationship, where:

- One user can have multiple login history entries.
- Each login session is linked to a specific user.

MongoDB Entry Example:

user:

```
{
  "_id": "63e8c2f0123456789abcdef1",
  "firstName": "Neil Marc",
  "lastName": "Bayron",
  "email": "2001101888@student.buksu.edu.ph",
  “password”:
"$2b$10$Awo8tvfP52LXpvcxD/RDPeC0PDINVJkQatb3jmCVOB4iJF5K2Yy06",
  "isActive": true,
  "isAdmin": false,
  "createdAt": "2024-12-12T12:00:00Z"
}
```

loginhistories:

```
{
  "_id": "63e8c2f1123456789abcdef2",
  "userId": "63e8c2f0123456789abcdef1",
  "loginTime": "2024-12-13T08:00:00Z",
  "logoutTime": "2024-12-13T10:00:00Z",
  "ipAddress": "192.168.1.1",
  "device": "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/128.0.0.0
Safari/537.36"
}
```

2.2.4 System Interface and Execution

The BukSU Fitness Gym Digital Logging System is a web-based platform, and users interact with the program through a web browser. This section outlines how the system environment is set up, activated, and terminated, along with how users communicate with the program.

System Environment Setup

The system relies on a Node.js backend, MongoDB Atlas database, and a modern frontend (React.js). Below are the steps to set up the environment for both development and deployment:

1. Prerequisites

- **Node.js:** Install Node.js (version 14 or higher, preferably the LTS versions) to run the server-side code.
- **MongoDB Atlas:** Set up a cloud database and configure the connection string.
- **NPM Packages:** Install the necessary dependencies (both in the backend and frontend) with the following command:
  `npm i` or `npm install`.

- **Environment Variables:**
  Prepare or create a .env file in the root directory with the following entries:

```
MONGO_URI=mongodb+srv://nbayron29:Testing123@buksu-fitness-gym-dls
te.tz34i.mongodb.net/?retryWrites=true&w=majority&appName=BukSU-Fi
tness-Gym-DLSTest
GOOGLE_CLIENT_SECRET=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
GOOGLE_CLIENT_ID=1048932778987-3r3dcqgdef1neflo6g2i61reld9slsap.ap
ps.googleusercontent.com
key=AIzaSyCj5Tx4KsOHaTAAPGg7nZF3w9Erm5xF6w4
GMAIL_REFRESH_TOKEN=1//04fFg455w_Ti4CgYIARAAGAQSNwF-L9Irq-ZHZhRyV2
1r9fMzzJWtU-SttxkwVLUxsTzGJfmpIvLPZKULrrPTpcvx2soS-Hy4a1A
GOOGLE_SHEETS_REFRESH_TOKEN=1//048zFzaqpkcboCgYIARAAGAQSNwF-L9IrYj
INd5zTkUGE9HPJIlwFKZxSQ2IUbnrK9Nz3-rXRDMstMAnt9mqrTI7O97Zv2N2SmrI
EMAIL_SENDER=2001101888@student.buksu.edu.ph
APP_PASSWORD="vdhx zeoo mcsv abjk"
RECAPTCHA_SITE_KEY=6LeZQIMqAAAAAAnW9EdB-WQ4cW_ZrDcoGyLf6o-l
RECAPTCHA_SECRET_KEY=6LeZQIMqAAAAACTwYnXIjgsERhq-FpumHrCR1ZMt
SMTP_HOST=smtp.gmail.com
SMTP_PORT=587
PORT=5000
JWT_SECRET=hawktuahskibidigyatt
FRONTEND_URL=http://localhost:3000
```

2. Running the Program

To execute the program, follow these steps:

1. **Start the Backend Server:**
   Navigate to the backend folder (named server) and run the following command: `npm start`
   The server starts and listens on the port defined in the .env file (5000).

2. **Start the Frontend:**
   Navigate to the frontend folder (named client) and start the development server: `npm start`
   This launches the web application in a local environment (http://localhost:3000)

3. **Access the System:**
   Open a web browser (preferably a Chromium-based browser) and navigate to http://localhost:3000 to start interacting with the system.

Parameters and Commands

- The system does not require additional parameters during execution.
- Environment Variables are preconfigured in the .env file for seamless integration with external APIs and databases.

How Users Interact with the System

1. User Communication with the Program

Gym Members:

- Open the web application using a browser.
- Use the Landing Page to log in via their credentials, barcode, or continue with Google
- Users can also sign up using Google Sign-In or manual credentials.
- In the landing page, users can look at the leaderboards or read the FAQs.

- Navigate through the User Dashboard to access functionalities like viewing visit history.

Admins:

- Log in via their dedicated credentials.
- Manage members, generate reports, update member info, and activate memberships

2. Starting the Program

- Users do not need to manually execute commands to start the program.
- They just need to enter the URL (localhost:3000) in their browser.

System Execution and Termination

1. Execution

- Run both backend (`npm start`) and frontend (`npm start`) servers to launch the system.
- Access the system through http://localhost:3000.

2. Termination

- To stop the backend server, just press ctrl+c on the terminal where the server is running.
- This shortcut both applies to the backend and frontend.

3. Logging Out

- Users can terminate their session by:
  Clicking the Logout button on their dashboard, which invalidates or deletes the session token and redirects them back to the landing page.

System Flow During Execution

1. **Gym Member/Administrator accesses the URL:**
    - The frontend (React/Vue.js) communicates with the backend server (Node.js) through RESTful APIs.
    - MongoDB Atlas handles data storage and retrieval.

2. **Session Management:**
    - Secure authentication tokens are managed to maintain user sessions.
    - After logging out, tokens are invalidated and deleted to prevent unauthorized access.

Chapter 3

SYSTEM DESIGN & DEVELOPMENT

3.1. Solution Design

The **Buksu Fitness Gym - Digital Logging System (DLS)** is designed to address the challenges of manual gym operations by providing an automated and streamlined solution for managing member activities, attendance, and administrative tasks. The system leverages modern platforms and technologies to create an efficient, user-friendly, and scalable system that improves both gym member experience and administrative workflows.

Problem Overview

The manual process of tracking gym attendance, managing memberships, and generating reports is time-consuming and prone to errors. Additionally, there is a lack of transparency and automation, leading to inefficiencies in operations and member dissatisfaction.

To solve these issues, the proposed system introduces a digital platform that automates key functionalities such as member registration, attendance logging, membership activation, and report generation.

Solution Overview

The solution integrates the following features:

1. **User-Friendly Interfaces**:
    - A responsive Landing Page and dashboard for gym members to log their attendance, track their visit history, and manage their memberships.
    - An Admin Panel for gym administrators to manage members, activate memberships, and generate detailed reports.

2. **Automated Attendance and Membership Management**:
    - Gym members can log their attendance through the system, and attendance records are automatically updated in the database.
    - Membership requests and activations are automated, with admin oversight.

3. **Seamless Report Generation**:
    - Reports are generated automatically based on attendance and membership data, formatted in spreadsheets, and securely stored in Google Drive.

4. **Leaderboard Feature**:
    - Encourages gym member engagement by displaying rankings based on attendance frequency, fostering a sense of competition and motivation.

5. **FAQ Section**:
    - Provides quick assistance to users, reducing admin workload.

Platforms and Technologies Used

1. **MongoDB Atlas** (NoSQL Database):
    - A cloud-based NoSQL database that stores user profiles, attendance records, and membership details in collections.
    - Ensures scalability, high availability, and efficient data handling.

2. **Google APIs**:
    - **Google Sign-In**: Simplifies authentication by allowing members to log in with their Google accounts.
    - **Gmail API**: Automates email notifications for membership activation, reminders, and report sharing.
    - **Google Sheets API**: Enables automated report generation in spreadsheet format.
    - **Google Drive API**: Provides secure storage for generated reports, ensuring accessibility for administrators.

3. **ReCaptcha API**:

- Adds a layer of security to prevent bots and unauthorized access during user authentication.

4. **Unsplash API**:
    - Enhances the system's visual appeal by providing random high-quality background images for the user interface.

5. **Web Development Technologies**:
    - **Frontend**: Built with modern web technologies (e.g., HTML, CSS, JavaScript, and frameworks like React or Vue.js) to create intuitive user interfaces.
    - **Backend**: Developed using **Node.js** and **Express.js**, ensuring efficient server-side logic and integration with external APIs.

Features of the Solution

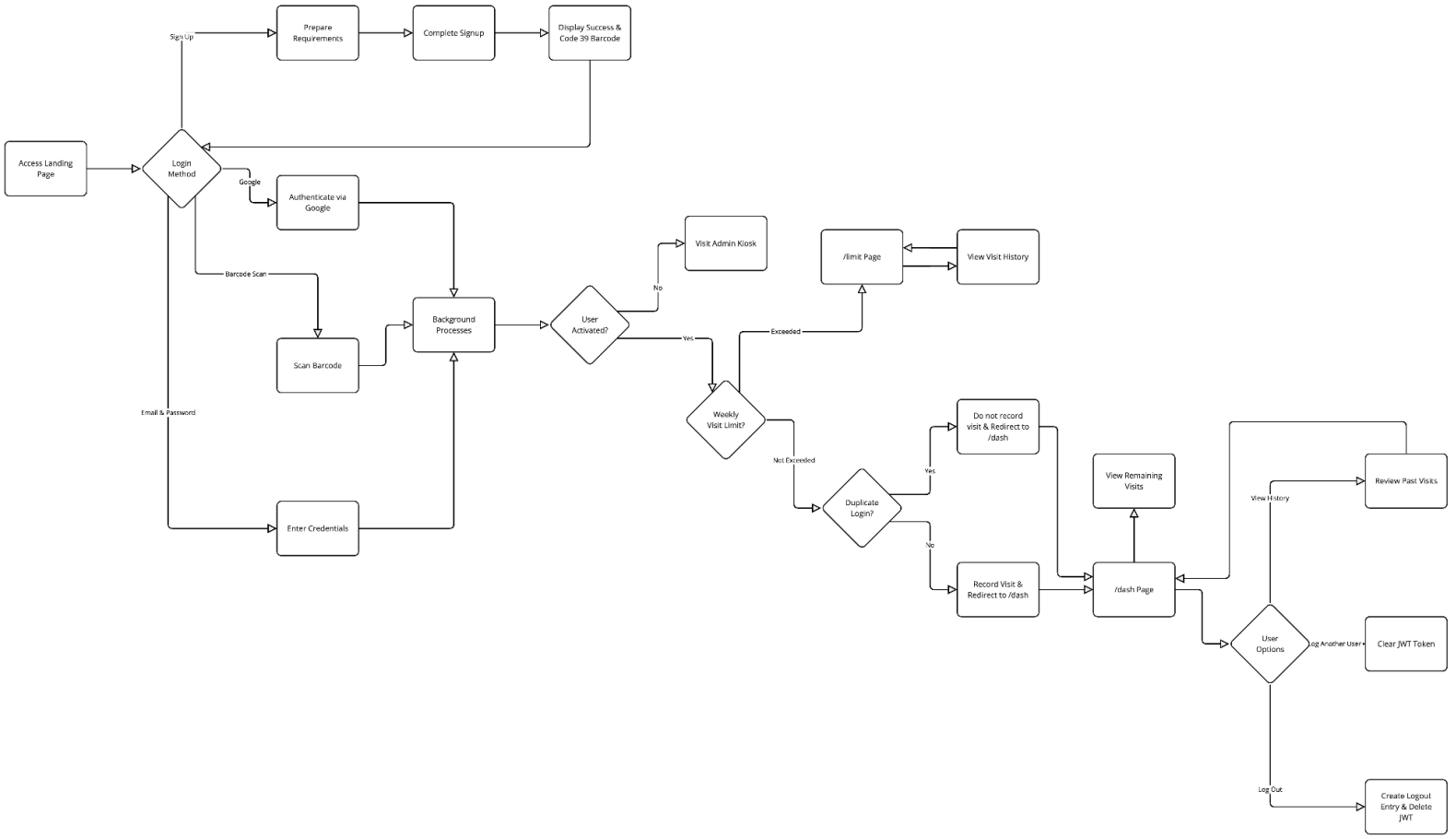
1. **Scalability and Flexibility**:
    - The use of MongoDB ensures the system can handle large datasets, adapt to changing data structures, and grow as the number of gym members increases.
2. **Automation**:
    - Key tasks like attendance logging, membership activation, and report generation are automated, saving time and reducing errors.
3. **Real-Time Updates**:
    - All changes (e.g., membership activations, attendance logs) are reflected immediately across the system, ensuring up-to-date information for users and admins.
4. **Enhanced User Engagement**:
    - The leaderboard fosters healthy competition among members, and the FAQ section provides quick answers to common queries.
5. **Secure and Reliable**:
    - HTTPS and OAuth 2.0 ensure secure communication and authentication.
    - MongoDB Atlas provides built-in encryption and backup options.
6. **Ease of Access**:

- Members and admins can access the system anytime, using the kiosk, or when deployed, it will be hosted on Amazon Web Services (AWS), Microsoft Azure, or Captive Portal.

The **Buksu Fitness Gym - Digital Logging System** solves the inefficiencies of manual gym management by automating critical tasks, enhancing user engagement, and providing administrators with robust tools to manage the gym's operations. By integrating modern platforms like MongoDB Atlas and Google APIs, the system ensures scalability, security, and a seamless experience for both gym members and staff. This digital solution transforms gym management into a streamlined and efficient process.

3.2. Architectural Views

3.2.1. Logical View (Activity Diagram)



This activity diagram outlines the process for user sign-up, login, and visit management.

1. Access Landing Page

- Users start at the **Landing Page** to either **Sign Up** or **Log In**.

2. Sign-Up Flow (for new users)

- **Prepare Requirements**
  Users complete required steps to sign up (e.g., provide identification, membership details).
- **Complete Signup**
  Finish the sign-up process and submit the information.
- **Display Success & Code 39 Barcode**
  The system generates a success message and provides a Code 39 Barcode for future use.

3. Login Method

Users choose a login method:

- **Google Authentication**
- **Barcode Scan**
- **Email & Password**

3a. Authenticate via Google

- If users log in via Google, the system proceeds to Background Processes.

3b. Scan Barcode

- Users scan their Code 39 Barcode, and the system processes it in Background Processes.

3c. Enter Credentials

- Users log in using Email & Password, which also leads to Background Processes.

4. Background Processes

After login, the system verifies the user's data.

4a. User Activated?

- **Yes:** Continue to the next step.
- **No:** Show a message telling the user they should visit the admin kiosk for activation.

5. Visit Management

Once the user is activated, the system checks visit criteria:

5a. Weekly Visit Limit?

- **Exceeded:** Redirect to the Limit Page where users can view their visit history.
- **Not Exceeded:** Proceed to check for duplicate logins.

5b. Duplicate Login?

- **Yes:** Do not record the visit and redirect the user to /dash.
- **No:** Record the visit and redirect to /dash (main dashboard).

6. Dashboard (/dash Page)

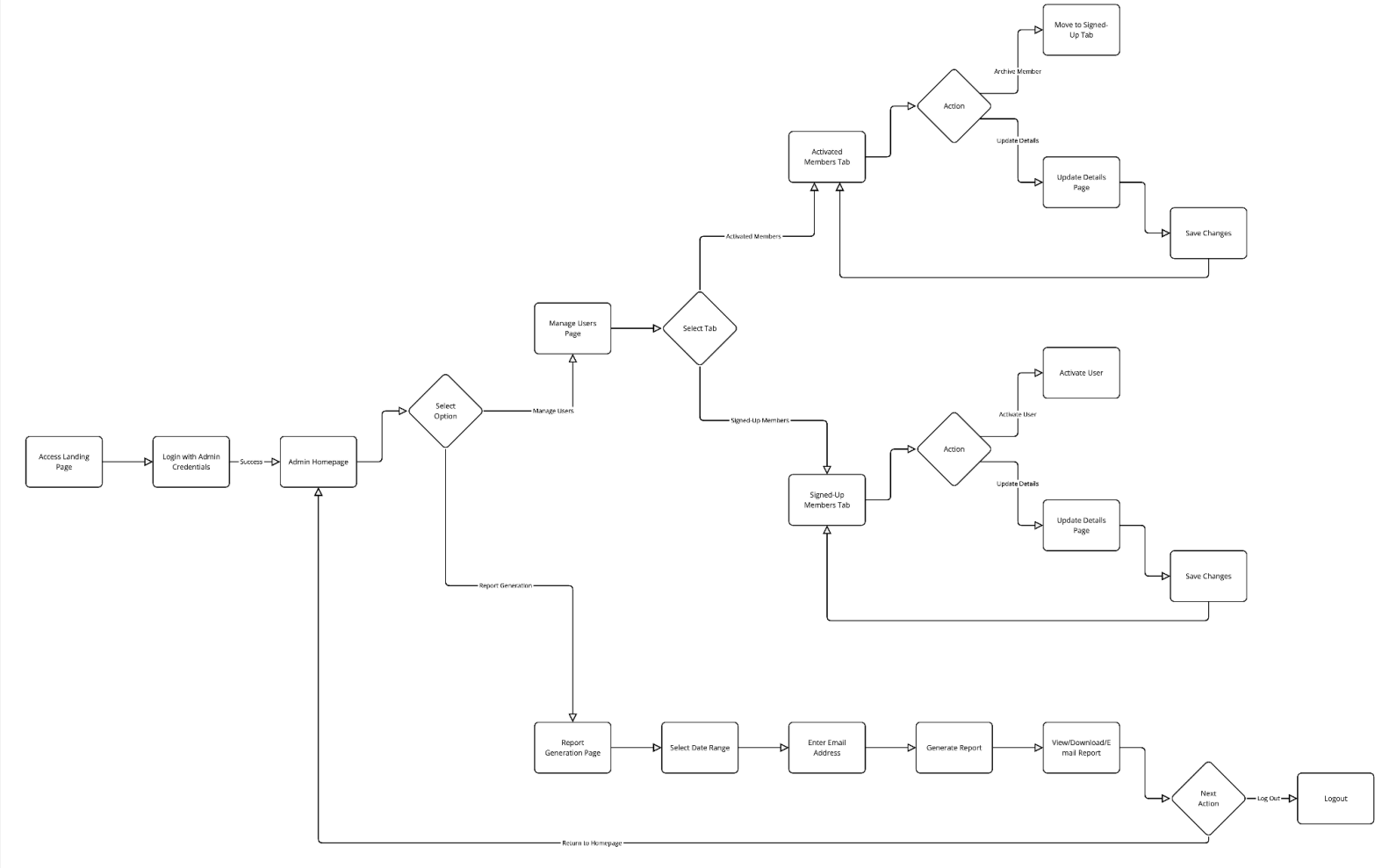
- Users land on the /dash page after successful login and visit recording.

Options include:

- **View Remaining Visits**: Check how many visits are left for the week.
- **User Options**: Access other functionalities like:
  - **Review Past Visits**
  - **Log Out**
  - **Log in Another User**

6a. Log Out Options

- **Clear JWT Token**: Clears user tokens for security.
- **Create Logout Entry & Delete JWT**: Logs out the user completely.



This activity diagram outlines the process for admin process flow. It details how administrators can manage users, update records, and generate reports.

1. Access Landing Page

- Admins start at the Landing Page.

2. Login with Admin Credentials

- Admins log in using their credentials.
- **Success**: Redirect to the Admin Homepage.

3. Admin Homepage

From the Admin Homepage, the admin can choose between two primary options:

1. **Manage Users**
2. **Report Generation**

4. Manage Users

- Admins navigate to the Manage Users Page.
- Select one of two tabs:
    - **Activated Members Tab**
    - **Signed-Up Members Tab**

4a. Activated Members Tab

- This tab displays already activated users.
- Admin can choose an Action:
    - **Move to Signed-Up Tab:** Deactivates the member and moves them back to the Signed-Up Members Tab.
    - **Update Details**: Redirect to Update Details Page, where changes are made.
        - Admin saves changes via Save Changes.

4b. Signed-Up Members Tab

- This tab displays users who have signed up but are not yet activated.
- Admin can choose an Action:
    - **Activate User**: Activates the user.
    - **Update Details**: Redirect to Update Details Page, where changes are made.
        - Admin saves changes via Save Changes.

5. Report Generation

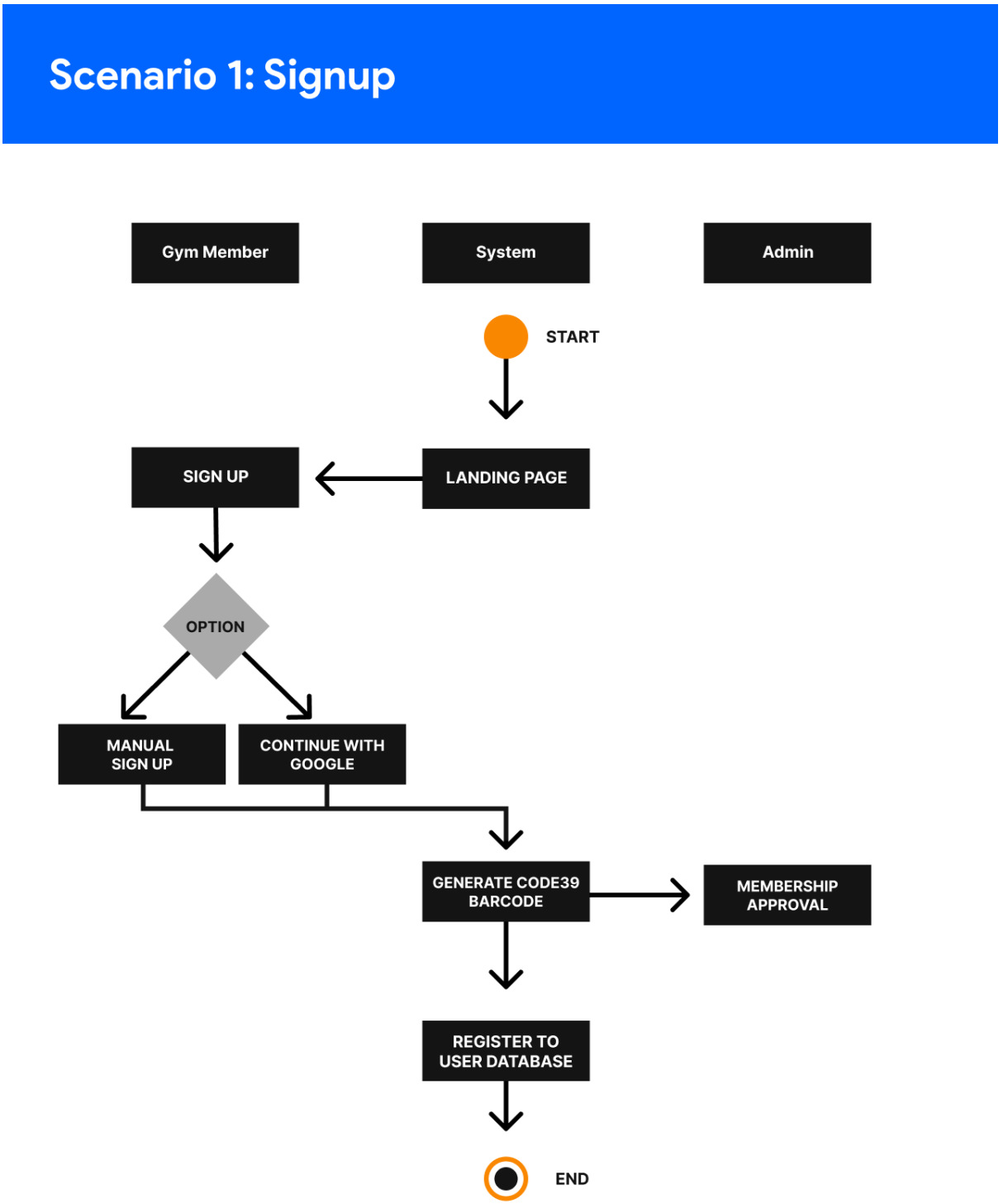
- From the Admin Homepage, select Report Generation.
- Steps for generating a report:
    1. Navigate to the Report Generation Page.

2. **Select Date Range**: Choose the time period for the report.
3. **Enter Email Address**: Input where the report will be sent.
4. **Generate Report**: System creates the report.
5. **View/Download/Email Report**: Admin can view, download, or email the generated report.

6. Next Actions

- After completing tasks, the admin can:
    - **Log Out**: Logs out of the system.
    - Return to the Admin Homepage to perform other tasks.

3.2.2. Process View (User Flow Diagram)

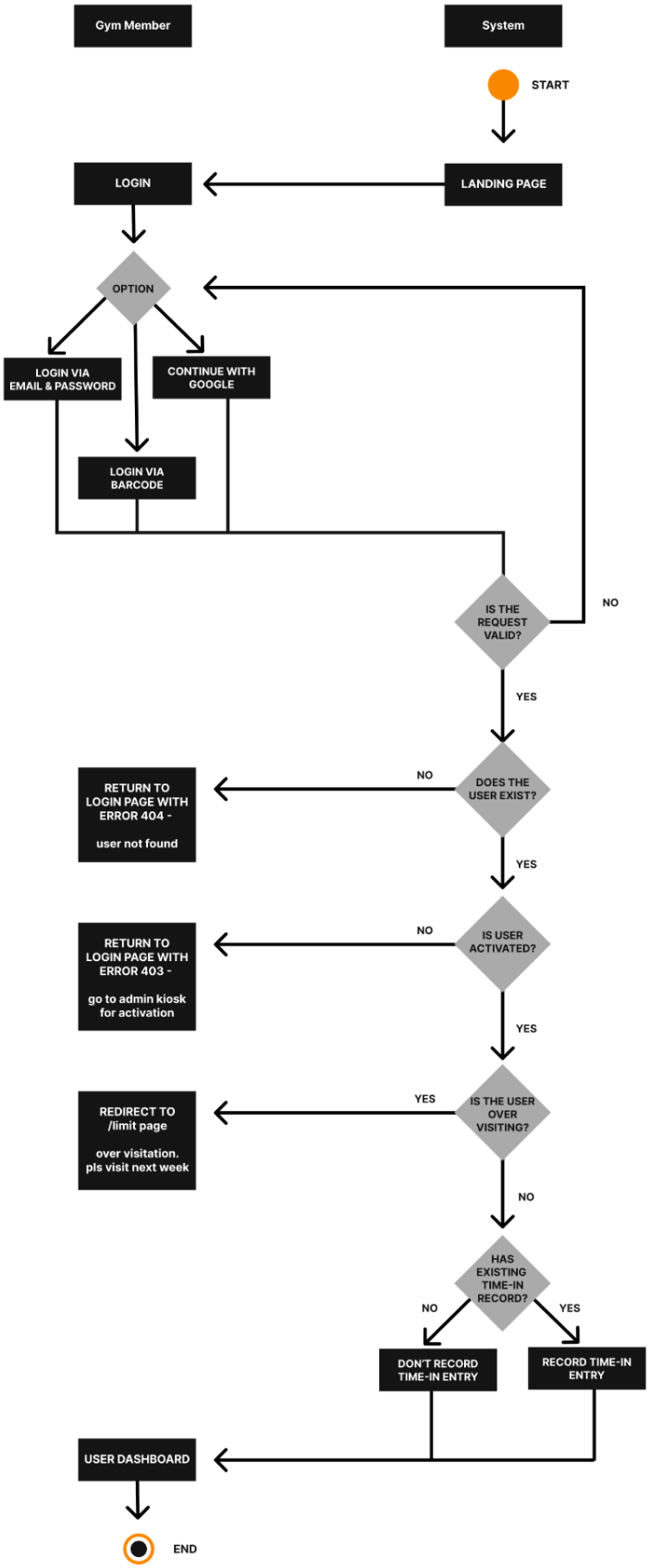


The **Scenario 1: Signup** user flow diagram represents the process for a gym member to register into the **Digital Logging System (DLS)**. The process begins when the system initiates at the

Landing Page, where a gym member can choose to **Sign Up**. At this stage, the member is presented with two options: **Manual Sign Up**, where they input their details manually, or **Continue with Google**, which allows them to register quickly using their Google account credentials.

Once the signup method is completed, the system generates a **Code39 Barcode** for the user, which serves as a unique identifier for future gym visits. At this point, the process requires **Membership Approval** from an **Admin**, who reviews the signup details and grants approval to activate the account. Following approval, the gym member is registered into the **User Database**, completing the signup process. The flow concludes successfully, ensuring the gym member is now part of the system and ready to use their barcode for gym access and activities. This streamlined process involves interactions between the **System**, the **Gym Member**, and the **Admin**, ensuring seamless onboarding and proper membership verification.

Scenario 2: Login



The **Scenario 2: Login** user flow diagram outlines the process for a gym member to log into the **Digital Logging System (DLS)** and access the user dashboard. The flow begins with the **System** initiating at the **Landing Page**, where the gym member selects the **Login** option. The member is then provided with three login methods:

1. **Login via Email & Password**,
2. **Continue with Google**, or
3. **Login via Barcode**.

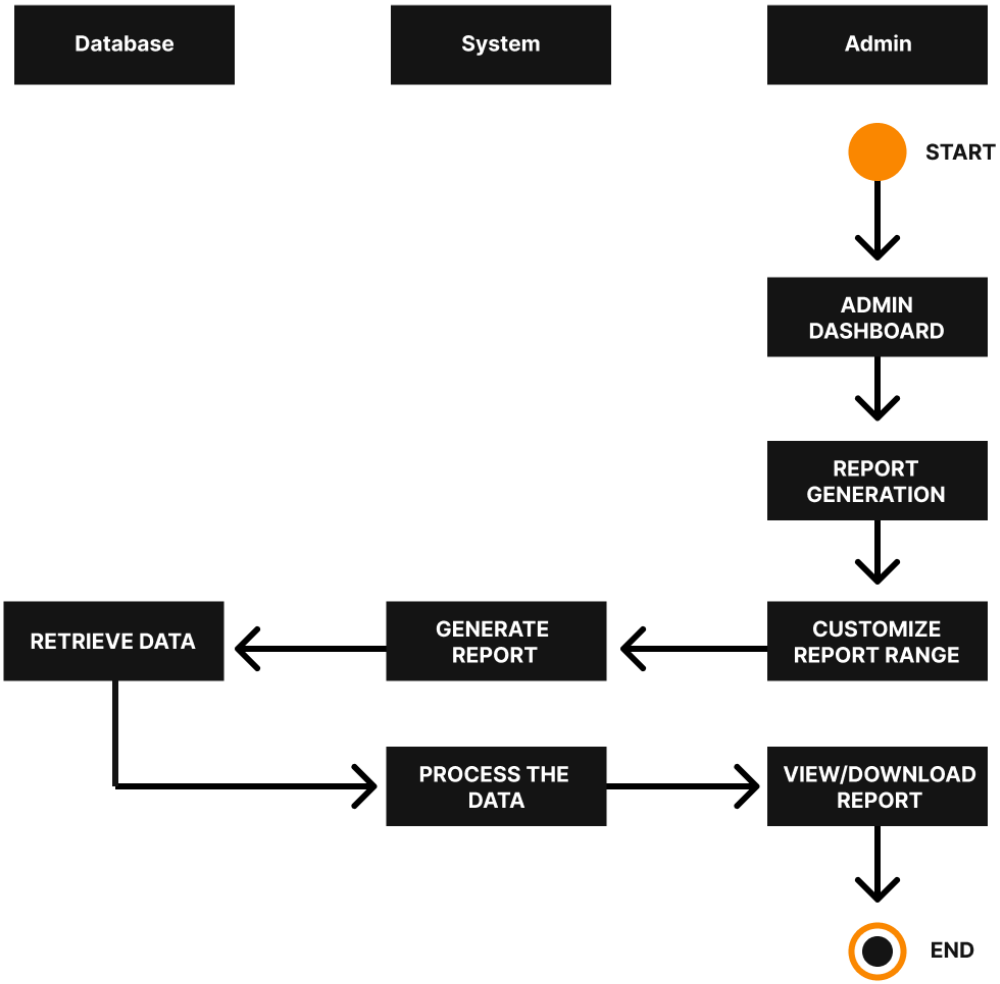
Once the login method is selected, the system checks **if the request is valid**. If it is not valid, the user is redirected to the login page with an error message (e.g., **Error 404: User not found**). If the request is valid, the system checks **if the user exists**. If the user does not exist, it returns the user to the login page with an error.

If the user exists, the system verifies **if the user is activated**. If not, the user receives an **Error 403** message and is instructed to visit the admin kiosk for activation. Once the user is activated, the system evaluates **if the user has exceeded their visitation limit**. If the user is over their visit limit, they are redirected to the **/limit page**, informing them to return the following week.

For users within their limits, the system checks **if an existing time-in record exists**. If a record is found, the system **does not record a new time-in entry** and redirects the user to their dashboard. If no time-in record exists, the system **records the time-in entry** and proceeds to the **User Dashboard**.

The process concludes with the gym member successfully accessing the **User Dashboard**, marking the end of the flow. This login process ensures proper validation, user activation, and tracking of gym visits, while handling errors and over-visitation scenarios efficiently.

Scenario 3: Report Generation



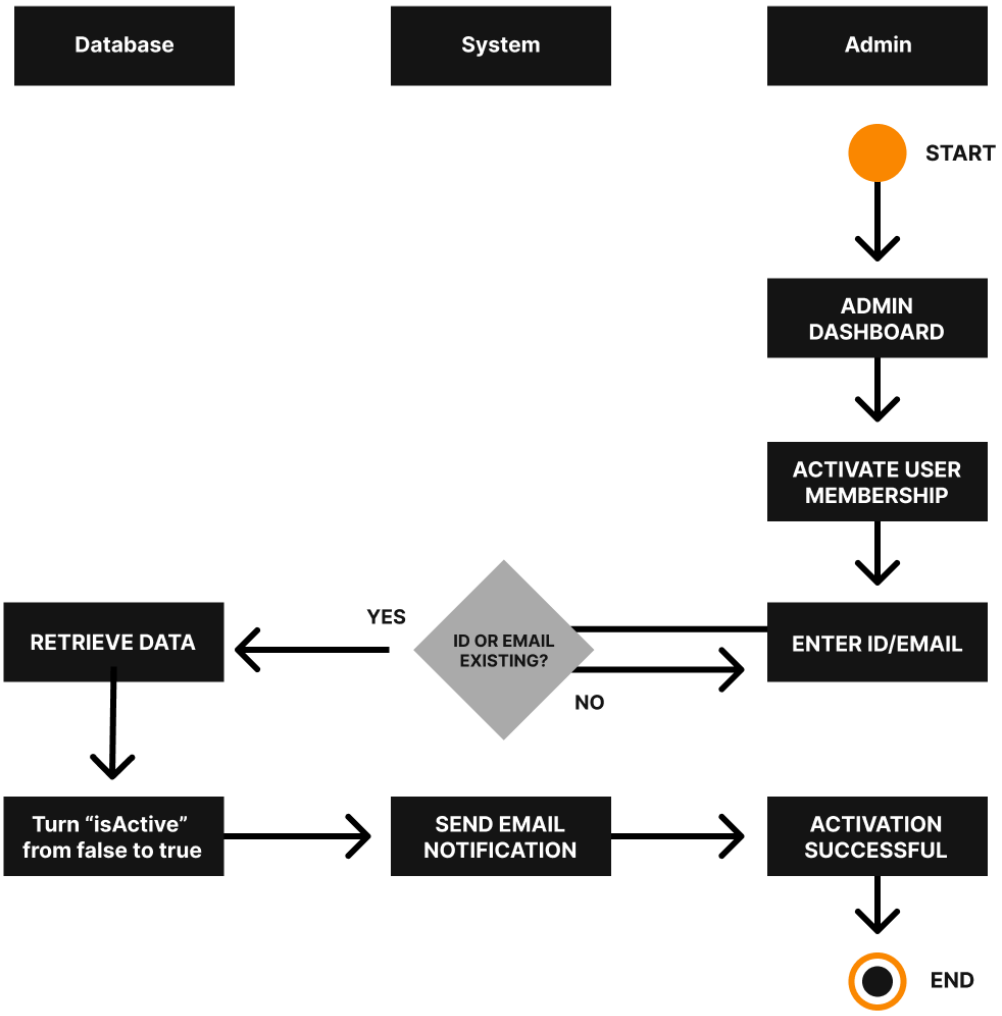
The **Scenario 3: Report Generation** user flow diagram details how an admin generates reports using the **Digital Logging System (DLS)**. The process begins with the **Admin** accessing the **Admin Dashboard**. From there, the admin selects the **Report Generation** option to initiate the report creation process.

The next step involves **customizing the report range**, where the admin specifies the desired parameters, such as date ranges or filters for the report. Once the parameters are set, the **System**

retrieves data from the **Database** to ensure the required information is available. The system then **processes the data**, preparing it for analysis and formatting.

After processing, the system proceeds to **generate the report** based on the customized parameters. The generated report is presented to the admin, who has the option to **view or download the report** for further use. At this stage, the report generation process concludes successfully.

Scenario 4: Membership Activation



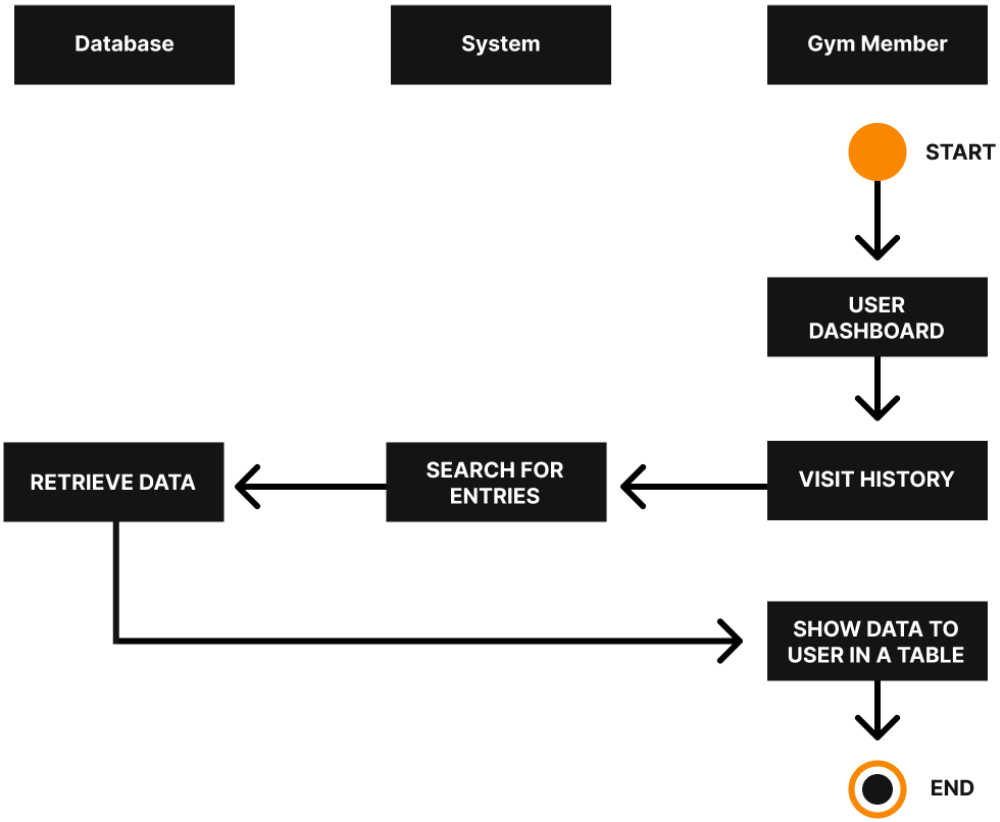
The **Scenario 4: Membership Activation** flowchart outlines the process for activating a gym member's account within the **Digital Logging System (DLS)**. The process begins with the **Admin** initiating the workflow from the **Admin Dashboard** and selecting the **Activate User Membership** option.

At this point, the admin inputs either the **ID or Email** of the gym member to verify their existence in the system. The system checks whether the provided ID or email exists in the **Database**. If the ID or email does not exist, the process stops, and the admin must correct the input.

If the ID or email is valid, the system retrieves the corresponding data from the database. The system then updates the user's status by changing the **"isActive"** field from false to true, effectively activating the membership. After successfully updating the status, the system sends an **Email Notification** to the gym member to inform them that their membership has been activated.

The process concludes with a confirmation that **Activation is Successful**, marking the end of the workflow. This streamlined approach ensures that user memberships are verified and activated efficiently while providing clear communication to the user about their updated status.

Scenario 5: Visit History



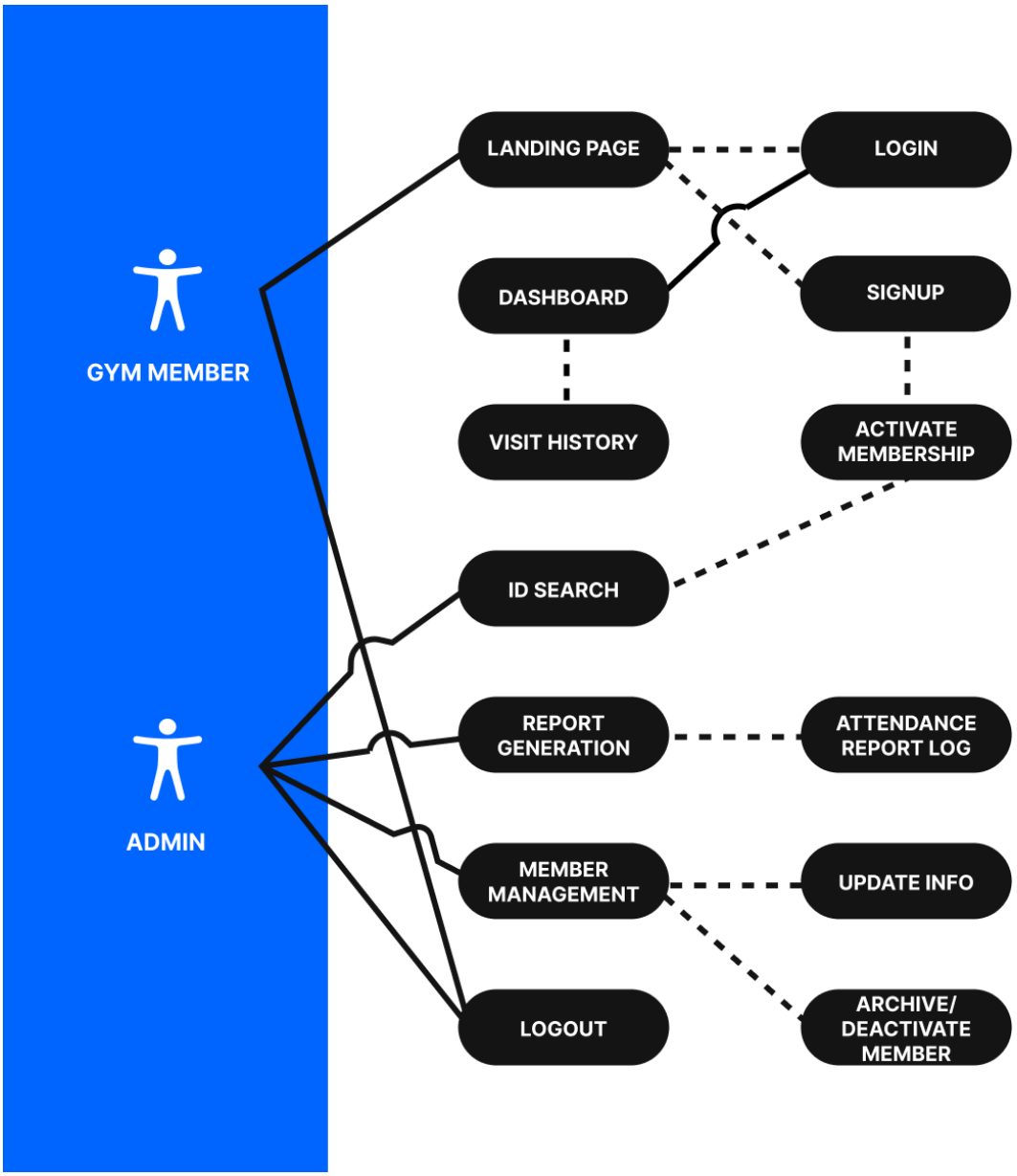
The **Scenario 5: Visit History** flowchart describes how a gym member can view their visit history within the **Buksu Fitness Gym Digital Logging System**. The process begins when the **Gym Member** accesses their **User Dashboard**. From the dashboard, the user selects the **Visit History** option to view their past gym visits.

The system initiates a **Search for Entries** by querying the **Database** to retrieve relevant data about the user's visit history. The database responds by providing the required information to the system. The system processes this data and **shows the information to the user in a table format** on the interface.

The process ends successfully, allowing the gym member to see a clear and organized summary of their visit history. This flow ensures that users have easy access to their visit records

through a seamless interaction between the **System** and the **Database**, improving transparency and user experience.

3.2.3. Scenario View (Use-Case Diagram)



This **use case diagram** represents the interaction between two primary user roles – **Gym Member** and **Admin** – within the **Digital Logging System (DLS)**. It highlights the actions (use cases) that each role can perform and the system components they interact with.

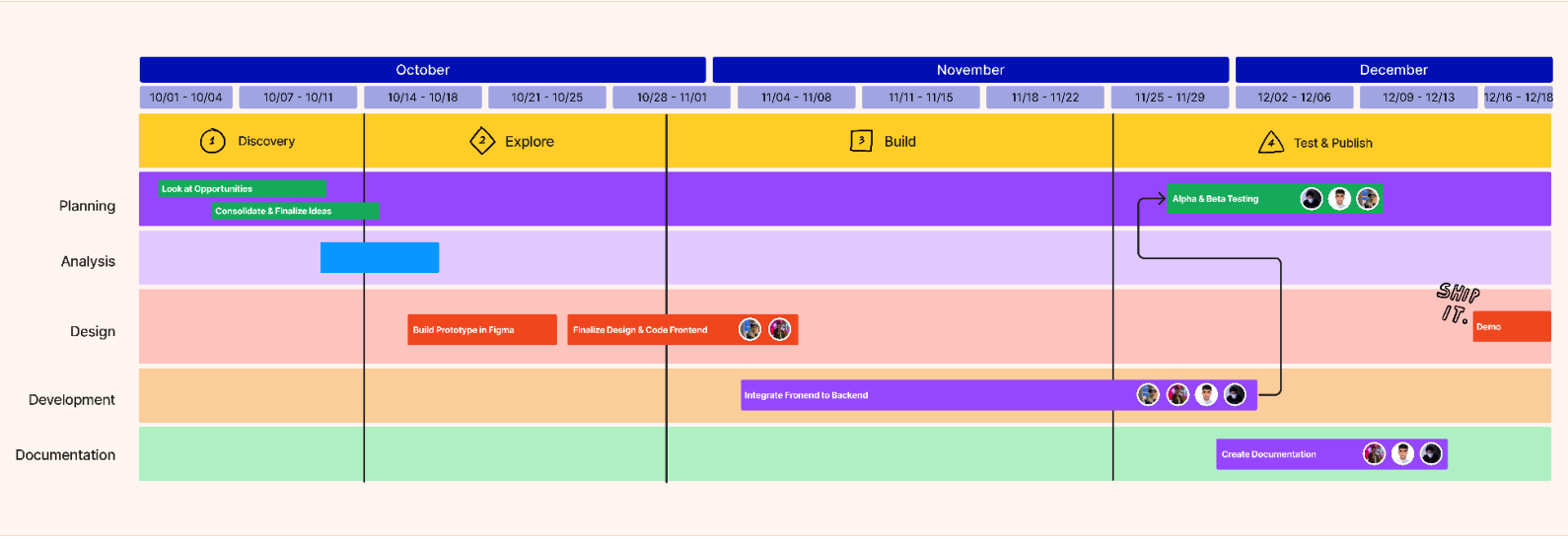
1. Gym Member Use Cases

- **Landing Page**: The starting point for gym members, allowing them to choose between the following:
    - **Login**: Members can log into their accounts.
    - **Signup**: New members can register for an account.
- **Dashboard**: Once logged in, gym members access the dashboard to view system options.
    - **Visit History**: Members can view their gym visit records.
    - **Activate Membership**: Members can request activation of their account if it's not yet active.
- **ID Search**: Members can search for specific records or IDs (e.g., for checking membership status).

2. Admin Use Cases

- **Landing Page**: Admins also begin at the landing page but have broader access.
- **ID Search**: Admins can search for user records or membership IDs for verification purposes.
- **Report Generation**:
    - **Attendance Report Log**: Admins can generate and view attendance reports based on specific parameters.
- **Member Management**:
    - **Update Info**: Admins can update user details, such as names or contact information.
    - **Archive/Deactivate Member**: Admins can deactivate memberships or archive user records as necessary.
- **Logout**: Admins can log out of the system after completing tasks.

3.4. Project Schedule



Phases and Key Tasks

1. **Planning**
    - Identifying opportunities for system improvements in gym management.
    - Consolidating and finalizing project ideas.
    - Outlining project scope, objectives, and deliverables.
    - Allocating roles and responsibilities to team members.
2. **Analysis**
    - Evaluating technical feasibility and project requirements.
    - Gathering user requirements (e.g., gym staff and administrators).
    - Reviewing existing gym processes for integration with the new system.
    - Documenting system workflows and required features.
3. **Design**
    - Building initial prototypes in Figma for user interfaces and experience.
    - Developing a database schema to manage gym records, logging, and user data.
    - Finalizing the design for the frontend interface, ensuring usability and clarity.
    - Creating wireframes for core functionalities (e.g., user log-ins, attendance tracking).
4. **Development**
    - Setting up the backend and frontend frameworks for the digital logging system.
    - Coding the frontend (UI) and integrating it with the backend database.
    - Developing APIs for logging features, user authentication, and notifications.
    - Integrating features for automated logging and data management.

- Conducting internal testing to ensure all modules work seamlessly.

5. **Testing**
    - Performing Alpha Testing to check system functionalities internally.
    - Conducting Beta Testing with some end-users to gather feedback.
    - Debugging and refining features based on user input.
6. **Documentation**
    - Writing system documentation, including user manuals and technical guides.
    - Creating a final project report with deliverables and outcomes.
    - Preparing for the Demo to showcase the system functionality.

3.5. Integration Strategies

3.5.1. API Integrations and Interactions

This section details the APIs integrated into the system, describing their functionality, input-output mechanisms, and how different software components interact with these APIs to facilitate data exchange or trigger specific actions.

- **Continue with Google API**

  Functionality: Enables users to sign in or sign up using their Google account credentials, streamlining the authentication process and ensuring secure access.

  Input: User's Google credentials (email and password) through OAuth 2.0.

  Output: Returns an access token and user information (name, email, profile picture, etc.) upon successful authentication.

  Integration:

  - Used in the Landing Page for login and signup functionalities.
  - Interacts with the User Database to store or verify user information.

  Key Interaction:

  - Frontend sends the OAuth request to the Google API.
  - API validates credentials and sends a response with user details.
  - Backend stores user details in the database for future authentication.

- **ReCaptcha API**

  Functionality: Provides an additional security layer to prevent bots from accessing the system during user interactions like login, signup, or membership activation.

  Input: User interaction response (token) on the frontend (e.g., checkbox or invisible ReCaptcha).

  Output: Validates the user as human and returns a success or failure response.

  Integration:

  - Embedded in the Landing Page and Admin Interface.
  - Blocks automated scripts and ensures legitimate usage of the system.

  Key Interaction:

  - Frontend generates a ReCaptcha challenge using the API.
  - Backend verifies the challenge token and allows further interaction if valid.

- **Gmail API**

  Functionality: Sends automated email notifications for important updates like account activation, membership confirmations, and reports.

  Input: Email metadata (recipient address, subject, body content) and authentication token.

  Output: Sends an email and returns a delivery status response (success/failure).

  Integration:

  - Triggered from the Admin Interface or during system workflows like membership activation.
  - Emails notifications for both Gym Members and Admins.

  Key Interaction:

- Backend generates email content and sends it via the Gmail API.
- API handles email delivery and provides a status response.

- **Google Sheets API**

  Functionality: Allows the system to create and update spreadsheets, used for report generation and storing attendance or membership data.

  Input: Data (e.g., member information, attendance logs) formatted as JSON, along with spreadsheet identifiers.

  Output: Updates the specified spreadsheet and returns confirmation of the update.

  Integration:

  - Integrated in Report Generation functionality/Module.
  - Enables Admins to review, share, or modify the reports.

  Key Interaction:

  - Backend sends the data to a specific Google Sheet using the API.
  - Updates are reflected in real time for external access or further analysis.

- **Google Drive API**

  Functionality: Stores a copy of the generated reports (spreadsheets) in Google Drive for secure storage and easy sharing.

  Input: File (e.g., spreadsheet in CSV or Excel format) and metadata (e.g., folder location, access permissions).

  Output: Uploads the file to Google Drive and returns the file URL or confirmation of the upload.

  Integration:

  - Triggered post Report Generation to archive the file automatically.
  - Ensures data backup and accessibility for authorized users.

Key Interaction:

- Backend uploads the report file to a predefined folder in Google Drive.
- A file link is generated for sharing or record-keeping.

- **Unsplash Photo API**

  Functionality: Fetches random, high-quality images for use as dynamic background visuals, enhancing the aesthetic appeal of the system.

  Input: Request parameters (e.g., keywords, resolution preferences) for fetching suitable images.

  Output: Returns URLs of images that match the input criteria.

  Integration:

  - Used on the Landing Page and other user-facing interfaces.
  - Adds a visually appealing element to the system without manually uploading images.

  Key Interaction:

  - Frontend requests a random photo URL via the API.
  - API responds with an image URL, which is dynamically rendered on the page.

3.5.2. Integration of Subject Components

The Buksu Fitness Gym - Digital Logging System is designed with interconnected components that collaboratively achieve the system's functionality. Each subject component is interdependent, contributing to the smooth operation of the platform. Below is a description of how these components integrate and interact:

- **User Interface (UI)**

Collaboration: Serves as the entry point for gym members and admins to interact with the system.

Integration:

- Connects to the Landing Page for login and signup.
- Interfaces with the Admin Interface for managing memberships and generating reports.
- Relies on backend processes to fetch data (e.g., visit history, membership details) and present it to users.

Dependencies:

- Authentication APIs (e.g., Continue with Google) for secure access.
- ReCaptcha API for security.
- Unsplash API for dynamic visuals.

- **User Database**

Collaboration: Acts as the central repository for storing user information, including personal details, login credentials, and membership data.

Integration:

- Receives data from the Signup and Membership Activation processes.
- Provides data to the Admin Interface for managing users and generating reports.
- Synchronizes with the Visit History Database to track user activity.

Dependencies:

- Interfaces with the Admin Interface for CRUD operations.
- Supports the Report Generation Component by providing necessary data.

- **Visit History Database**

Collaboration: Tracks and records gym visits for each member.

Integration:

- Logs visit data from the User Interface when members check in or check out.
- Provides attendance data to the Report Generation Component for creating attendance logs.

Dependencies:

- Works closely with the User Database to link visit records to specific members.
- Supports the Admin Interface in generating reports and visualizing member activity trends.

- **Admin Interface**

Collaboration: Central hub for administrators to manage gym operations, including member management and report generation.

Integration:

- Fetches and updates user data through the User Database.
- Generates attendance and membership reports via the Report Generation Component.
- Stores reports in external systems using the Google Drive API.

Dependencies:

- Relies on Google Sheets API to format and create reports.
- Interfaces with the Membership Activation Component to approve or deny membership requests.

- **Membership Activation Component**

Collaboration: Handles requests for activating gym memberships and updates user statuses.

Integration:

- Receives membership requests from the User Interface.
- Updates the User Database to reflect membership status.
- Notifies users of activation status using the Gmail API.

Dependencies:

- Requires admin approval through the Admin Interface.
- Interfaces with the Gmail API for sending notifications.

- **Report Generation Component**

  Collaboration: Automates the creation of attendance and membership reports.

  Integration:

  - Pulls data from the User Database and Visit History Database.
  - Creates spreadsheets using the Google Sheets API.
  - Saves a copy of reports to Google Drive via the Google Drive API.

  Dependencies:

  - Relies on data integrity and availability from other components (databases, admin inputs).
  - Uses external integrations for report storage and access.

- **External APIs**

  Collaboration: Enhance the system by providing secure access, automation, and dynamic content.

  Integration:

  - **Continue with Google:** Simplifies login/signup processes.
  - **ReCaptcha:** Ensures system security.
  - **Gmail API:** Sends automated email notifications.

- **Google Sheets & Drive APIs:** Manages reports and archival.
- **Unsplash API:** Supplies background images to the user interface.

Dependencies:

- Require seamless interaction between the system's backend and external platforms to ensure proper data exchange.

- **Leaderboard Component**

  Collaboration:

  - The Leaderboard component provides a competitive and motivational feature for gym members by displaying rankings based on metrics such as attendance frequency, workout completion, or other tracked activities.
  - Collaborates with the Visit History Database to retrieve attendance data and calculate rankings.
  - Displays results dynamically on the Landing Page for all users to view.

  Integration:

  - Pulls data from the Visit History Database to calculate and display rankings.
  - Utilizes the User Interface to render the leaderboard on the Landing Page.
  - Optionally, integrates with the Report Generation Component to include leaderboard data in reports for admins.

  Dependencies:

  - **Visit History Database:** Provides the attendance records required to calculate rankings.
  - **User Database:** Links attendance data to specific users.
  - **User Interface:** Ensures proper presentation of the leaderboard.

- **FAQ Component**

  Collaboration:

  - The FAQ component provides a repository of common questions and answers to assist users, reducing dependency on admin support.
  - Collaborates with the User Interface to ensure accessibility and visibility on the Landing Page.

  Integration:

  - Includes a list of preloaded FAQs stored in the User Interface layer.

  Dependencies:

  - **User Interface:** Displays FAQs in a structured and easily navigable format.

3.5.3. Data Flow and Communication

The Buksu Fitness Gym - Digital Logging System ensures seamless communication between its various components through well-defined pathways and protocols. These pathways facilitate secure and efficient data exchange, ensuring the system's reliability, scalability, and responsiveness.

3.5.3.1. Data Flow Pathways

The system employs interconnected modules and layers to streamline communication and data processing.

User Interface (UI):

- Acts as the entry point for gym members and administrators.

- Collects inputs such as login credentials, membership requests, attendance logs, and report generation requests.

Backend Logic Layer:

- Processes user requests received from the UI.
- Implements rules, validations, and workflows (e.g., login authentication, membership activation, leaderboard calculation, role-based access control).
- Coordinates between the database and external APIs.

Database Layer:

- Built on MongoDB Atlas, it stores and retrieves data such as user profiles and visit history.
- Ensures data integrity, security, and scalability for structured and unstructured data.

External APIs:

- Used for functionalities like authentication (Google Sign-In), notifications (Gmail), report generation (Google Sheets), file storage (Google Drive), and dynamic backgrounds (Unsplash).

3.5.3.1.2. Data Flow Between Modules

The system's data flow is designed to facilitate the interaction between its key modules:

- **Login and Authentication Module:**

  Input: Users enter their credentials, sign in with Google, or scan their barcode via the UI.

Processing: The backend validates the credentials against the “users” collection in MongoDB.

Output: A session token is generated, granting the user access to their dashboard.

- **Membership Activation Module:**

  Input: Gym members can sign up to the Digital Logging System through the signup UI.

  Processing: Admins approve or deny the request, and the backend updates the membership status in the users collection.

  Output: The member receives an email notification confirming the activation status.

- **Visit History Tracking Module:**

  Input: Gym members check in or check out through the UI.

  Processing: The backend logs the timestamp and updates the visitHistory collection.

  Output: The updated visit history is accessible to members and admins via their dashboards.

- **Leaderboard Module:**

  Input: Attendance data is automatically processed to calculate rankings.

  Processing: Backend logic aggregates data from the visitHistory collection and sorts it to generate the leaderboard.

  Output: The leaderboard is displayed dynamically on the landing page.

- **Report Generation Module:**

  Input: Admins request attendance logs via the Admin Interface.

  Processing: The backend compiles data from “users” and “loginhistories” collections, formats it using the Google Sheets API, and uploads the file to Google Drive.

  Output: A downloadable link to the report is provided to the admin.

3.5.3.1.3. Communication Mechanisms

The system uses secure and efficient communication mechanisms for data transmission and processing:

Data Transmission Protocols:

- HTTPS ensures encrypted communication between the User Interface and the backend.
- RESTful API requests are used for communication with external services and MongoDB.

Message Exchange Formats:

- JSON is the primary format for data exchange between the UI, backend, and external APIs.

Error Handling:

- Input validation is implemented at the UI and backend levels to ensure data completeness and correctness.

- Errors (e.g., API failures, invalid inputs) are logged and displayed to users with appropriate messages.

3.5.3.1.4. Data Processing Details

The system handles data in four key stages:

1. **Data Input:**
    - Captured via web forms or user actions, including login credentials, check-ins, and admin commands.
    - Inputs are validated for accuracy, completeness, and format.

2. **Data Transmission:**
    - Securely transmitted between the frontend and backend using HTTPS and RESTful APIs.
    - Authentication tokens (e.g., OAuth for Google APIs) are included for external API requests.

3. **Data Storage:**
    - Stored in MongoDB Atlas collections (e.g., users, loginhistories).
    - The NoSQL structure supports both structured (user profiles) and unstructured (visit history) data.

4. **Data Output:**
    - Displayed on the UI (e.g., visit history, leaderboard) or exported as reports (e.g., Excel, PDF).
    - Output formats are generated through external APIs (e.g., Google Sheets).

3.5.3.1.5. System Data Exchange and Handling

The system employs the following principles to ensure effective data exchange and handling:

User Authentication and Authorization:

- Only authenticated users can access the system.
- Permissions are managed to restrict admin-specific functionalities (e.g., report generation, user management).

Real-Time Updates:

- Membership activations, attendance logs, and leaderboard rankings are updated in real time.

Audit Trails:

- Logs are maintained for all critical actions (e.g., check-ins, report generation) to ensure transparency and traceability.

Scalability:

- MongoDB Atlas supports the dynamic scaling of data as the number of gym members grows.

Chapter 4
SECURITY POLICY

4.1. IS Policy Document

The Information Security Policy for the BukSU Fitness Gym - Digital Logging System outlines the principles, practices, and guidelines to ensure the confidentiality, integrity, and availability of the system's data and services.

4.1.1. Hardware Security Policy

The purpose of this hardware security policy is to protect all hardware assets involved in the system, ensuring their availability, reliability, and secure operation.

Scope:

This policy applies to all physical hardware used in the system, including barcode scanner, and any other equipment used for the gym.

Policy:

1. **Access Control.** Only authorized personnel are permitted to access, configure, or maintain hardware components.
2. **Maintenance/Updates.** The hardware must be regularly inspected.
3. **Environmental Protection.** The hardware must be stored and operated in environments with proper temperature control, and humidity levels to prevent damage.
4. **Disposal.** When hardware is no longer in use, sensitive data must be securely wiped before disposal.
5. **Physical Security.** All hardware components must be located in secure, access-controlled areas to prevent unauthorized physical access.

The hardware security policy restricts access to authorized personnel, ensuring proper maintenance, configuration, and secure operation of all components. Hardware must be stored in controlled environments, with sensitive data securely erased before disposal and physical security measures in place to prevent unauthorized access.

4.1.2. Software Security Policy

Scope:

The objective of the Software Security Policy is to ensure the secure operation, maintenance, and usage of all software components in the BukSU Fitness Gym Digital Logging System. It aims to prevent unauthorized access, data breaches, and vulnerabilities that could compromise the system's functionality and user trust.

Policy:

The Software Security Policy mandates that all software used in the system must be regularly updated to eliminate vulnerabilities and ensure compatibility with the latest security standards. Access to software configurations and administrative tools is restricted to authorized individuals, and user authentication must use secure protocols, such as Google OAuth2. Sensitive data handled by the software must be encrypted both in transit and at rest using industry-standard encryption methods. Regular security audits and vulnerability assessments are required to identify and mitigate potential risks. Unauthorized or outdated software must be promptly removed to maintain system security and integrity.

4.1.3. Network Communications Policy

Objectives:

The objective of the Network Communications Policy is to ensure secure, reliable, and efficient communication across the network infrastructure supporting the system.

Policy:

The policy of the network communication requires that all data transmitted within the system is encrypted using industry-standard protocols such as Transport Layer Security (TLS) to protect against interception and tampering. Regular monitoring of network traffic is conducted to identify and mitigate potential threats, including unauthorized access attempts or malicious activity. Updates to network hardware and firmware must be applied promptly to address vulnerabilities. In case of network disruptions, contingency plans and redundant communication pathways must be in place to ensure uninterrupted system operations. Unauthorized devices or unsecured connections are strictly prohibited to maintain the integrity and security of the network.

4.1.6. People Policy

Scope:

The objective of the People Policy is to ensure that all personnel interacting with the system understand their roles and responsibilities, follow security protocols, and contribute to the secure and efficient operation of the system.

Policy:

The People Policy states that everyone regardless of user, administrator, or staff should follow defined security protocols and ethical guidelines for their proper utilization of the system. Authorized personnel are supposed to keep the integrity and confidentiality of the data and thus not to share the login credentials or to abuse the access they received. Administrators, in turn, must acquire technical education such as system operations and incident response, as well as good data processing. The main thing that admin plays is the operation of the system which includes deleting bad data and removing attacked files to back up its existence. Users should observe the instructions of the system such as secure login practices and appropriate use of barcodes for attendance logging.

4.2. IT Security Management Framework

The IT Security Management Framework for the BukSU Fitness Gym Digital Logging System establishes a structured approach to managing and safeguarding the system's information assets, ensuring the confidentiality, integrity, and availability of data.

4.2.1. Legal Compliance

The BukSU Fitness Gym Digital Logging System adheres to relevant laws, regulations, and industry standards to ensure compliance and avoid legal repercussions.

1. **Data Privacy:** Applicable data protection laws, such as the Data Privacy Act of 2012 (RA 10173), ensures the confidentiality, integrity, and availability of sensitive user information.
2. **Cybersecurity Standards:** The system follows international best practices for information security management, such as those outlined in ISO/IEC 27001.
3. **Audit and Monitoring Standards:** Follows recognized practices like **COBIT (Control Objectives for Information and Related Technologies)** for auditing and monitoring system activities, ensuring operational transparency and accountability.

4. **Copyright:** software components, APIs, and frameworks used in the system are properly licensed and accredited, ensuring compliance with copyright and intellectual property laws.
5. **Access Control Compliance:** It ensures the implementation of secure access protocols, aligned with institutional policies, to restrict unauthorized access and safeguard sensitive system information.

4.2.2. Security Governance Standard

BukSU Fitness Gym Digital Logging System establishes the framework for managing and directing security efforts within the organization.

1. **Roles and Responsibilities:** It defines the security related roles, such as data owners , and end users, with clear accountability for maintaining security protocols.
2. **Risk Management:** Implement a risk assessment process to identify, evaluate, and mitigate security risks regularly.
3. **Policy Alignment:** Ensure all security policies, such as access control, data protection, and incident response, align with university goals and relevant regulations (e.g., Data Privacy Act of 2012, GDPR).
4. **Compliance:** Align the system with international standards like **ISO/IEC 27001** for information security management and **NIST Cybersecurity Framework** for best practices in risk assessment and mitigation.

4.3. Network Security Testing

4.3.1. Network and Server Profiling

Network and server profiling involves identifying and documenting the key components, configurations, and operational parameters of the network infrastructure and server environment used by the BukSU Fitness Gym Digital Logging System.

Network Profiling:

Network Topology: The system uses a client-server architecture where users access the system through web browsers connected to a centralized cloud-based server.

Firewall and Access Control: Firewalls are implemented to filter traffic and block unauthorized access.

Bandwidth Management: The Network is designed to filter traffic and block

Server Profiling:

Web Server: The application is hosted on a Node.js

Database Server: MongoDB Atlas NoSQL database is used to store attendance records, and system configuration.

Backup Server: Backup server of database records is performed.

Authentication: Google OAuth2 service handles user authentication.

4.3.2. CVSS (Common Vulnerability Scoring System)

The Common Vulnerability Scoring System (CVSS) is a standardized framework used to assess and quantify the severity of security vulnerabilities within a system. The CVSS framework can be used to assess vulnerabilities in the system, such as weak authentication protocols or unpatched software.

- **Vulnerability:** Outdated Software Library

  Base Score: 7.5 (High) due to the potential for exploitation.

  Temporal Score: 6.5, adjusted based on the availability of patches.

  Environmental Score: 5.5, as the specific impact may vary based on system architecture.

- **Vulnerability:** Insufficient Barcode Validation

  Base Score: 6.0 (Medium) due to limited exploitability.

  Temporal Score: 5.8, considering the ease of mitigation through updates.

  Environmental Score: 7.0, as unauthorized gym access poses a higher risk in this context.

Benefits of CVSS in BukSU Fitness Gym Digital Logging System

Using CVSS ensures a structured and consistent evaluation of vulnerabilities, helping system administrators prioritize fixes and allocate resources effectively. It aligns with best practices for risk management, ensuring the BukSU Fitness Gym Digital Logging System remains secure and resilient against evolving threats.

4.4. Solution Design

4.4.1. Policy Design

Policy Design for the system establishes structured guidelines to ensure efficiency, security and compliance with institutional and regulatory standards.

Access Control Policy: Only authorized users can access the system.

Data Protection Policy: This includes attendance records, encrypted storage and transmission.

User Policy: Users are required to follow secure login practices, protect their credentials, and use the system responsibly.

Hardware Policy: The hardware used in the system is stored in secure, access-controlled areas.

Software Policy: All software components are regularly updated to address vulnerabilities.

4.4.2. Logical Design

The logical design of the BukSU Fitness Gym - Digital Logging System defines the conceptual framework and operational flow of the system. This design is intended to provide a clear understanding of the relationships, processes, and data structures, ensuring smooth system implementation and management.

System Architecture

The system is designed with a modular structure to ensure scalability, maintainability, and efficient data flow. The primary components include:

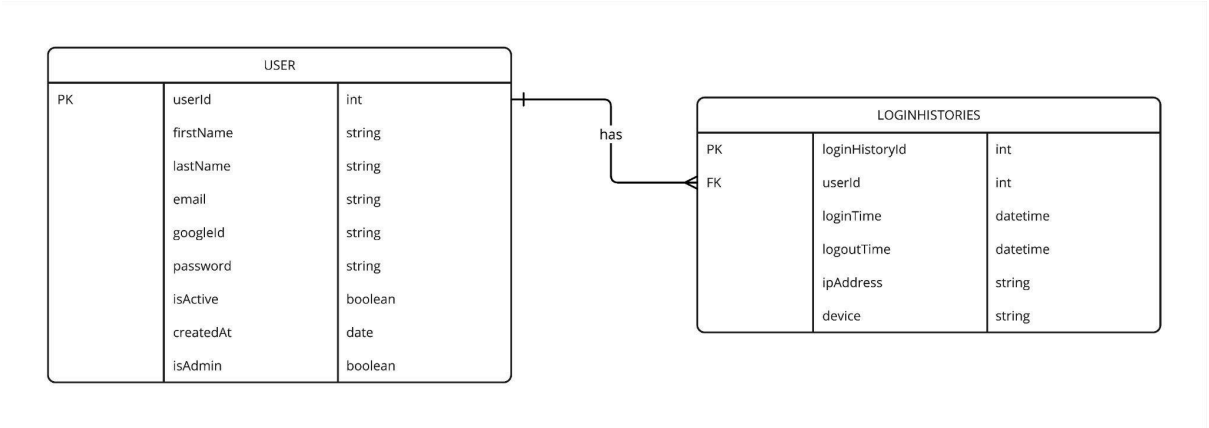
1. **Users**
    - **Members:** Log and view attendance & personal profiles.
    - **Staff/Admin:** Manage user accounts, monitor attendance, and generate reports.

2. **Core Modules**
    - **Authentication Module:** Verifies login credentials for users.
    - **Attendance Logging Module:** Handles check-ins and check-outs.
    - **Database Management Module:** Manages and stores user and attendance data.
    - **Admin Panel:** Provides functionalities for user management, such as activation and profile updates.

3. **External Systems**
    - Optional integration with biometric or card-based attendance systems.

Entity-Relationship Diagram (ERD)

The core entities and their relationships in the database schema are:



Entities:

1. user:
    - Attributes:
        - **firstName (String):** User's first name.
        - **lastName (String):** User's last name.
        - **email (String):** Unique identifier for the user.
        - **googleId (String):** Google-based login identifier.
        - **password (String):** Optional hashed password for manual logins.
        - **isActive (Boolean):** Indicates whether the member is activated or not.
        - **createdAt (Date):** Timestamp of account creation.
        - **isAdmin (Boolean):** Identifies admin users.

    - Relationships:
        - One user has multiple loginhistories entries.

2. loginhistories:
    - Attributes:
        - **userId (ObjectId):** References the associated User.
        - **loginTime (Date):** Timestamp of login.
        - **logoutTime (Date):** Timestamp of logout.
        - **ipAddress (String):** IP address during login.
        - **device (String):** Device details during login.
    - Relationships:
        - Many loginhistories entries belong to one User.

Data Flow

Level 1: Key System Processes

1. **Login Process:**
    - **Input:** User credentials (email/password or Google ID).
    - **Process:**

    - System validates credentials using the User collection.
    - If valid, creates a new LoginHistory entry with loginTime, ipAddress, and device.
- **Output:** Login success or failure message.

2. **Attendance Logging:**
    - **Input:** ID scan or manual entry.
    - **Process:**
        - Retrieves the user record.
        - Updates the LoginHistory collection with the current timestamp.
    - **Output:** Attendance record confirmation.

3. **Member Activation:**
    - **Input:** Admin selects a user marked as inactive.
    - **Process:**
        - System toggles the isActive field in the User collection to true.
    - **Output:** Confirmation of activation and updated user status.

4. **Update Member Details:**
    - **Input:** Admin selects a user and updates the firstName or lastName.
    - **Process:**
        - System retrieves the user's record and applies changes.
    - **Output:** Confirmation of updated details.

5. **Report Generation:**
    - **Input:** Admin selects attendance report range (e.g., monthly, weekly, or custom), then enters an email address (optional).
    - **Process:**
        - System retrieves relevant data from User and LoginHistory collections.

        - Compiles and formats the report. If an email has been entered, it will send an invite to the Google Sheet.
    - **Output:** A Google Sheet link with the report.

Logical Processes

1. **Member Activation in Admin Panel:**
    - Admin views a list of inactive members (isActive: false).
    - Selects a member and toggles isActive to true.
    - Saves the updated status.
2. **Update Member Details:**
    - Admin selects a user and edits the firstName or lastName.
    - The changes are saved in the User collection.
3. **Log Attendance:**
    - Members log in through manual entry or device scan.
    - Attendance is recorded in the LoginHistory collection.
4. **Admin Dashboard:**
    - Admin manages user accounts, views attendance logs, and generates reports.

Logical Interfaces

1. **Member Login Interface:**
    - **Inputs:** Email/password, Barcode Login, or Continue with Google.
    - **Outputs:** Login confirmation or error message.
2. **Admin Panel:**
    - **Features:**
        - View and filter users by status (active/inactive).
        - View current number of gym members inside the building.
        - View the most recent signup.
        - Activate members with a toggle.
        - Edit member details (first name, last name).
        - Generate and download activity reports.
3. **Attendance System:**

- Inputs: Device scans or manual entries.
- Outputs: Real-time attendance confirmation.

4.4.3. Physical Design

The Physical Design section describes the actual implementation of the system, including the technical environment, database storage details, and deployment setup. This section focuses on how the system is implemented in the real world, based on the logical design.

System Architecture

The BukSU Fitness Gym - Digital Logging System is implemented using the following components:

1. **Frontend:**
    - **Technology:** React.js with Bootstrap.
    - **Purpose:** Provides a user-friendly interface for members and administrators to interact with the system.
    - **Hosting:** Runs locally on a development server (localhost). If deployed, it will be hosted on a cloud-based platform such as Vercel.

2. **Backend:**
    - **Technology:** Node.js with Express.js framework.
    - **Purpose:** Handles business logic, API endpoints, and integration with the database.
    - **Hosting:** Executes on a local Node.js server (http://localhost:5000). If deployed, it will be hosted on AWS, Heroku, Microsoft Azure, or similar cloud platforms for reliability.

3. **Database:**
    - **Technology:** MongoDB (NoSQL database).
    - **Purpose:** Stores user information, attendance logs, and other system data.
    - **Hosting:** MongoDB Community Edition installed locally or through a MongoDB Atlas development cluster.

4. **Authentication:**
    - **Technology:** JSON Web Tokens (JWT) and OAuth 2.0 (for Google sign-in).
    - **Purpose:** Ensures secure login and session management for users.

5. **Third-Party Tools:**

    - **Google Sheets**
        - **Purpose:** For exporting and managing user reports, attendance logs in a spreadsheet format for easy sharing and collaboration.
        - **Integration:** API integration for automatic data synchronization or exporting reports.

    - **Google Drive**
        - **Purpose:** For storing exported files, such as attendance summaries, reports, and other gym-related documents, in a centralized and secure location.
        - **Integration:** API integration for automated file uploads and organization.

    - **ReCaptcha**
        - **Purpose:** Ensures bot protection during user login, registration, and form submissions to maintain system security.
        - **Integration:** Implemented on login and registration pages as an additional verification step.

- **Unsplash API**
    - **Purpose:** For dynamically fetching high-quality images, such as for enhancing the design of the admin dashboard or member profile sections.
    - **Integration:** API calls to fetch relevant images based on predefined themes or categories.

- **Gmail API**
    - **Purpose:** For sending automated emails, such as registration confirmations, password reset links, and system notifications to users.
    - **Integration:** Configured to work with the backend for seamless email delivery.

- **Google OAuth**
    - **Purpose:** For secure and seamless user authentication, especially when integrating Google sign-in or other third-party login systems.
    - **Integration:** Used to manage user access via Google or other OAuth-supported platforms.

Database Design

The database is implemented using MongoDB with the following collections:

1. **users Collection:**
    - Stores user details and credentials.
    - Indexed on email for fast retrieval.
2. **loginhistories Collection:**
    - Stores login and logout details for each user.
    - Indexed on userId for quick access.

Deployment Environment

1. **Development Environment:**
    - **Frontend:** React development server on localhost:3000.
    - **Backend:** Node.js server running on localhost:5000.
    - **Database:** MongoDB Community Edition hosted locally.
2. **Version Control:**
    - GitHub repository for managing source code.
    - Continuous Integration/Continuous Deployment (CI/CD) pipelines for automated builds and deployments.

Security Measures

1. **Authentication:**
    - Use of JWT for token-based authentication.
    - Google OAuth for social login.
2. **Data Protection:**
    - Passwords hashed using bcrypt.
    - Encrypted communication between frontend, backend, and database using HTTPS and SSL/TLS.
3. **Access Control:**
    - Role-based access for users (e.g., Admin vs. Member).
    - Database access restricted through environment variables and IP whitelisting.

System Hardware Requirements

Since the BukSU Fitness Gym - Digital Logging System is hosted and run locally during development and testing, the hardware requirements focus on ensuring a smooth and efficient setup on a single machine.

Minimum Hardware Requirements

1. **Processor:**
    - Dual-core processor (e.g., Intel Celeron or equivalent).
2. **Memory:**
    - 4 GB RAM (8 GB recommended for better performance).
3. **Storage:**
    - At least 20 GB of free disk space.
    - Additional storage for MongoDB database files if stored locally.
4. **Graphics:**
    - Standard integrated graphics (not graphics-intensive).
5. **Network:**
    - Local network connection for API testing (localhost, 127.0.0.1).

Software Requirements

1. **Operating System:**
    - Windows 10 or later, macOS, or Linux.
2. **Development Tools:**
    - **Node.js** (v14 or later): For running the backend.
    - **MongoDB Compass:** For local database storage.
    - **npm**: For package management.
3. **Frontend Requirements:**
    - **React.js:** For building and running the user interface.
    - **Browser:** Google Chrome or equivalent for frontend testing.
4. **Other Tools:**
    - **Postman:** For API testing.
    - **VS Code** or any IDE: For coding and debugging.

Local Deployment Setup

1. **Backend:**
    - Run the backend using Node.js with a local server (http://localhost:5000).
2. **Frontend:**
    - Run the React development server locally (http://localhost:3000).

3. **Database:**
    - Host the MongoDB database locally using MongoDB Community Edition.

4.4.4. IS implementation Design (if applicable)

The IS Implementation Design focuses on practical steps to integrate security

protocols:

Authentication Setup:

Configure Google OAuth to provide a secure and user-friendly login experience. This setup ensures that only authorized users can access the system while minimizing the risks of unauthorized access.

Encryption Protocols:

Implement SSL/TLS protocols for securing data transmission between users, servers, and the database. Use AES-256 encryption for encrypting sensitive data stored in the MongoDB Atlas database to safeguard confidentiality and integrity.

Database Configuration:

Set up the MongoDB Atlas cloud database with appropriate security settings, including IP whitelisting and role-based access control. Optimize the database for efficient query execution and real-time data updates.

4.4.5. Intrusion and Detection Systems

Intrusion and Detection Systems (IDS) are implemented in the system to monitor, detect, and respond to unauthorized access or malicious activities.

1. **Network Intrusion Detection System (NIDS):** Monitors network traffic for suspicious activities, such as unauthorized access attempts or unusual traffic patterns, and generates alerts for potential threats.
2. **Host-Based Intrusion Detection System:** Tracks activity on individual servers, including file integrity checks and login behavior, to detect unauthorized changes or malicious activity.

3. **Log Monitoring:** Analyzes system and application logs continuously to identify irregularities or security events, ensuring comprehensive auditing of all activities.

4.4.6. Secure Process Model

The Secure Process Model outlines the structured approach for designing, developing, deploying, and maintaining the BukSU Fitness Gym Digital Logging System with a focus on security at every stage.

During the planning phase, we establish security requirements to guarantee that the system functions securely and adheres to relevant data protection. These requirements encompass strong data encryption, secure user authentication through Google OAuth2, and role-based access control. By incorporating these security measures from the outset, we can identify and address potential vulnerabilities before development starts, creating a solid foundation for the system's security.

The system architecture is designed with a strong emphasis on security. The database is organized to maintain strict data integrity, ensuring that sensitive information cannot be accessed or altered without proper authorization. Furthermore, we apply the principle of least privilege through role-based access control, limiting system access to authorized personnel only. This strategy reduces potential attack surfaces and bolsters the overall security of the system.

Prior to deployment, the system undergoes security testing to uncover and resolve vulnerabilities. We conduct vulnerability scanning and penetration testing to mimic potential attacks and confirm that the system can endure real-world threats. Code reviews are performed to ensure compliance with secure coding standards, while functional and performance testing verifies that the system operates as intended without compromising security.

4.5. Business Continuity Plan

The Business Continuity Plan (BCP) for the BukSU Fitness Gym Digital Logging System is designed to maintain system operations with minimal interruption during unforeseen events, including natural disasters, cyberattacks, or hardware malfunctions. This plan features automated daily backups of essential data, which are securely stored offsite to facilitate quick recovery in the event of data loss.

4.5.1 Incident Response Plan

The Incident Response Plan (IRP) for the BukSU Fitness Gym Digital Logging System outlines the steps to detect, respond to, and mitigate security incidents effectively.

Detection and Identification: Continuous monitoring detects unusual behavior or potential security incidents, generating alerts for immediate review.

Containment: Steps are taken to isolate affected components, such as locking unauthorized accounts or blocking suspicious traffic, to prevent further damage.

Investigation and Analysis: The root cause and extent of the incident are analyzed using detailed system logs and other diagnostic tools.

Remediation and Recovery: Corrective measures, such as patching vulnerabilities or restoring data from backups, are implemented to bring the system back to full functionality.

Communication: Stakeholders, including administrators and authorities, are notified as required to maintain transparency and compliance.

Post-Incident Review: The incident is reviewed to identify lessons learned and improve future response strategies.

4.5.2. Disaster Recovery Plan

The Disaster Recovery Plan (DRP) for the BukSU Fitness Gym Digital Logging System ensures the system's ability to recover and resume operations following significant disruptions, such as natural disasters, cyberattacks, or hardware failures.

Data Backups and Recovery:

- Complete critical data are automatically daily backed up securely off site.
- Recovery processes are regularly tested to ensure timely and accurate recovery of all backups.

Redundancy and Failover

- Infrastructure such as MongoDB Atlas provide redundancy in cloud, this prevents single point failure.
- Failover is automated redirecting traffic to the system in case of outage.

Recovery Time Objectives and Recovery Point Objectives.

- The system aims for an RTO of under 4 hours, ensuring operations resume promptly.
- The RPO is set to daily backups, limiting potential data loss to a maximum of 24 hours.

Emergency Response Protocols:

- Designated personnel are tasked with executing the DRP, including restoring systems, assessing damages, and communicating with stakeholders.

Post-Recovery Review:

- After recovery, a thorough review is conducted to assess the effectiveness of the plan and implement improvements for future incidents.

Chapter 5
CONCLUSION

5.1. Reflection on Architecture Decisions

The architectural design of the BUKSU Fitness Gym Digital Logging System was shaped by a combination of technical requirements, user needs, and project constraints. This reflection evaluates the successes and shortcomings of the decisions made to provide insights into their implications on system performance, scalability, and overall functionality.

One of the key successes was the decision to implement **Google Authentication (OAuth2)** for user login. This choice enhanced security, simplified user access, and leveraged a widely trusted platform, reducing the need to manage sensitive credentials directly. Additionally, replacing biometric technology with **barcode-based sign-ins** proved to be a more practical and cost-effective solution for the system, ensuring reliability and ease of use for gym members.

Despite these successes certain challenges appear during the implementation. Early issues included inaccurate visit counts and user ID mismatches that caused delays in loading visit histories. These were addressed by refining database queries and enhancing error handling mechanisms. However, these setbacks revealed the need for more rigorous testing and validation processes during development.

The primary weakness identified was a lack of foresight in accounting for potential edge cases, particularly in handling null or unexpected data (e.g., user IDs). While these were resolved, they delayed progress and highlighted areas where the architecture could be more robust. Incorporating more extensive error handling and automated testing earlier in the development cycle would have mitigated these issues.

Reflecting on the architectural decisions for the **BUKSU Fitness Gym Digital Logging System** reveals a journey of adaptation and growth. While some decisions required significant adjustments to align with constraints, the focus on scalability, security, and user experience ensured a successful outcome. The lessons learned provide valuable guidance for future projects, emphasizing the importance of flexibility, thorough planning, and continuous improvement in system design.

5.2. Difficulties Encountered

The development of the **BUKSU Fitness Gym Digital Logging System**, several challenges were encountered, stemming from both technical limitations and the evolving scope of the project. One significant difficulty arose during the integration of **Google Authentication (OAuth2)**, where initial issues included configuring redirect URIs and managing token-based authentication securely. These complexities required additional research and testing, delaying other development tasks.

Another challenge was ensuring the accurate display of visit counts and user histories. Early on, mismatched user IDs and null data values caused features like "view visit history" to fail or load incorrectly. Addressing these issues involved revising database queries and implementing more robust error handling, which introduced delays and underscored the need for better validation processes.

The transition from **biometric authentication** to a **barcode-based sign-in system** also posed difficulties. While this decision resolved cost and feasibility concerns, it required significant changes to the system's architecture and user workflows. This adjustment, while necessary, forced the abandonment of initial plans for biometric capabilities, limiting the system's functionality compared to the original vision.

These challenges required the team to adapt quickly, revise plans, and explore alternative solutions. While some planned features were modified or abandoned due to these constraints, the process fostered valuable learning and problem-solving skills, which contributed to the project's ultimate success and provided insights for future endeavors.

5.3. Improvements and Extensions

The **System** has demonstrated its effectiveness in managing gym member sign-ins and visit history tracking, yet there are areas that could benefit from improvement and further development. A key area for enhancement is the system's **error handling and validation mechanisms**, particularly in managing edge cases, such as null or mismatched data entries. Strengthening these aspects would improve the system's reliability and user experience by reducing instances of data-loading delays or errors.

Another area for improvement involves the **user interface and experience**. While functional, the interface could be optimized to provide a more seamless and visually engaging interaction, especially

for features like viewing visit history. Adding more intuitive navigation and providing instant feedback to users could enhance satisfaction and ease of use.

From a performance perspective, optimizing **database queries** and incorporating automated backups or replication features in **MongoDB Atlas** would improve scalability and data integrity as the system grows. Additionally, enhancing reporting capabilities to include visual analytics, such as charts for visit trends, would add value for administrators.

Several deviations from the original project plan also highlight opportunities for future extensions. The most significant was the shift from **biometric authentication** to **barcode-based sign-ins** due to cost and feasibility constraints. Future iterations of the system could explore hybrid authentication options, incorporating both methods to offer flexibility and enhanced security.

5.4. Conclusion

The **System** was both a challenging and rewarding experience, offering valuable insights into real-world system implementation and problem-solving. The project presented a fair balance of complexity, particularly due to the integration of modern authentication systems, such as **Google Authentication (OAuth2)**, and the transition to a **barcode-based sign-in** mechanism. These aspects required extensive research, meticulous debugging, and adaptability to overcome technical and practical challenges.

From the problem, we understood the critical importance of balancing **user convenience**, **system reliability**, and **scalability** within the constraints of time and resources. The system's dynamic requirements, such as accurately tracking user visit histories and managing authentication securely, deepened our understanding of designing efficient database architectures and robust user workflows. Additionally, the challenges encountered highlighted the necessity of rigorous testing and the value of flexibility when deviations from initial plans become unavoidable.

Throughout the project, we gained new skills in areas such as **cloud database management**, **error handling**, and **agile development methodologies**. These learnings not only enhanced the project but also provided a solid foundation for tackling similar projects in the future. If more time were available, we would have focused on implementing advanced features like **real-time analytics**, **hybrid**

authentication systems, and seamless integration with third-party fitness applications. These enhancements would further elevate the system's functionality and user experience.

For future researchers and developers working on similar projects, we recommend emphasizing thorough initial planning and **comprehensive testing strategies** to anticipate potential issues with data handling and user authentication.

References

Collegianer Publication. (n.d.). [Post about BukSU fitness gym digital logging system]. Facebook. Retrieved December 11, 2024, from https://www.facebook.com/collegianerpub/posts/717240440420340/

International Organization for Standardization. (2022). ISO/IEC 27001: Information security management systems. Retrieved from https://www.iso.org/isoiec-27001-information-security.html

National Institute of Standards and Technology. (2020). NIST special publication 800-53: Security and privacy controls for information systems and organizations. U.S. Department of Commerce. Retrieved from https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-53r5.pdf

OWASP Foundation. (n.d.). OWASP secure coding practices quick reference guide. Retrieved from https://owasp.org/www-project-secure-coding-practices-quick-reference-guide/

National Institute of Standards and Technology. (n.d.). Framework for improving critical infrastructure cybersecurity. U.S. Department of Commerce. Retrieved from https://www.nist.gov/cyberframework

National Privacy Commission. (2012). Data Privacy Act of 2012 (RA 10173). Retrieved from https://privacy.gov.ph/data-privacy-act/

General Data Protection Regulation (GDPR). (2018). General Data Protection Regulation (EU). Retrieved from https://gdpr-info.eu/

ISACA. (2019). COBIT 2019 framework: Governance and management objectives. Retrieved from https://www.isaca.org/resources/cobit

Microsoft Corporation. (n.d.). Azure network design principles. Retrieved from https://learn.microsoft.com/en-us/azure/architecture/framework/networking/

FIRST.org. (2019). Common vulnerability scoring system (CVSS) version 3.1: User guide. Retrieved from https://www.first.org/cvss/

International Organization for Standardization. (2022). ISO/IEC 27002: Code of practice for information security controls. Retrieved from https://www.iso.org/standard/27002.html

IEEE. (n.d.). Software development lifecycle (SDLC). Retrieved from https://www.computer.org/education/sdlc

Appendices

These are the list of supporting documents that you need to provide:

Appendix A: Summary of Achievements

We successfully digitized and automated gym attendance and membership management processes at **Bukidnon State University**. The following key achievements were accomplished:

1. **Streamlined User Registration and Login**
    - Users can now sign up seamlessly using their BukSU email or Google accounts, with options to manually enter credentials or scan ID barcodes.
    - A generated barcode provides quick and efficient access to the gym facilities.
2. **Efficient Attendance Logging**
    - The system automatically logs **time-in** and **time-out** for every user visit, recording attendance data with precision.
    - Weekly visit limits (e.g., **2-3 visits per week**) are enforced to ensure fair gym access for all members.
3. **Comprehensive Admin Panel**
    - Administrators can activate members, update member details, and manage signups through an intuitive interface.
    - Tools to **add, edit, archive, or archive members** simplify gym membership management.
4. **Attendance Report Generation**
    - The system generates detailed attendance reports based on date ranges, which can be viewed or sent via email.
    - Reports include key data such as **member name**, **email**, **time-in**, **time-out**, and **duration spent**, aiding in efficient monitoring and analysis.
5. **Enhanced User Experience**
    - Users can view their **visit history**, track the number of weekly visits, and receive notifications for successful actions (e.g., membership activation).
    - The system provides a user-friendly interface with clear prompts and visual feedback.
6. **Real-Time Updates**
    - The **Current Gym Users** feature displays live gym occupancy, updated every 30 seconds, ensuring real-time transparency.

Appendix B: Member's Contributions and Roles

The development of the Digital Logging System was a team effort, with each member playing an important role to complete the project successfully. Everyone contributed their skills and worked together to make sure all parts of the system were planned, built, and tested properly. Below is a summary of each member's contributions:

1. **Programmer**

   The programmer handled the coding and development of the system. They worked on key features like the login system (Google OAuth and barcode scanning), attendance tracking, and membership activation. They also set up the database to store user information and visit logs, making sure the system worked smoothly.

2. **System Designer**

   The system designer created the overall structure of the system. This included making diagrams like flowcharts and database designs to guide the development. They made sure the system was organized and easy to understand for everyone working on it.

3. **Coordinator and Documentation Lead**

   The coordinator (leader) made sure the team stayed on track and finished tasks on time. They worked on the documentation, including user guides and reports, and helped during testing. The coordinator also kept the team organized and ensured everyone communicated well to achieve the project goals.

Each team member’s hard work and unique contributions were key to completing the system. By working together and supporting each other, the team successfully created a functional and user-friendly system that meets the needs of the BukSU Fitness Gym. This project showed how teamwork, clear communication, and shared effort can lead to great results.

Appendix C: API Documentation

The Digital Logging System (DLS) uses internal APIs to handle core functionalities like user management, attendance tracking, and report generation. These APIs ensure smooth operations, real-time updates, and improved user experience for both gym members and administrators. Below is an overview of the main APIs:

1. User Attendance API

Role: Logs and tracks user visits to the gym.
Functionality:

- Records time-in and time-out for each user visit.
- Calculates the duration of gym usage.

Benefits:

- **Accuracy**: Ensures accurate tracking of user attendance.
- **Real-Time Updates**: Monitors current gym usage and updates visit histories automatically.

2. User Management API

Role: Manages user accounts within the system.
Functionality:

- Handles user registration, login, and activation.
- Allows updating or archiving user information as needed.

Benefits:

- **Flexibility**: Enables administrators to manage user details efficiently.
- **Security**: Integrates secure authentication for user verification.

3. Visit History API

Role: Retrieves attendance history for individual users.
Functionality:

- Provides detailed logs of each user's visits, including time-in, time-out, and visit durations.
- Allows users to view and track their visit history.

Benefits:

- **Transparency**: Users can access their attendance records.
- **Convenience**: Simplifies monitoring for both users and administrators.

4. Leaderboard API

Role: Displays a ranking of gym members based on their visit frequency.
Functionality:

- Compiles data on user visits to generate leaderboards.
- Updates rankings in real time.

Benefits:

- **Engagement**: Encourages gym members to use the facility regularly.
- **Recognition**: Highlights the most active users.

5. Report Generation API

Role: Generates attendance reports for administrative use.
Functionality:

- Produces reports based on selected date ranges (e.g., daily, weekly, monthly).
- Provides the option to email reports to administrators.

Benefits:

- **Efficiency**: Simplifies the process of tracking attendance over time.
- **Organization**: Allows administrators to maintain accurate records for analysis and reporting.

6. System Monitoring API

Role: Monitors real-time gym usage.
Functionality:

- Tracks the number of users currently inside the gym.
- Provides administrators with real-time updates.

Benefits:

- **Control**: Helps manage gym capacity to avoid overcrowding.
- **Insights**: Offers data to improve gym operations and planning.

Appendix D: External API Documentation

The Digital Logging System (DLS) integrates several external APIs to enhance functionality, improve security, and deliver a better user experience. Below is an overview of the external APIs used in the system:

1. Continue with Google (Google OAuth API)

Role: Provides secure user authentication.
Functionality:

- Allows users to sign up or log in using their Google accounts.
- Simplifies the authentication process without needing manual credentials.

Benefits:

- **User Convenience**: Reduces the need for creating new accounts.
- **Security**: Ensures safe and verified login through Google's platform.

2. ReCaptcha API

Role: Prevents automated bots from accessing the system.
Functionality:

- Provides CAPTCHA challenges during registration or login.
- Verifies that the user is human before granting access.

Benefits:

- **Security**: Protects the system from spam and malicious automated requests.
- **Reliability**: Ensures legitimate user activity.

3. Google Sheets API

Role: Exports and stores attendance records.
Functionality:

- Automatically generates and saves attendance logs to Google Sheets.
- Allows admins to access and analyze attendance data conveniently.

Benefits:

- **Organization**: Provides a structured and accessible format for reports.
- **Integration**: Seamlessly connects system data with Google Sheets for reporting.

4. Google Drive API

Role: Stores and manages system-generated reports.
Functionality:

- Enables the storage of attendance and activity reports in Google Drive.
- Allows users and admins to access files securely.

Benefits:

- **Centralized Storage**: Keeps reports organized and easy to locate.
- **Accessibility**: Provides access to reports from any device with Google Drive.

5. Gmail API

Role: Sends email notifications to users and admins.

Functionality:

- Sends success emails after user registration and other key actions.
- Automates the delivery of attendance reports to admins.

Benefits:

- **Communication**: Keeps users informed about their account activities.
- **Automation**: Reduces manual email tasks for the admin.

6. Unsplash API

Role: Enhances the visual appeal of the system by providing dynamic backgrounds.

Functionality:

- Provides high-quality background images for the system's user interface.
- Ensures a visually engaging and professional design.

Benefits:

- **Aesthetics**: Improves the overall user experience with beautiful visuals.
- **Resourceful**: Offers a wide variety of free-to-use images.

Appendix E: Design Patterns Documentation

This section outlines the design patterns utilized in the **Digital Logging System (DLS)**. By leveraging these patterns, the system achieves better organization, maintainability, and scalability. Below are the key design patterns implemented in the project:

1. Model-View-Controller (MVC) Pattern

Description: The MVC pattern divides the application into three components—Model, View, and Controller—promoting separation of concerns and modular development.

Implementation:

- **Model**: Manages data and business logic, such as user records, attendance logs, and reports.
- **View**: Represents the user interface, including pages like the dashboard, login, and attendance history.
- **Controller**: Processes requests, connects the view and model, and defines backend routes (e.g., login, sign-up, and attendance tracking).

Benefits:

1. **Separation of Concerns**: Makes it easier to maintain and update individual components.
2. **Scalability**: New features can be added to the model, view, or controller without disrupting existing functionality.

2. Component-Based Architecture

Description: The system uses a component-based architecture for its front-end, ensuring reusability and simplified UI management.

Implementation:

- React components are used to build individual UI elements such as buttons, forms, tables, and dashboards.

- Components like LoginForm, UserDashboard, and VisitHistoryTable manage their own logic and state.
- Communication between components is achieved using **props** and **callbacks** for data flow.

Benefits:

1. **Reusability**: Components can be reused across different parts of the application.
2. **Maintainability**: Updates to one component do not affect others, simplifying debugging and improvements.
3. **Scalability**: New components can be created and integrated without significant changes to the application.

3. State Management

Description: State management is essential for handling dynamic data and user interactions in the application.

Implementation:

- React's useState is used to manage local state for components, such as form inputs, current attendance, and login status.
- useEffect is utilized to fetch data or trigger actions when components mount, such as retrieving the number of gym members currently logged in.

Benefits:

1. **Simplicity**: Built-in React hooks provide an efficient and easy-to-use approach to state management.
2. **Performance**: Optimizes rendering by managing state locally within components, improving application responsiveness.

4. Singleton Pattern

Description: Ensures that only one instance of a class or resource is created and reused across the application.

Implementation:

- Applied to the database connection to ensure a single connection instance is used throughout the backend, reducing overhead and improving efficiency.

Benefits:

1. **Resource Efficiency**: Reduces redundant connections and optimizes system resources.
2. **Consistency**: Centralized management of database interactions ensures data integrity.

5. Observer Pattern

Description: The Observer pattern is used to allow components or services to react to changes in the application state or events.

Implementation:

- Applied in email notifications (e.g., successful registration and report generation) where an event triggers the system to send an email using the Gmail API.
- Notifications update users without requiring direct component connections.

Benefits:

1. **Decoupling**: Components can respond to events without being tightly integrated.
2. **Dynamic Updates**: Users are notified in real-time of system changes or updates, improving user experience.

6. Factory Pattern

Description: The Factory pattern is used to create objects without specifying their exact class, enabling flexibility in object creation.

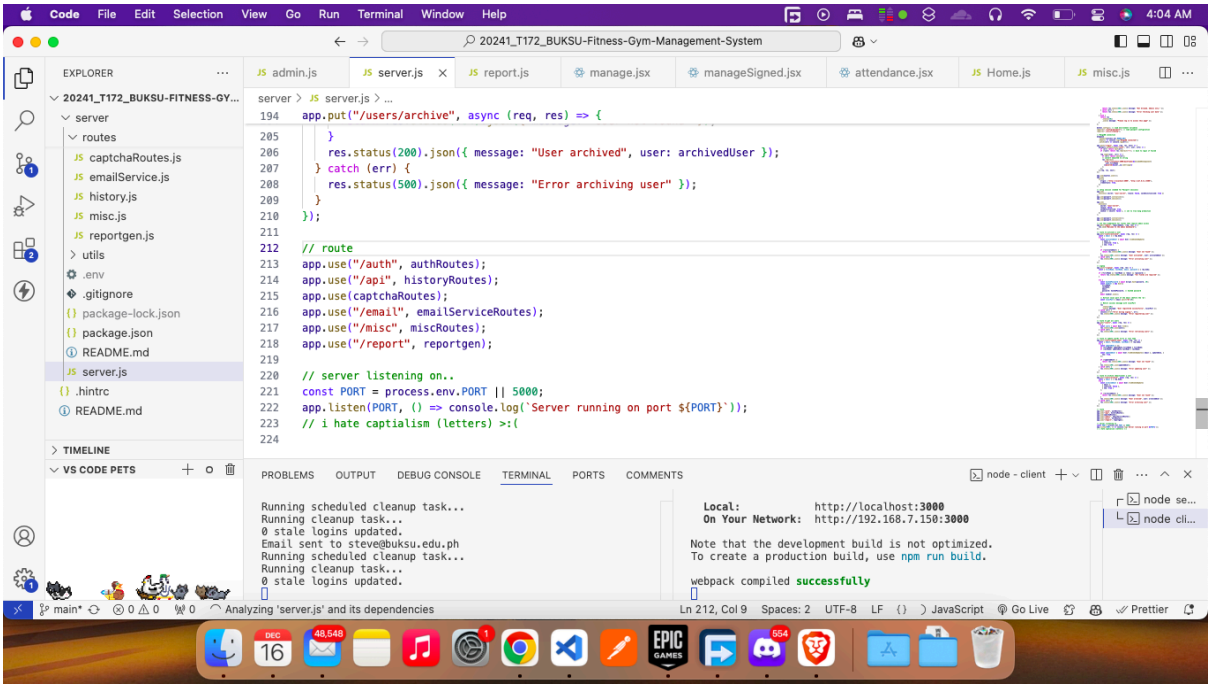
Implementation:

- Used for generating different user roles (e.g., admin, member) and handling role-specific logic such as permissions and access controls.

Benefits:

1. **Flexibility**: Simplifies the creation of objects based on user roles.
2. **Maintainability**: Adding new roles or features requires minimal changes to the codebase.

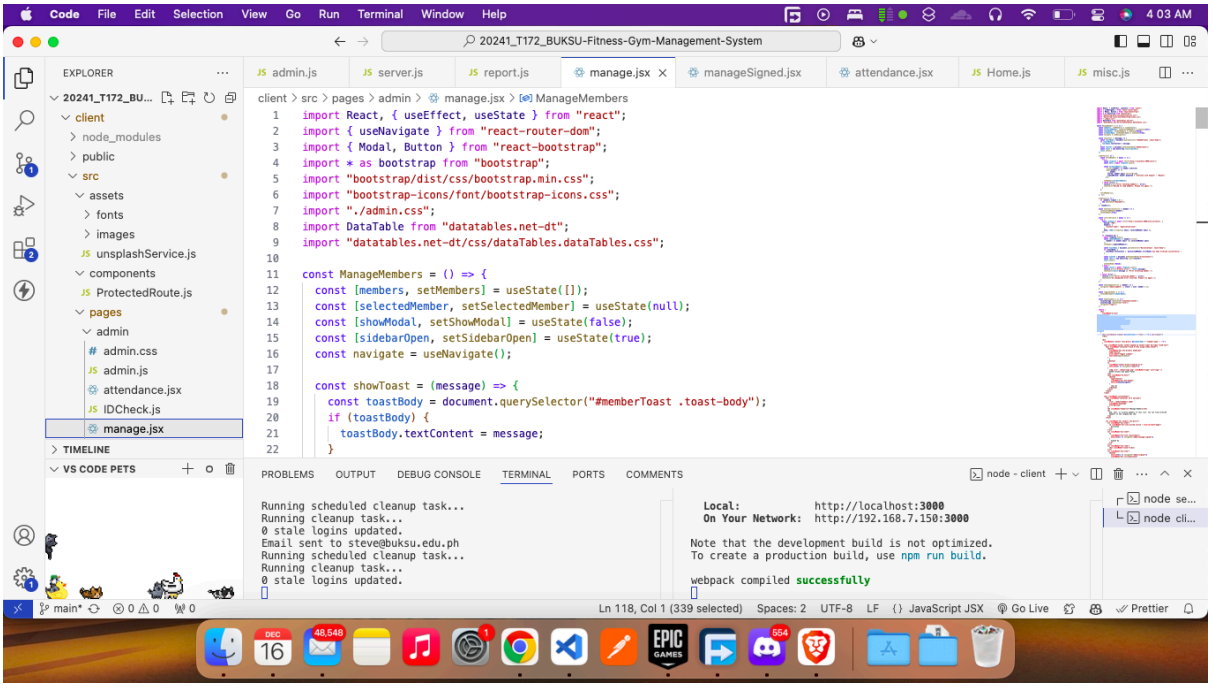
Appendix F: Sample Code Snippets



```
app.put("/users/archive", async (req, res) => {
    }
    res.status(200).json({ message: "User archived", user: archivedUser });
  } catch (err) {
    res.status(500).json({ message: "Error archiving user" });
  }
});

// route
app.use("/auth", authRoutes);
app.use("/api", historyRoutes);
app.use(captchaRoutes);
app.use("/email", emailServiceRoutes);
app.use("/misc", miscRoutes);
app.use("/report", reportgen);

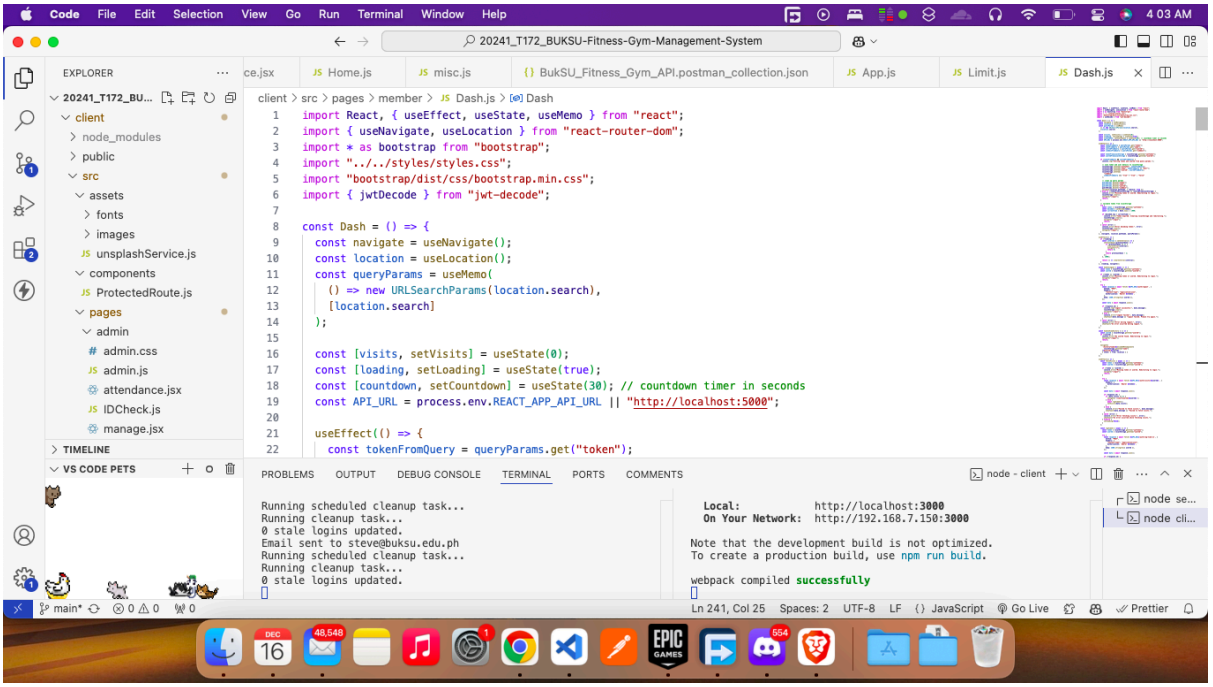
// server listening on..
const PORT = process.env.PORT || 5000;
app.listen(PORT, () => console.log(`Server running on port ${PORT}`));
// i hate captialism (letters) >:(
```

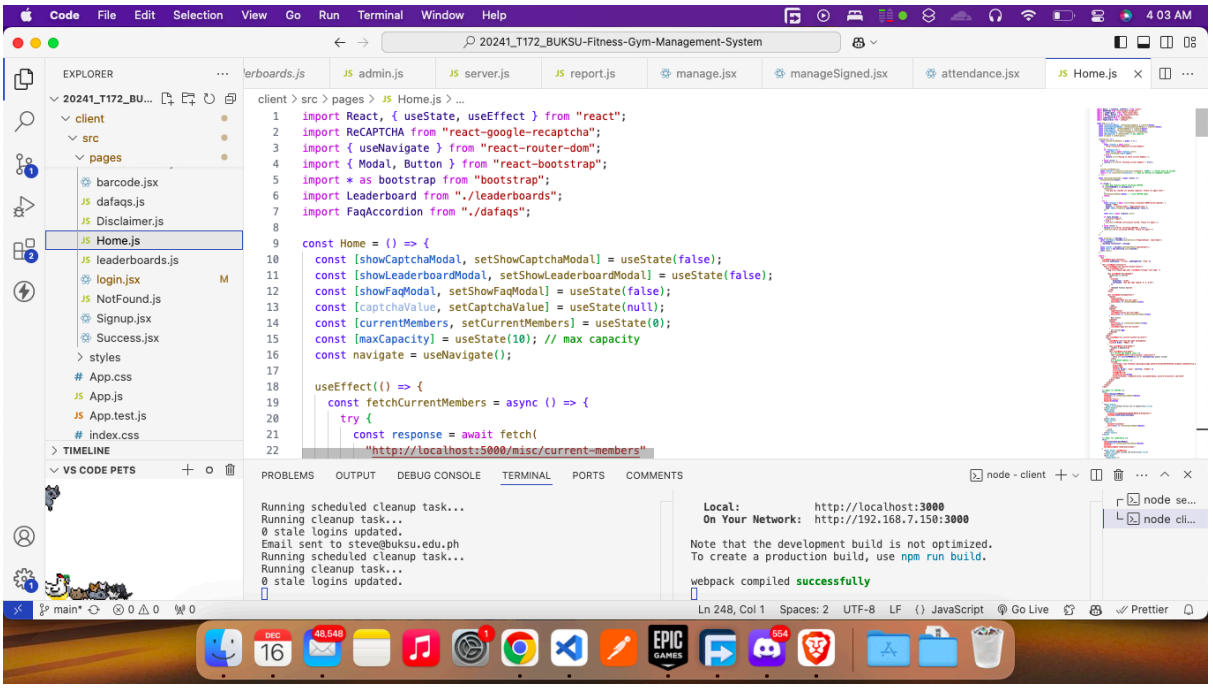


```
import React, { useEffect, useState } from "react";
import { useNavigate } from "react-router-dom";
import { Modal, Button } from "react-bootstrap";
import * as bootstrap from "bootstrap";
import "bootstrap/dist/css/bootstrap.min.css";
import "bootstrap-icons/font/bootstrap-icons.css";
import "./admin.css";
import DataTable from "datatables.net-dt";
import "datatables.net-dt/css/dataTables.dataTables.css";

const ManageMembers = () => {
  const [members, setMembers] = useState([]);
  const [selectedMember, setSelectedMember] = useState(null);
  const [showModal, setShowModal] = useState(false);
  const [sidebarOpen, setSidebarOpen] = useState(true);
  const navigate = useNavigate();

  const showToast = (message) => {
    const toastBody = document.querySelector("#memberToast .toast-body");
    if (toastBody) {
      toastBody.textContent = message;
    }
```


Code File Edit Selection View Go Run Terminal Window Help
4 03 AM
20241_T172_BUKSU-Fitness-Gym-Management-System
EXPLORER
20241_T172_BU...
client
node_modules
public
src
assets
fonts
images
unsplashService.js
components
ProtectedRoute.js
pages
admin
admin.css
admin.js
attendance.jsx
IDCheck.js
manage.jsx
TIMELINE
VS CODE PETS
ce.jsx
Home.js
misc.js
BukSU_Fitness_Gym_API.postman_collection.json
App.js
Limit.js
Dash.js
client > src > pages > member > Dash.js > Dash
import React, { useEffect, useState, useMemo } from "react";
import { useNavigate, useLocation } from "react-router-dom";
import * as bootstrap from "bootstrap";
import "../../styles/styles.css";
import "bootstrap/dist/css/bootstrap.min.css";
import { jwtDecode } from "jwt-decode";
const Dash = () => {
const navigate = useNavigate();
const location = useLocation();
const queryParams = useMemo(
() => new URLSearchParams(location.search),
[location.search]
);
const [visits, setVisits] = useState(0);
const [loading, setLoading] = useState(true);
const [countdown, setCountdown] = useState(30); // countdown timer in seconds
const API_URL = process.env.REACT_APP_API_URL || "http://localhost:5000";
useEffect(() => {
const tokenFromQuery = queryParams.get("token");
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS COMMENTS
node - client
Running scheduled cleanup task...
Running cleanup task...
0 stale logins updated.
Email sent to steve@buksu.edu.ph
Running scheduled cleanup task...
Running cleanup task...
0 stale logins updated.
Local: http://localhost:3000
On Your Network: http://192.168.7.150:3000
Note that the development build is not optimized.
To create a production build, use npm run build.
webpack compiled successfully
node se...
node cli...
main*
Ln 241, Col 25 Spaces: 2 UTF-8 LF JavaScript Go Live Prettier


Code File Edit Selection View Go Run Terminal Window Help
4 03 AM
20241_T172_BUKSU-Fitness-Gym-Management-System
EXPLORER
20241_T172_BU...
client
src
pages
barcode.jsx
dafaqs.js
Disclaimer.js
Home.js
leaderboards.js
login.jsx
NotFound.js
Signup.jsx
Success.jsx
styles
App.css
App.js
App.test.js
index.css
TIMELINE
VS CODE PETS
erboards.js
admin.js
server.js
report.js
manage.jsx
manageSigned.jsx
attendance.jsx
Home.js
client > src > pages > Home.js > ...
import React, { useState, useEffect } from "react";
import ReCAPTCHA from "react-google-recaptcha";
import { useNavigate } from "react-router-dom";
import { Modal, Button } from "react-bootstrap";
import * as bootstrap from "bootstrap";
import Leaderboard from "./leaderboards";
import FaqAccordion from "./dafaqs";
const Home = () => {
const [showCaptchaModal, setShowCaptchaModal] = useState(false);
const [showLeaderboardModal, setShowLeaderboardModal] = useState(false);
const [showFaqModal, setShowFaqModal] = useState(false);
const [captchaValue, setCaptchaValue] = useState(null);
const [currentMembers, setCurrentMembers] = useState(0);
const [maxCapacity] = useState(10); // max capacity
const navigate = useNavigate();
useEffect(() => {
const fetchCurrentMembers = async () => {
try {
const response = await fetch(
"http://localhost:5000/misc/current-members"
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS COMMENTS
node - client
Running scheduled cleanup task...
Running cleanup task...
0 stale logins updated.
Email sent to steve@buksu.edu.ph
Running scheduled cleanup task...
Running cleanup task...
0 stale logins updated.
Local: http://localhost:3000
On Your Network: http://192.168.7.150:3000
Note that the development build is not optimized.
To create a production build, use npm run build.
webpack compiled successfully
node se...
node cli...
main*
Ln 248, Col 1 Spaces: 2 UTF-8 LF JavaScript Go Live Prettier

Appendix F: Excel Computation of the Worksheets

Task	Start Date	End Date	Duration (Days)	Assigned Resources
Requirements Gathering	05/10/2024	10/10/2024	5	3 Members
System Design	15/10/2024	23/10/2024	8	2 Members
Frontend Development	28/10/2024	05/11/2024	7	2 Members
Backend Development	10/11/2024	02/12/2024	24	4 Members
API Integration	17/11/2024	25/11/2024	9	2 Members
Testing & Debugging	25/11/2024	09/12/2024	14	3 Members
Documentation	29/11/2024	13/12/2024	14	3 Members
		Total Duration (Days)	**81**	

This table represents the project timeline and resource allocation for various tasks required to complete the project.

Appendix G: Coding Standards and Guidelines

This section outlines the coding standards and guidelines followed during the development of the **BukSU Fitness Gym - Digital Logging System**. Adhering to these standards ensures that the codebase remains clean, consistent, maintainable, and scalable.

1. General Coding Guidelines

- **Consistency**: Use consistent naming conventions, indentation, and formatting throughout the project.
- **Code Readability**: Write code that is easy to read and understand by others. Add meaningful comments to explain complex logic.

- **Reusability**: Break code into reusable functions, components, and modules to reduce duplication.
- **Error Handling**: Implement proper error handling to prevent application crashes and ensure graceful degradation.
- **Version Control**: Use GitHub for tracking code changes, committing frequently with clear and descriptive commit messages.

2. Naming Conventions

- **Variables and Functions**: Use camelCase for naming variables, functions, and constants.
    - Example: currentUser, getVisitHistory().
- **Classes**: Use PascalCase for class and component names.
    - Example: UserDashboard, AttendanceLogger.
- **Files and Folders**: Use kebab-case for filenames and directories.
    - Example: login-form.jsx, user-dashboard.css.
- **Database Collections**: Use lowercase plural names for database collections.
    - Example: users, visits, reports.

3. File and Directory Structure

Organize files into a logical structure to promote modularity and clarity.

4. Code Formatting

- Use **2 spaces** for indentation (consistent across all files).
- Use **Prettier** or ESLint for automatic formatting to enforce consistent styling.
- Place **braces** {} on the same line for blocks and functions.
- Add a **newline** at the end of every file.

5. Code Documentation

- Use comments to document functions, classes, and modules.
- Clearly describe the parameters, return values, and purpose of each function.

6. Security Guidelines

- **Environment Variables**: Use .env files to store sensitive information like database URIs, JWT secrets, and API keys.
- **Input Validation**: Validate all user inputs to prevent SQL injection, XSS, or invalid data.
- **JWT**: Use **JSON Web Tokens** for secure authentication and session management.

7. Error Handling and Logging

- Use try-catch blocks to handle errors gracefully.
- Log errors using a centralized logging mechanism.
- Return user-friendly error messages without exposing sensitive details.

8. API Standards

- Follow **RESTful API principles** for backend endpoints.
- Use standard HTTP methods:
    - **GET**: Retrieve data.
    - **POST**: Add new data.
    - **PUT**: Update existing data.
    - **DELETE**: Remove data.
- Use appropriate status codes for API responses:
    - 200: OK
    - 201: Created
    - 400: Bad Request
    - 401: Unauthorized
    - 404: Not Found
    - 500: Internal Server Error

9. Testing Standards

- Use **Postman** for testing API endpoints.
- Perform testing for critical backend functions.

The **BukSU Fitness Gym - Digital Logging System (DLS)** adheres to a set of coding standards and guidelines to ensure consistency, security, and maintainability. These guidelines promote clean, efficient, and scalable code, enabling future development and collaboration.

Appendix H: Project Management Artifacts

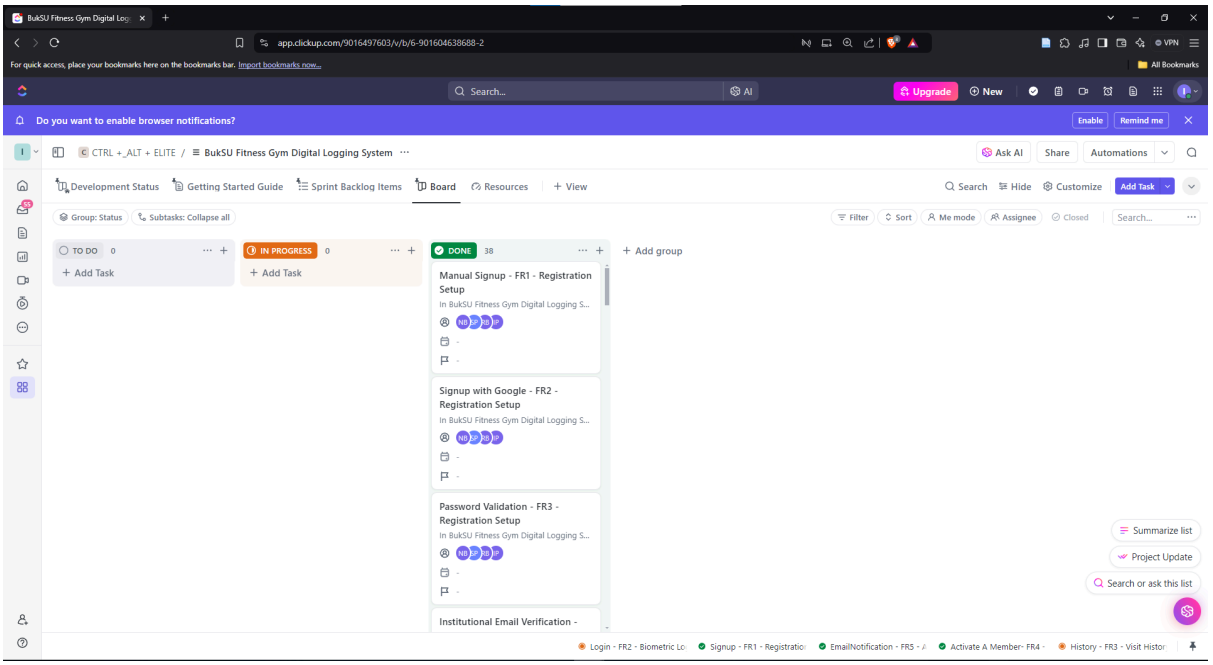
1. **Project Overview**
    - **Objective:** The primary goal of this project was to develop a digital logging system for the BukSU Fitness Gym that streamlines attendance tracking and improves administrative efficiency.
    - **Scope**: The system aimed to replace the manual logging process with a web-based platform featuring user authentication, attendance tracking, and report generation capabilities.
2. **Gantt Chart**

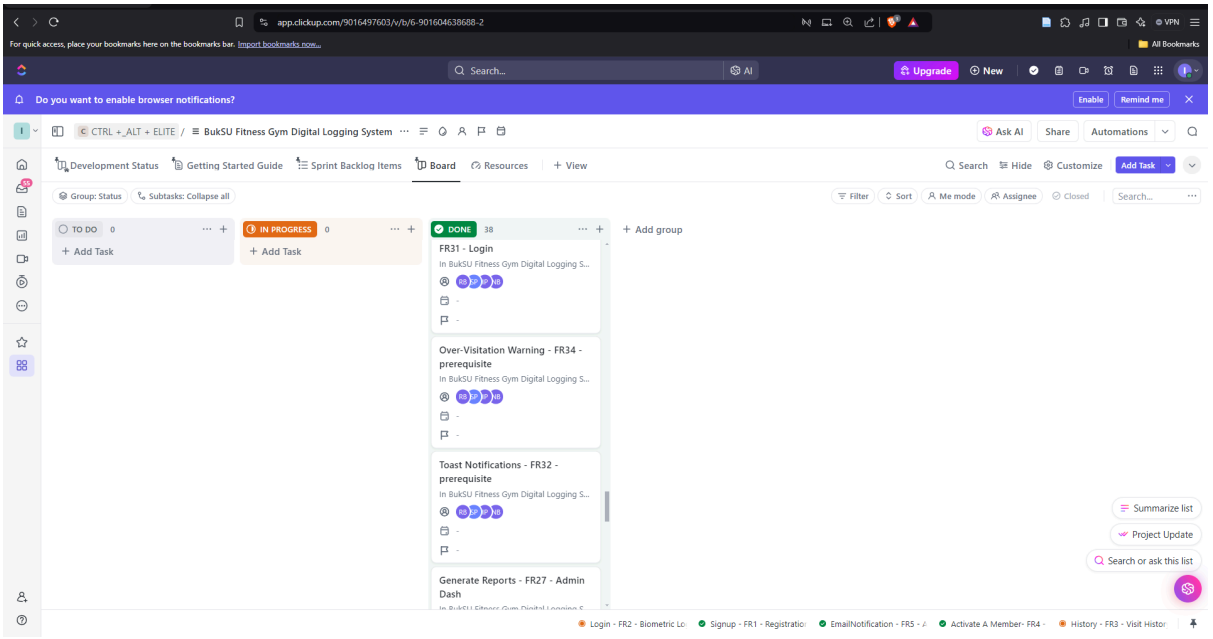
TASK	START DATE	END DATE	DURATION	DEPENDENCY
Requirements Gathering	05/10/2024	10/10/2024	5	**None**
System Design	15/10/2024	23/10/2024	8	**Requirements Gathering**
Frontend Development	28/10/2024	05/11/2024	7	**System Design**
Backend Development	10/11/2024	02/12/2024	24	**System Design**
API Integration	17/11/2024	25/11/2024	9	**Backend Development**

Testing & Debugging	25/11/2024	09/12/2024	14	**Frontend Development & Backend Development**
Documentation	29/11/2024	13/12/2024	14	**Testing & Debugging**

The Gantt Chart above illustrates the timeline and dependencies of tasks involved in the project.

3. Task Tracker

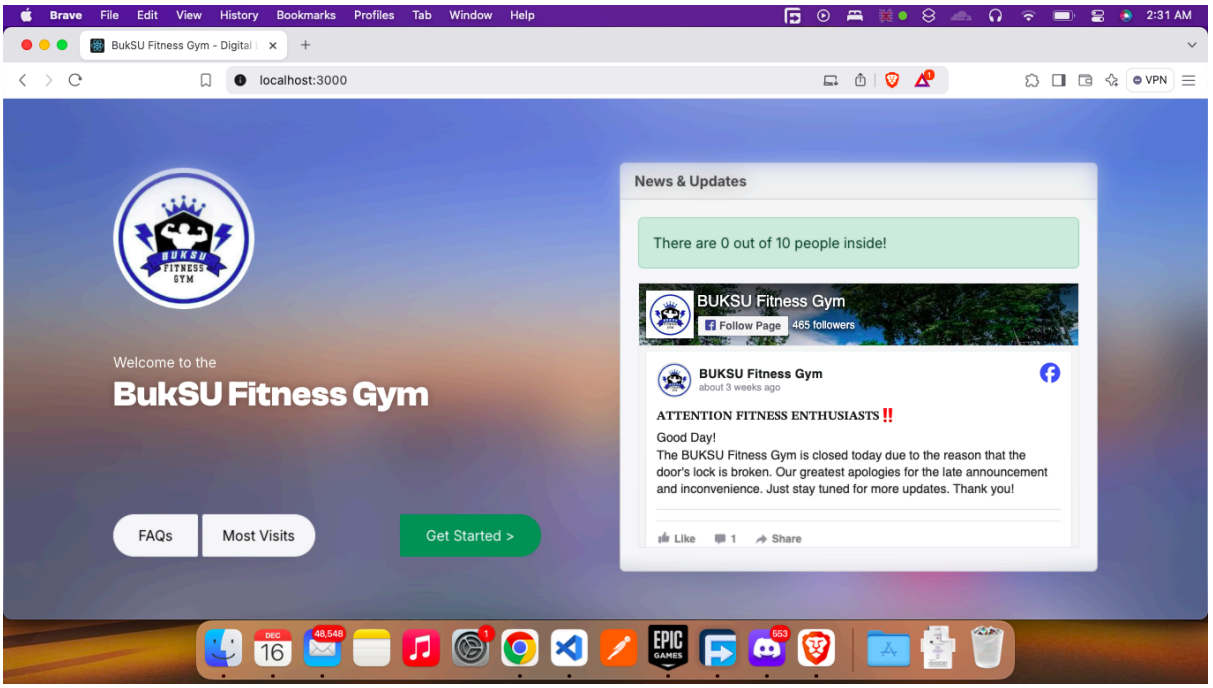




The screenshots above highlight our use of a Kanban board, a practical and visual tool for managing the development process throughout the project lifecycle. The board is organized into clear columns such as "To Do," "In Progress," and "Done," representing different stages of task completion. This setup offers a straightforward and systematic way to track progress, making it easy for everyone on the team to stay aligned. It promotes an efficient workflow, and helps us stay on top of deadlines while ensuring every task gets the attention it deserves.

In this project, a total of 38 tasks were carefully planned and successfully completed. The Kanban board played a vital role in helping the team stay organized and focused. It provided a clear view of each task's status, allowed us to prioritize work efficiently, and made it easier to identify and address potential bottlenecks early on. This collaborative approach ensured that every team member knew what needed attention and when, fostering a smooth and productive workflow.

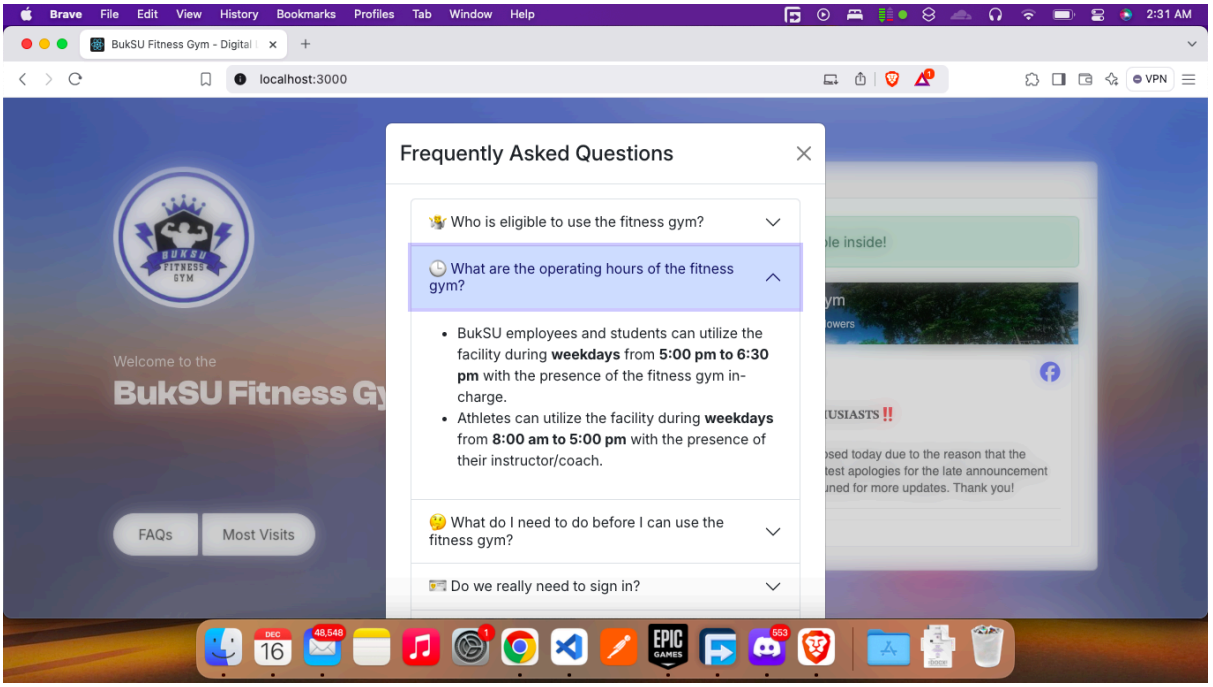
Appendix I: Screenshots of System

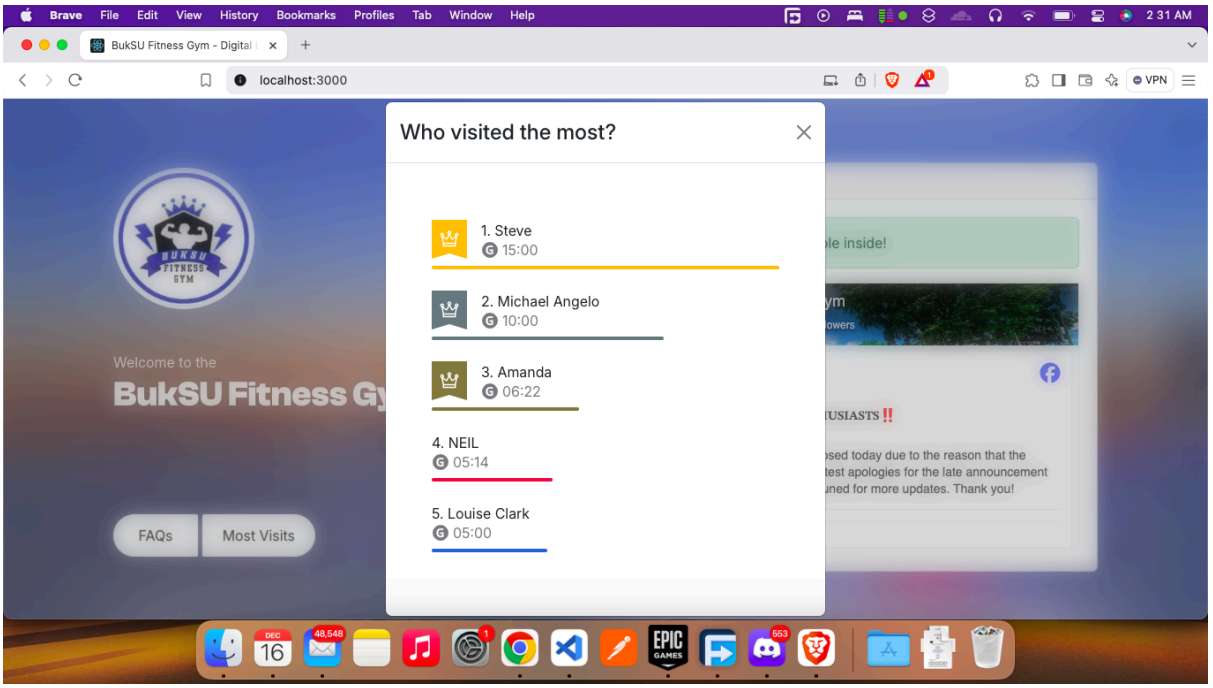


Screenshot 1: The Landing Page.

This screenshot showcases the Digital Logging System (DLS) landing page, which serves as a central hub for gym members to access important information and features. On the right side, there is a News & Updates section, which provides real-time notifications about the gym's status and announcements. A green banner at the top of this section displays, “There are 0 out of 10 people inside!”, indicating the gym’s current occupancy and helping members determine if it’s an ideal time to visit. Below this, an embedded Facebook feed from the BUKSU Fitness Gym page shares updates and announcements.

At the bottom of the screen, two buttons labeled “FAQs” and “Most Visits” are visible. The FAQs button leads users to commonly asked questions for guidance, while the Most Visits button highlights frequent gym users. The overall layout is clean and user-friendly, featuring a gradient background that creates a professional yet calming visual appeal. The use of green buttons and banners adds contrast, drawing attention to key actions and notifications.





Screenshot 2 & 3: The FAQs and Most Visits Button.

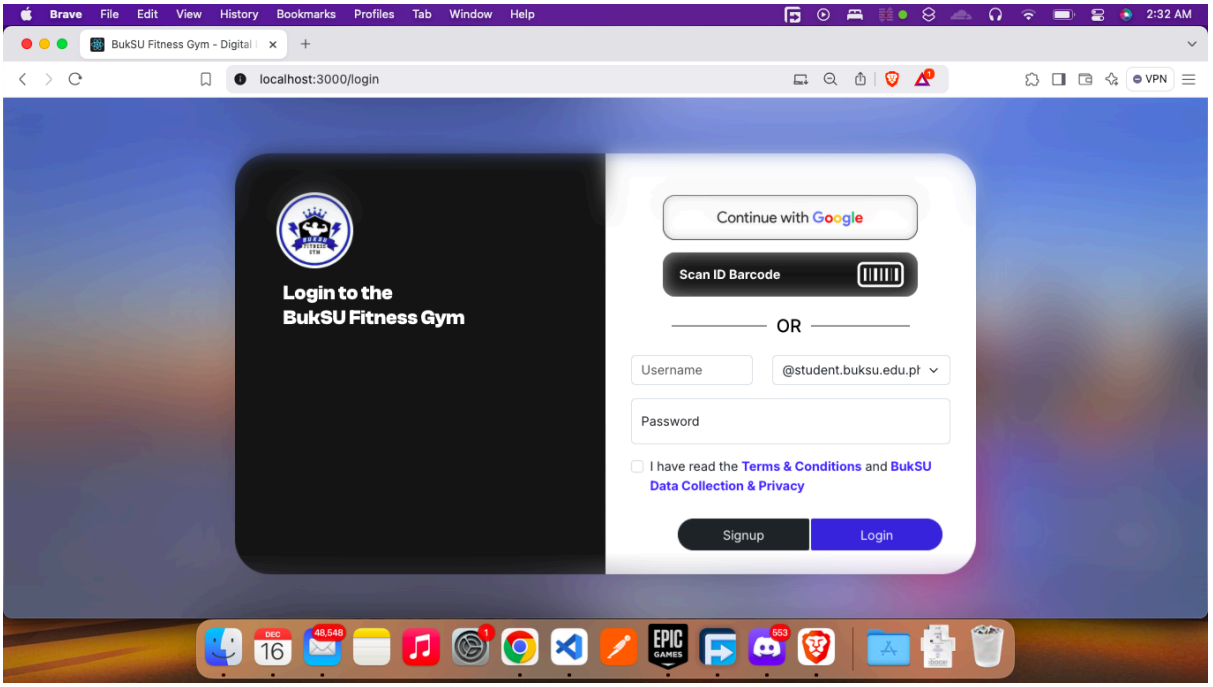
These screenshots showcase additional features of the Digital Logging System (DLS), focusing on visit statistics and Frequently Asked Questions (FAQs) for gym users.

In the first screenshot, a modal titled “Who visited the most?” displays a leaderboard of gym visitors, ranked by their total recorded visit durations. This feature visually represents user activity using

progress bars of varying colors and lengths, making it easy to identify the most active gym members. It adds a gamified and informative aspect to the system by recognizing frequent visitors.

The second screenshot focuses on the FAQs section, where common questions are addressed for user convenience. One expanded question, “What are the operating hours of the fitness gym?”.

This FAQ section is designed to clarify user queries regarding gym usage, rules, and operational details. Together, these features enhance the system’s functionality: the leaderboard motivates users through visit tracking, while the FAQ section improves accessibility by providing clear and structured answers to commonly asked questions. Both contribute to a user-friendly and informative experience for gym members.



Screenshot 4: Login Page.

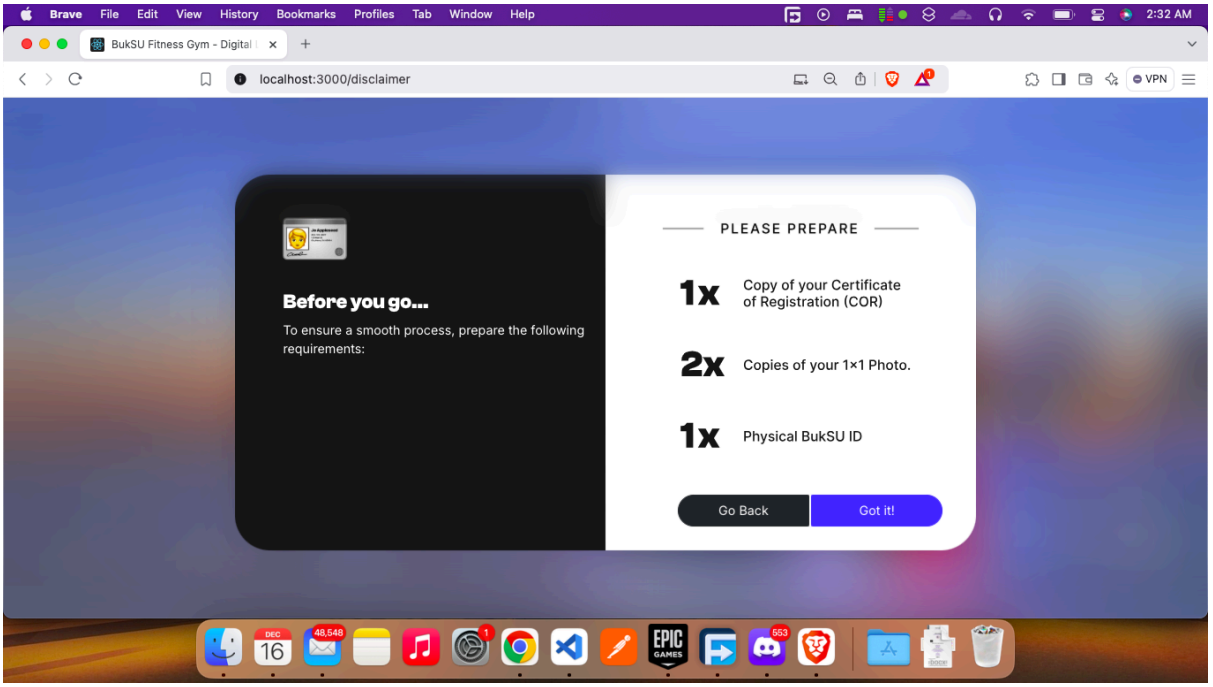
This screenshot displays the Login Page for the Digital Logging System. The page is divided into two sections: the left side features a black background with the BukSU Fitness Gym logo and a heading that reads, “Login to the BukSU Fitness Gym”. This section provides a clear and branded welcome to the user.

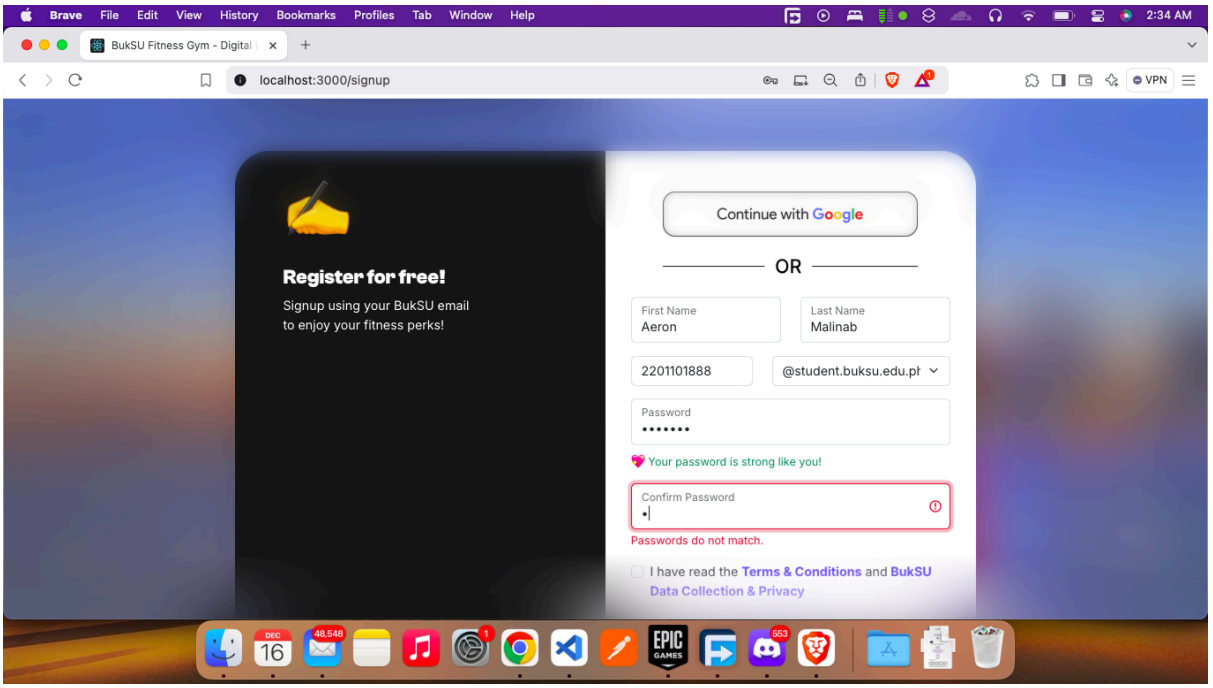
On the right side, the login form offers three options for users to log into the system:

1. **Continue with Google:** A button at the top allows users to log in via their BukSU-issued Google accounts, simplifying the process for those with linked credentials.
2. **Scan ID Barcode: T**here is an option for users to scan their ID barcodes using a scanner, which is tied and based on the username (before the @ symbol in the email).
3. **Manual Login:** Users can manually log in by entering their Username and Password. A dropdown menu next to the username field indicates the system's email domain, specifically @student.buksu.edu.ph, suggesting that this system primarily serves students of Bukidnon State University.

A checkbox appears below the password field, requiring users to confirm that they have read and agreed to the Terms & Conditions and the BukSU Data Collection & Privacy policy. At the bottom of the form, there are two buttons: "Signup", which allows new users to create an account, and "Login", which authenticates existing users.

The design of the page is clean and modern, with a balanced color scheme that combines black, white, and purple. The contrasting layout ensures that users can quickly identify their login options, and the clear call-to-action buttons streamline the process.

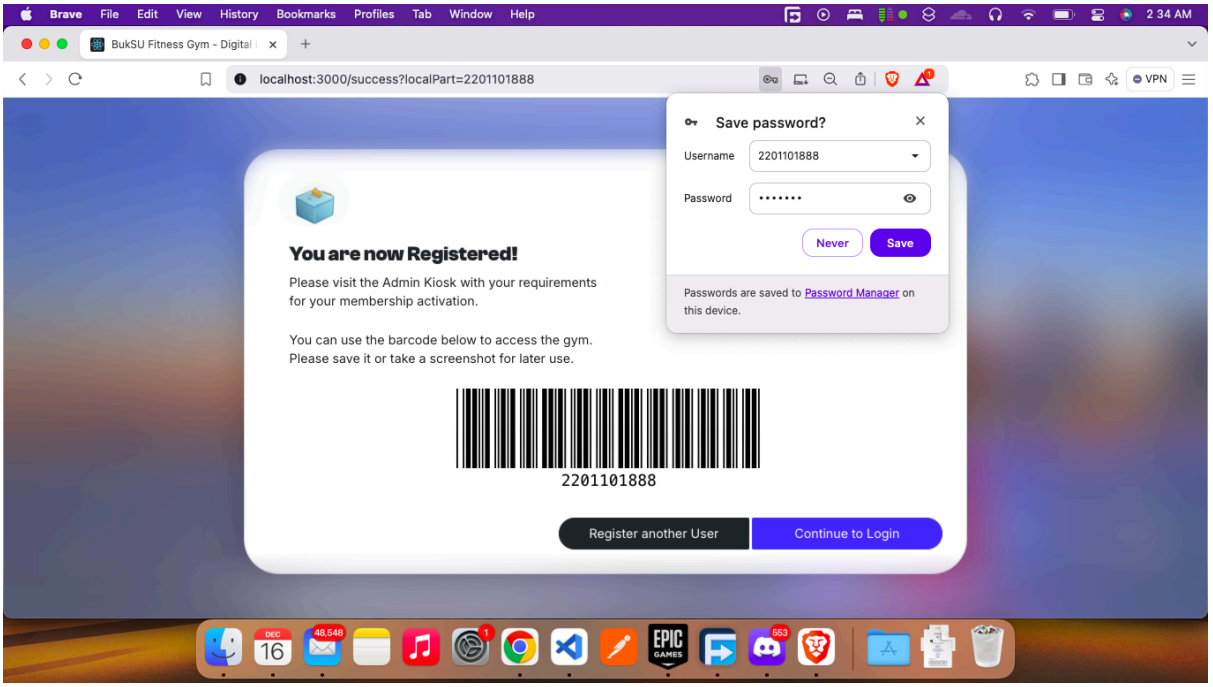




Screenshot 5 & 6: The Disclaimer and Signup Pages.

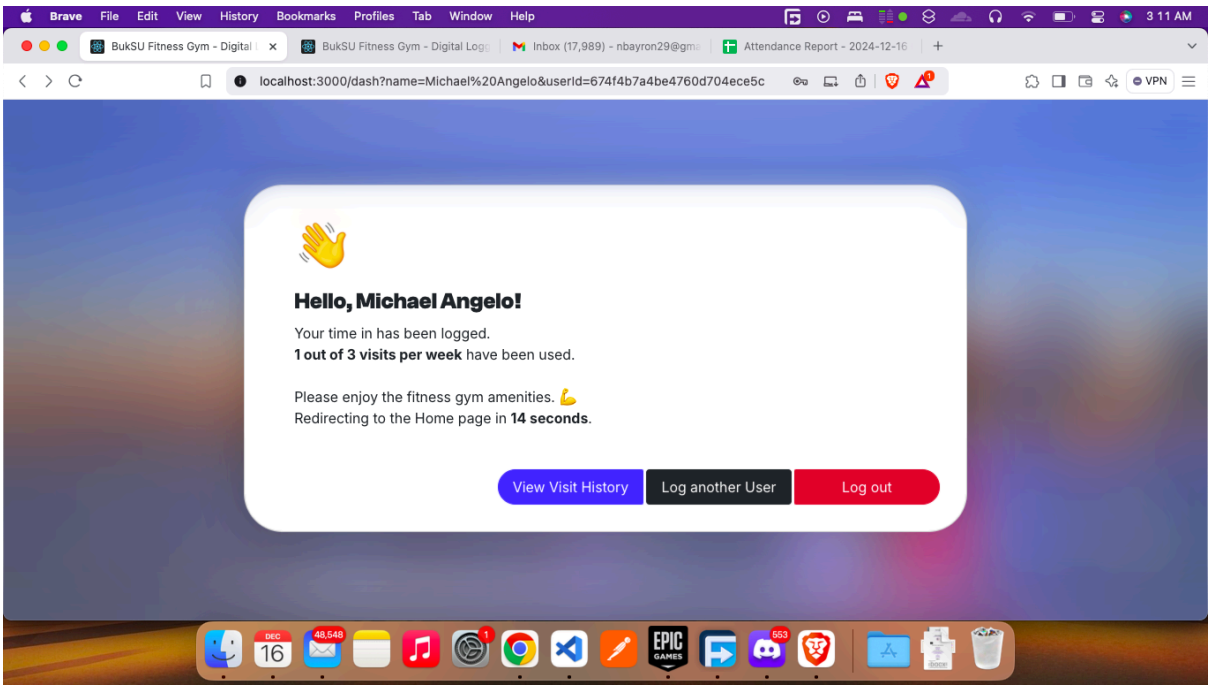
The two screenshots display the Disclaimer Page and the Signup Page of the Digital Logging System (DLS). The Disclaimer Page informs users about the required documents needed for registration. It also provides options to proceed or go back.

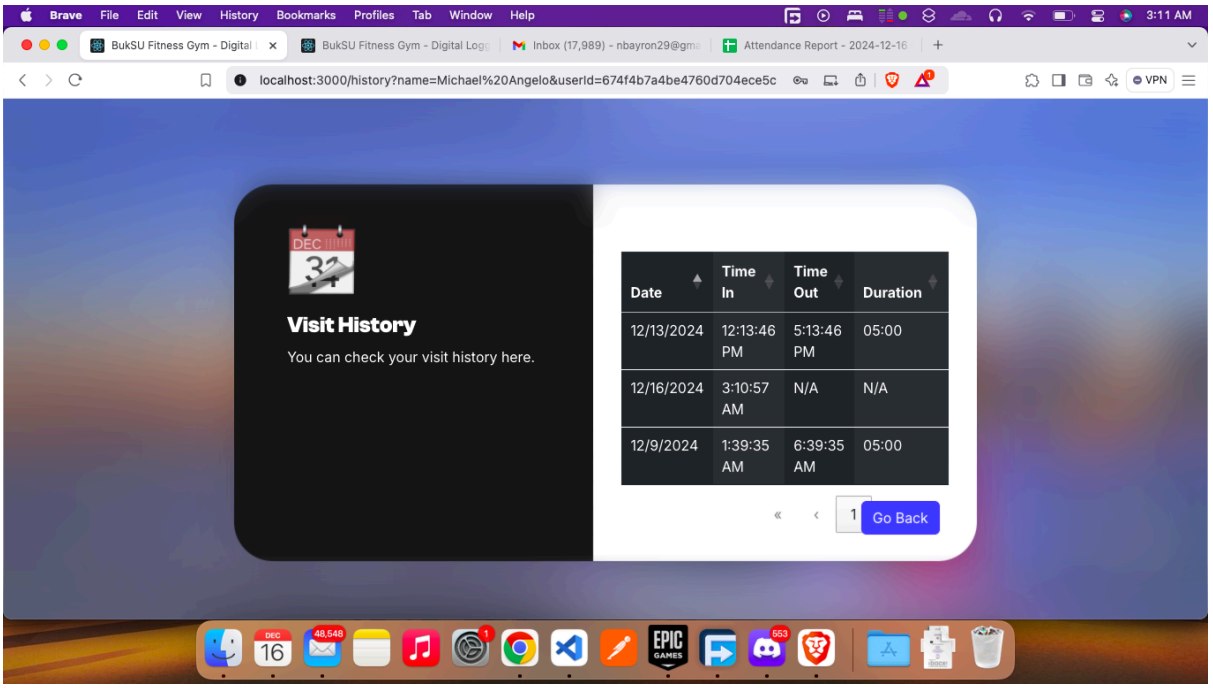
The Signup Page allows users to register for the system. Users can either sign up using their Google account or manually fill out their details, such as their name, username, and password. The page includes a checkbox to agree to the system's terms and conditions and buttons to navigate back or complete the registration. Together, these pages ensure a smooth and straightforward onboarding process for users.



Screenshot 7: Registration Successful Page.

This screenshot shows the Registration Success Page of the Digital Logging System (DLS). It confirms successful registration and instructs the user to visit the Admin Kiosk for membership activation. A unique barcode with the user ID (2201101888) is displayed for gym access, with a prompt to save or screenshot it. Users can choose to “Register another User” or “Continue to Login” to proceed.

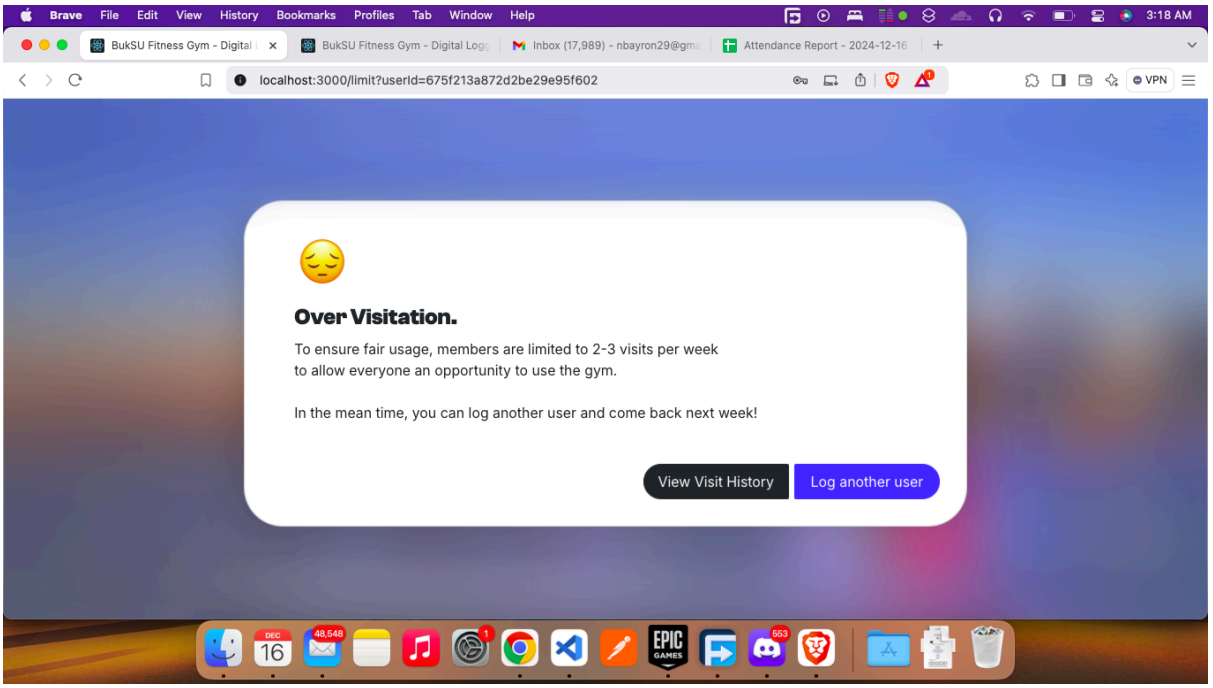




Screenshot 8 & 9: The User Dashboard and Visit History

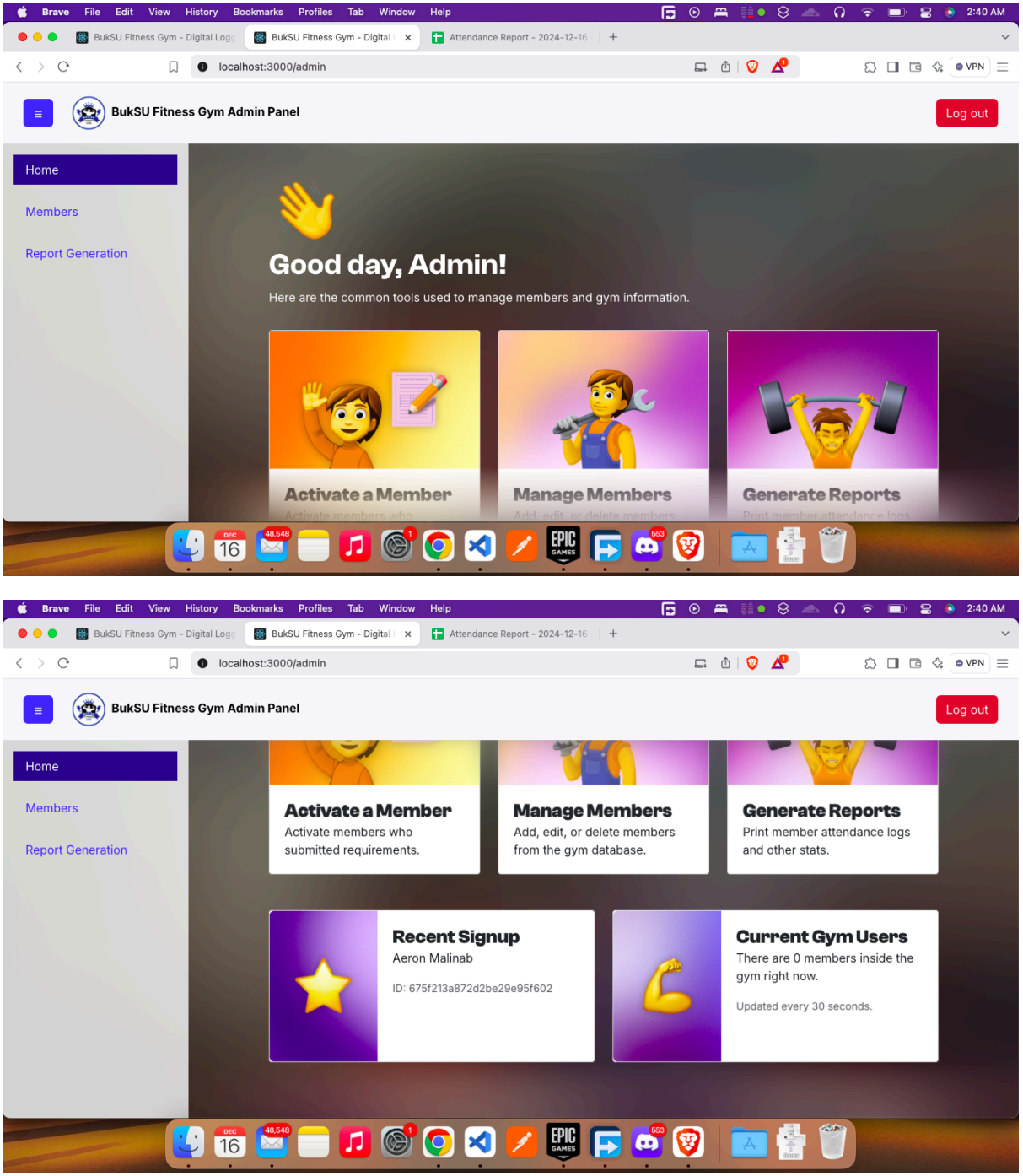
These screenshots show the User Dashboard and Visit History of the Digital Logging System (DLS). In the first screenshot, a welcome message confirms that the user's time has been logged, showing 1 out of 3 visits per week. Options are provided to View Visit History, Log another User, or Log out, with an automatic redirect to the homepage after a countdown.

The second screenshot displays the Visit History page, where the user can view their past gym visits in a table format. It includes the Date, Time In, Time Out, and Duration of each visit. A Go Back button allows users to return to the previous page. Together, these pages provide users with confirmation of their gym visits and a detailed history for tracking their attendance.



Screenshot 10: Over Visitation Page.

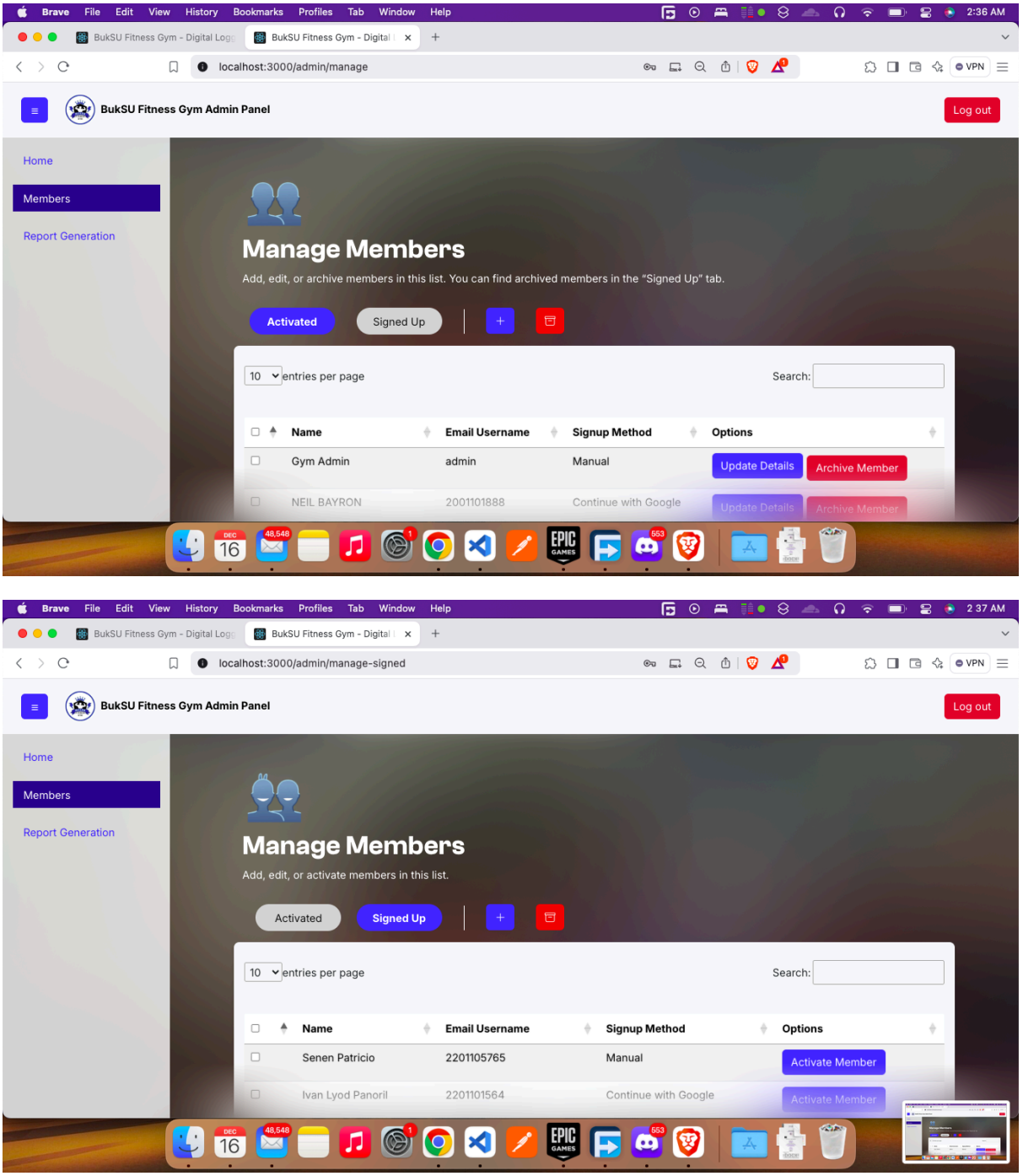
This screenshot informs users that they have exceeded the gym's limit of 2-3 visits per week to ensure fair usage for all members. The message encourages users to return the following week and provides options to either View Visit History or Log another user. This page ensures that gym usage remains balanced while offering users the opportunity to manage their visits effectively.



Screenshot 11 & 12: The Admin Panel Home Page.

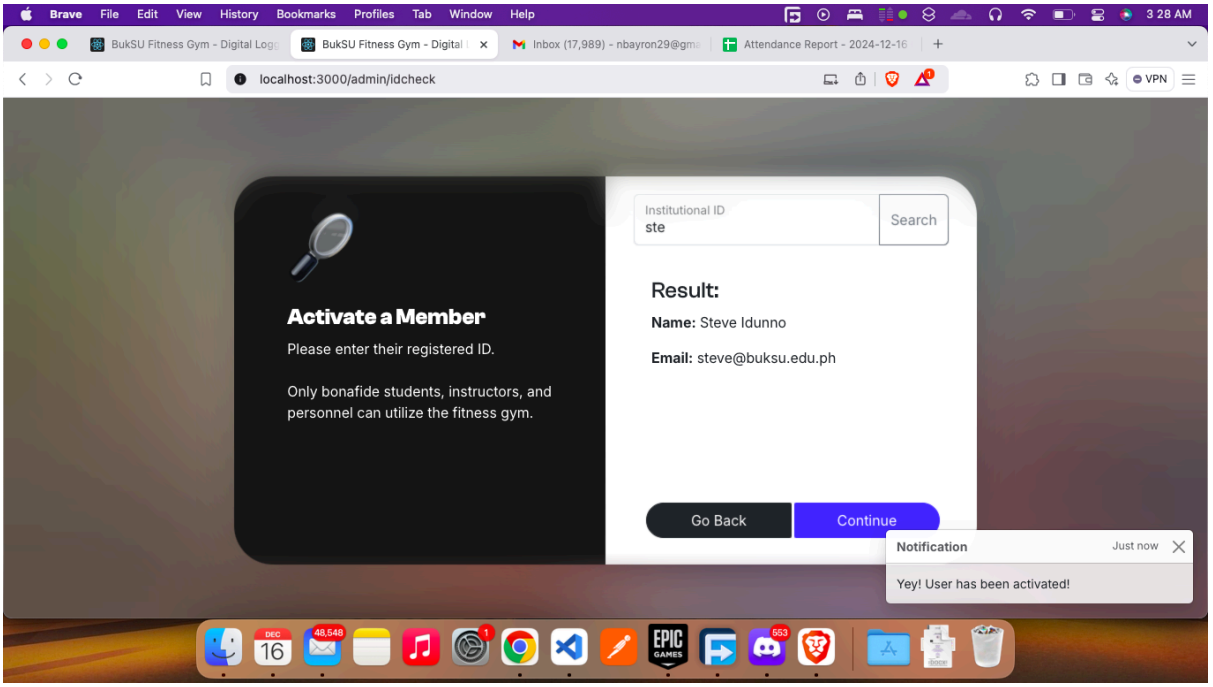
These screenshots show the Admin Panel of the Digital Logging System (DLS). The panel welcomes the admin with tools to Activate a Member, Manage Members (add, edit, or archive users), and Generate Reports for attendance logs. Additional sections display a Recent Signup with user details and the Current Gym Users, showing real-time gym occupancy, updated every 30 seconds. The

navigation menu on the left allows quick access to Home, Members, and Report Generation. Overall, the panel provides admins with essential tools to manage gym operations efficiently.



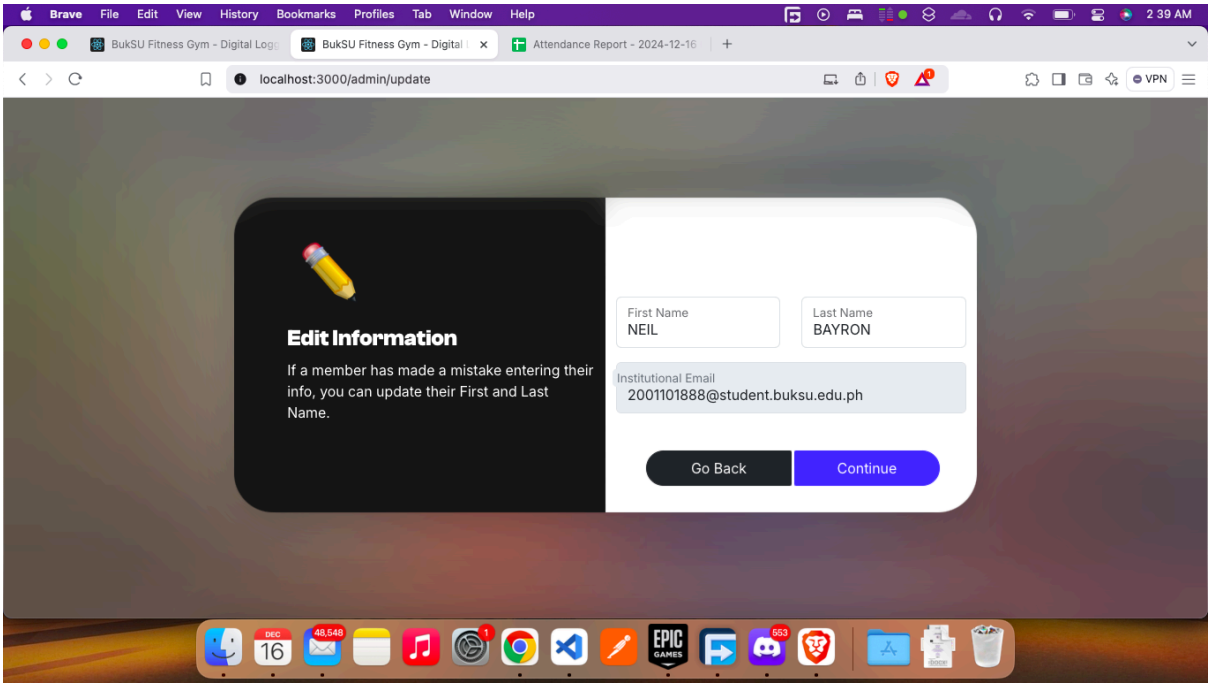
Screenshot 13 & 14: The Manage Members page in the Admin Panel.

These screenshots display the Manage Members page of the BukSU Fitness Gym Admin Panel. Admins can activate, update, or archive members in two tabs: Signed Up and Activated. The table lists user details, including Name, Email Username, and Signup Method, with action buttons like Activate Member, Update Details, or Archive Member. A search bar and pagination options help navigate the list efficiently, ensuring smooth member management.



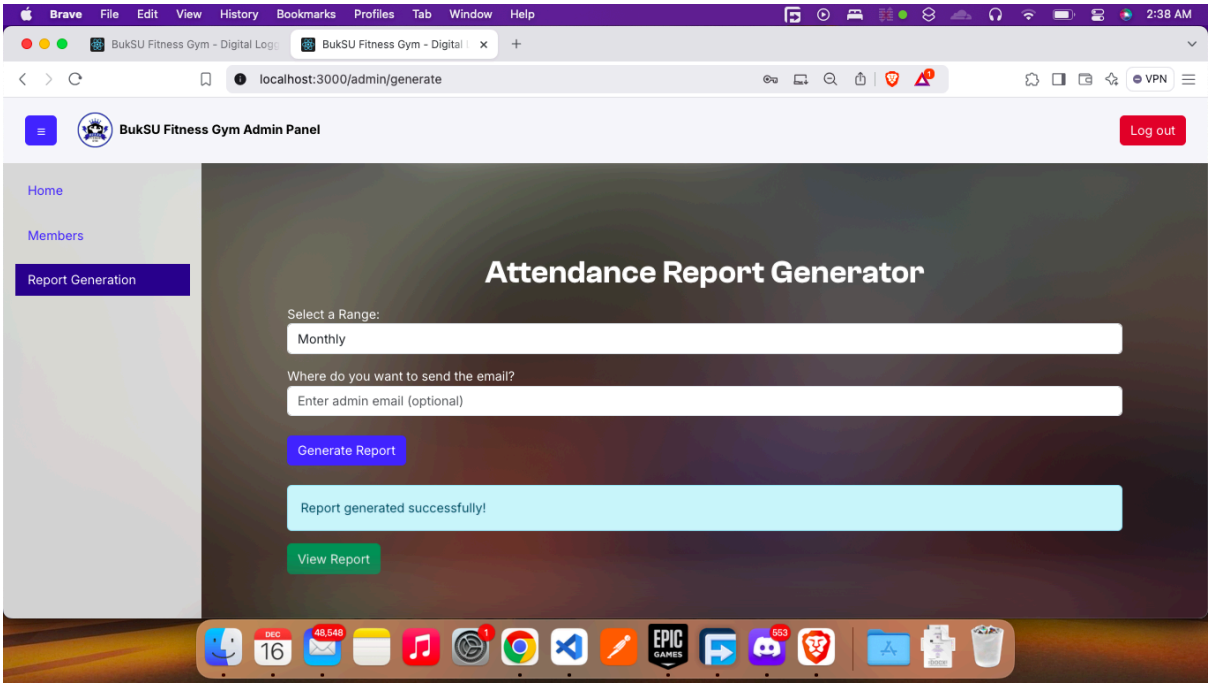
Screenshot 15: Activate a Member Page in the Admin Panel.

This screenshot shows the Activate a Member page in the BukSU Fitness Gym Admin Panel. Admins can search for a user's registered ID to activate their membership. Once a valid ID is entered, the user's details, including their Name and Email, are displayed. A notification confirms successful activation with the message: "Yey! User has been activated!". Admins can either go back or continue to the next step, ensuring a quick and efficient activation process.



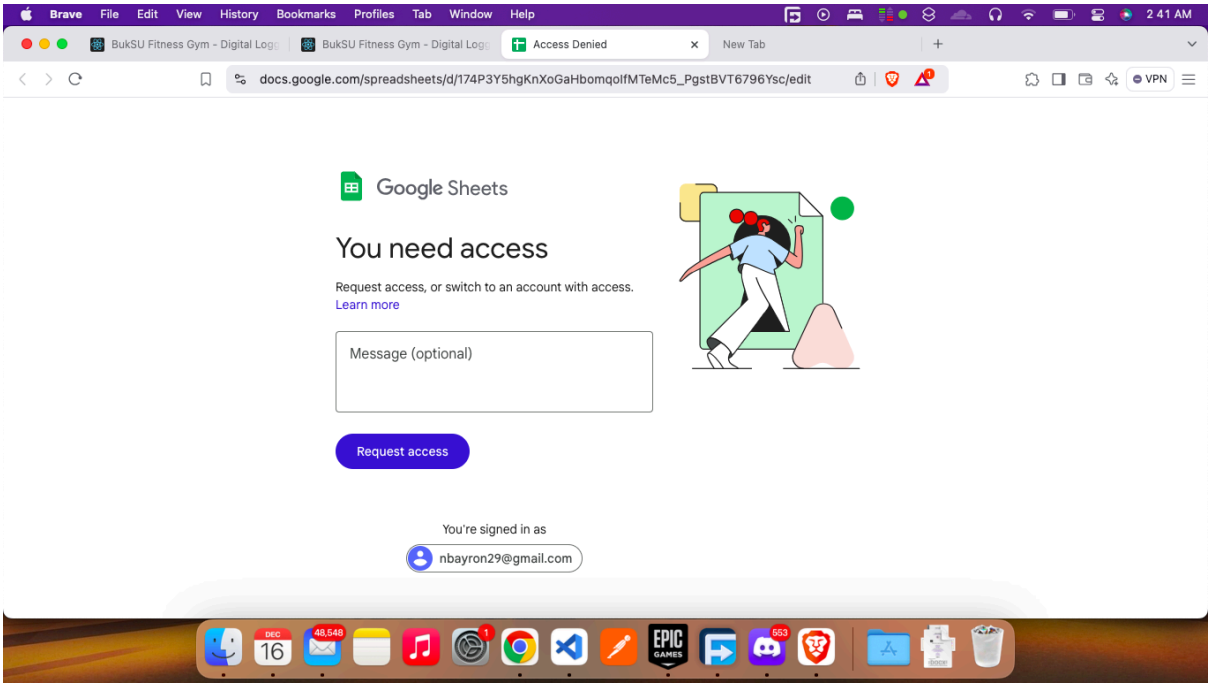
Screenshot 16: Edit Member Information

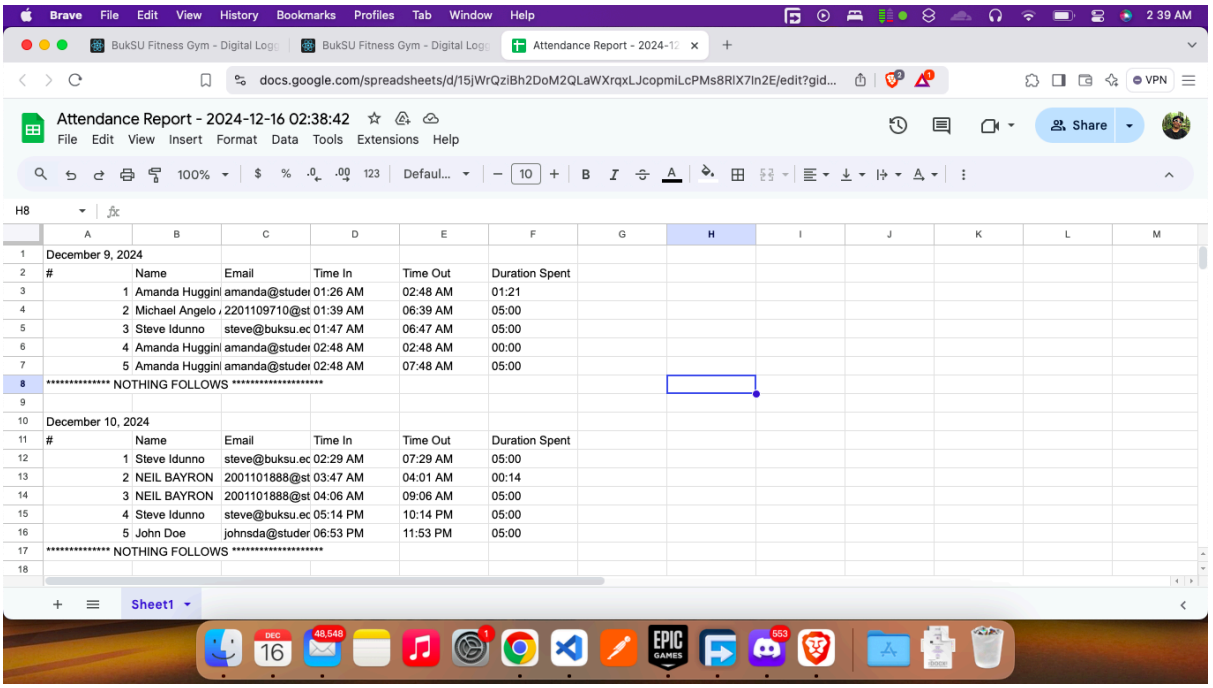
This screenshot shows the Edit Information page in the BukSU Fitness Gym Admin Panel. It allows admins to update a member's First Name, Last Name, and Institutional Email in case of any errors. The page features input fields for editing details and buttons for "Go Back" or "Continue" to save changes, ensuring accurate member information management.



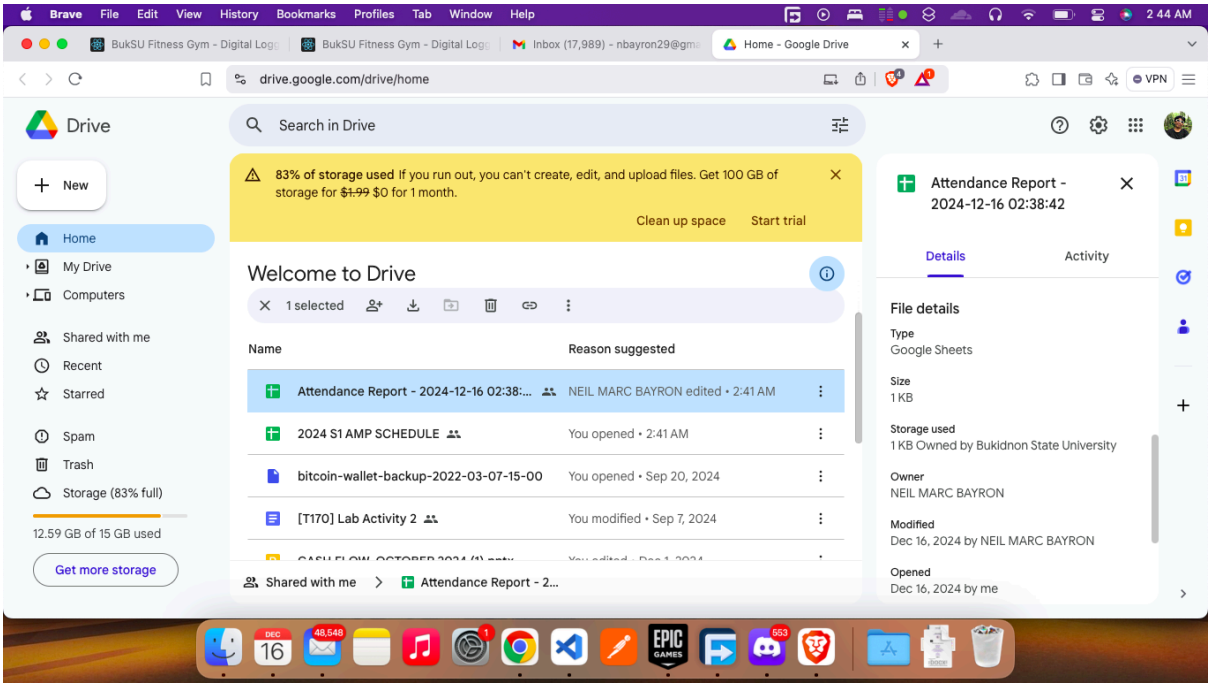
Screenshot 17: Attendance Report Generator

This screenshot displays the Attendance Report Generator page in the BukSU Fitness Gym Admin Panel. Admins can select a range (Daily, weekly, monthly, or custom date range) and provide an email address to send the report. After clicking Generate Report, a confirmation message indicates that the report was generated successfully. A View Report button allows the admin to review the generated report. This feature simplifies attendance tracking and report management for the gym.





	A	B	C	D	E	F
1	December 9, 2024					
2	#	Name	Email	Time In	Time Out	Duration Spent
3	1	Amanda Huggin	amanda@studer	01:26 AM	02:48 AM	01:21
4	2	Michael Angelo	2201109710@st	01:39 AM	06:39 AM	05:00
5	3	Steve Idunno	steve@buksu.ec	01:47 AM	06:47 AM	05:00
6	4	Amanda Huggin	amanda@studer	02:48 AM	02:48 AM	00:00
7	5	Amanda Huggin	amanda@studer	02:48 AM	07:48 AM	05:00
8	********** NOTHING FOLLOWS **********					
9						
10	December 10, 2024					
11	#	Name	Email	Time In	Time Out	Duration Spent
12	1	Steve Idunno	steve@buksu.ec	02:29 AM	07:29 AM	05:00
13	2	NEIL BAYRON	2001101888@st	03:47 AM	04:01 AM	00:14
14	3	NEIL BAYRON	2001101888@st	04:06 AM	09:06 AM	05:00
15	4	Steve Idunno	steve@buksu.ec	05:14 PM	10:14 PM	05:00
16	5	John Doe	johnsda@studer	06:53 PM	11:53 PM	05:00
17	********** NOTHING FOLLOWS **********					
18						



Screenshot 18, 19, & 20: Attendance Report and how it looks.

These screenshots relate to the Attendance Report feature of the BukSU Fitness Gym Digital Logging System. The first screenshot shows a Google Sheets Access Denied page, due to sensitive information only admins can see the file. The second screenshot displays an Attendance Report spreadsheet with recorded visit details, including Name, Email, Time In, Time Out, and Duration Spent for each visit. The third screenshot shows that the file is successfully saved to Google Drive. The report organizes entries by date and ensures tracking of gym usage, offering admins clear insights into attendance patterns.

Appendix J: Stakeholder Validation Documents

BUKIDNON
STATE UNIVERSITY
MALAYBALAY CITY, BUKIDNON

Fortich Street, Malaybalay City
Bukidnon 8700
Tel (088) 8013-5661 to 5663; TeleFax (088) 813-2717
www.buksu.edu.ph
buksupressoffice@buksu.edu.ph

Letter of Partnership

Cristitito P. Ornopia

Physical Education Department

Fortich Street, Malaybalay City, Bukidnon 8700

Dear Cristitito P. Ornopia,

This letter serves as a formal agreement of partnership between **Ctrl+Alt+Elite** and **Cristitito P. Ornopia** for the development of a web-based system to address the efficiency of logging users to the Fitness Gym.

Ctrl+Alt+Elite shall develop a web-based system that allows for the seamless digital logging management system. The system shall include features such as:

- **Landing Page**: Simple page that lists out Gym Information and Availability. Gym members can sign up here and request for biometric recording.
- **Login through Biometrics:** fitness gym members can login to use the fitness gym through biometrics
- **Visit History:** Users can see the gym visit history.
- **User Management:** Admins can easily manage add or archive members in the admin panel and register fitness gym biometric data
- **Report Generation:** Admins can seamlessly print out Gym Visitor Logs. They can also filter data results.

The project shall commence in September 2024 and be completed and deployed by April 2025.

Both parties shall maintain the confidentiality of any proprietary information disclosed to each other in connection with this project.

Either party may terminate this agreement upon material breach of its obligations under this agreement or upon the occurrence of an event that makes it impossible or impracticable for either party to perform its obligations under this agreement.

In witness whereof, the parties have executed this Letter of Partnership on this 16th day of September 2024.

Prepared By:

Neil Marc C. Bayron
2001101888@student.buksu.edu.ph

Rienier Mari Borong
2201104929@student.buksu.edu.ph

Ivan Lord Panoril
2201101564@student.buksu.edu.ph

Sergay Patricio
2201105765@student.buksu.edu.phl

Approved By:

Czarissa Louise A. Navidad
czarissanavidad@buksu.edu.ph
Instructor, Systems Integration & Architecture 1

Klevie Jun R. Caseres
kleviecaseres@buksu.edu.ph
Instructor, Integrative Programming & Technologies

Gil Nicholas T. Cagande
gilcagande@buksu.edu.ph
Instructor, Advanced Database Systems

Joseph L. Abella
josephabella@buksu.edu.ph
Instructor, Information Assurance & Security 1

Accepted By:

Cristitito B. Ornopia
cristititoornopia@buksu.edu.ph
Associate Professor I
Bachelor of Physical Education, Major in Schools Physical Education